

MSA 성능 튜닝 가이드

for developers

김용욱, 삼성SDS

소개

마이크로서비스 적용을 가이드하고 함께 개발하고 있습니다.



- 시스템에 적합한지 판단
- 목표에 맞게 서비스 설계
- 가이드 및 함께 개발



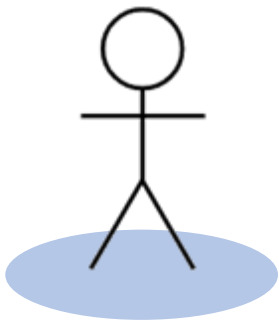
스프링캠프
MSA 개발 가이드 (2024)
MSA 트랜잭션 개발 가이드 (2025)

MSA 성능 튜닝 가이드

1. 성능 튜닝 개요
2. MSA의 성능 튜닝
3. 케이스 스터디
 - ① 비효율적인 구조
 - ② 비효율적인 코드
 - ③ 같은 데이터를 반복해서 조회

대상 청중

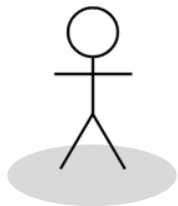
개발자



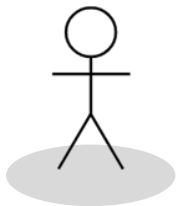
주니어

MSA가 낫선 시니어

SW
아키텍트



테크니컬
아키텍트



* 이야기할 내용

- 어플리케이션 코드 레벨 팁
- 데이터 캐싱 & 복제

* 안할 내용

- 인프라/시스템 구성
- 인프라 레벨의 스케일 아웃/인 등

몸에도 좋고 맛도 좋은 복어

복어의 효능

天界의 玉餐

중국 송나라의 시성 소동파가 "사람이 한번 죽는 것과 맛먹는 맛"이라며 "천계의 옥찬"이라 극찬했을 정도로 맛이 뛰어난 재료입니다.



영양 가득한 식재료

항암 예방 효과

함유되어있는 폴리펩티드 추출물의 암세포를 억제하고 항암 효과를 가지고 있습니다.

피로회복 효과

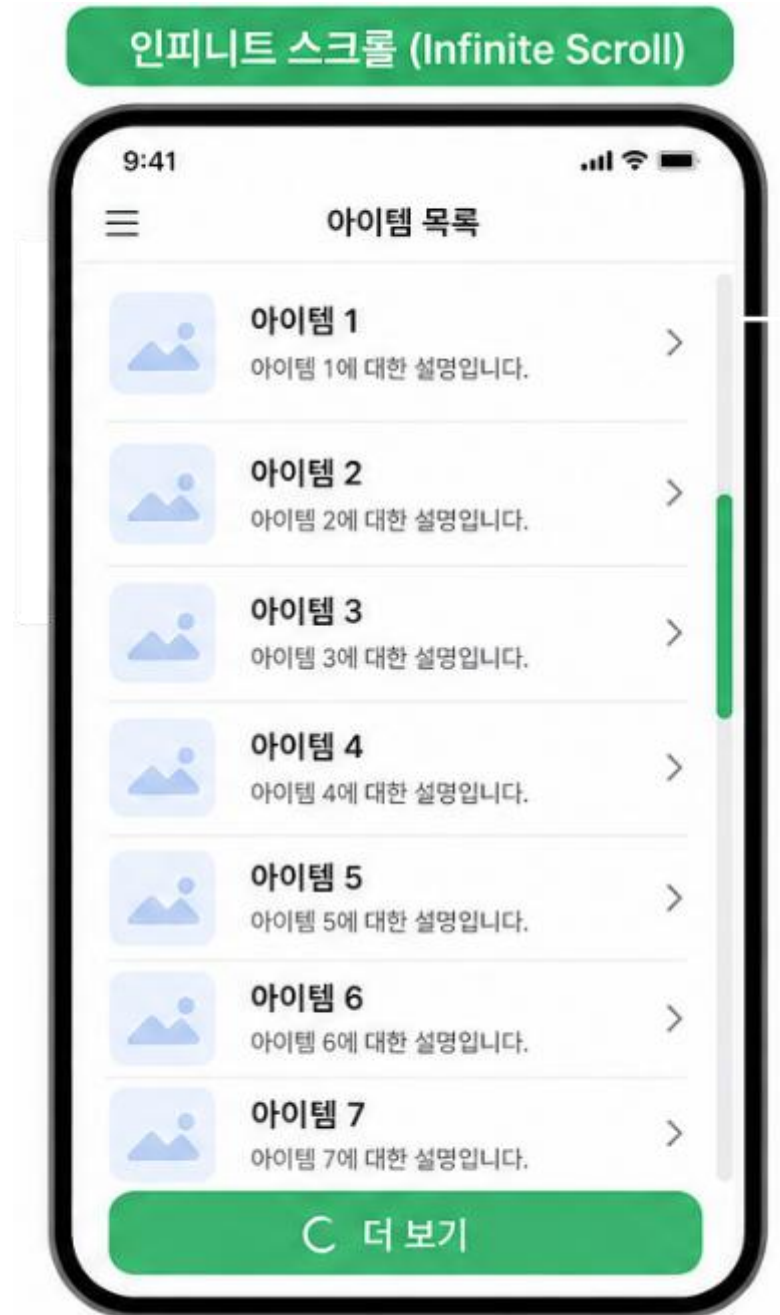
복어에 남은 일부독이 체내열을 만들어 피로를 회복시키는 것에 매우 좋다고 합니다.

숙취해소 효과

아세트알데히드의 분해를 촉진해 손상된 간의 빠른 회복에 효과적입니다.



웹 시스템에서 페이징은 기본입니다.
하지만 데이터 정합성이 안맞을 수도 있습니다.



1. 성능 튜닝 개요

성능이란?

요청을 처리하는 시간과, 단위 시간 동안 동시에 처리하는 양을 통칭

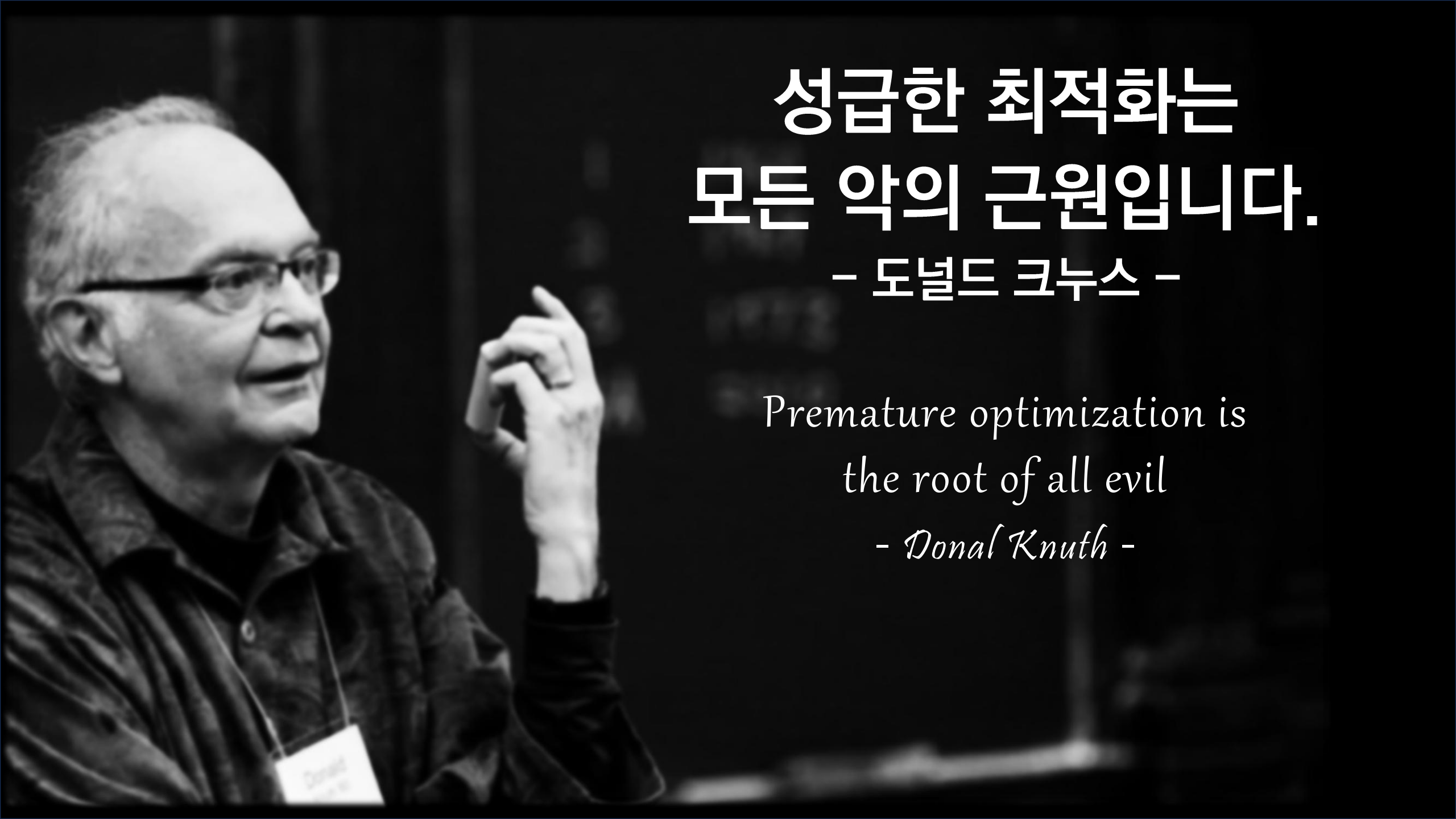
1. 처리 시간(Latency): 한 요청이 시작부터 완료까지 걸리는 시간

2. 처리 량(Throughput): 단위 시간에 처리 가능한 요청 수

- * TPS (Transactions Per Second)

- * RPS (Requests Per Second)

* 성능 튜닝: 주어진 리소스에서 더 좋은 성능을 내도록 개선하는 것



성급한 최적화는
모든 악의 근원입니다.

- 도널드 크누스 -

Premature optimization is
the root of all evil

- Donal Knuth -

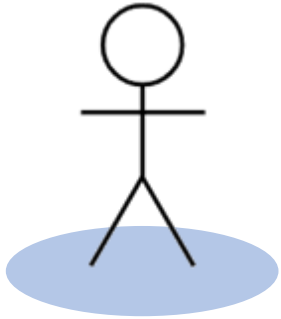




2. MSA 성능 튜닝

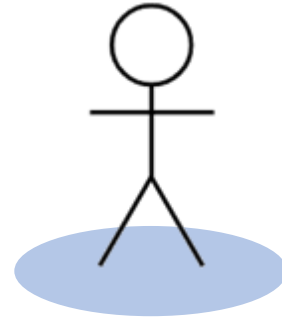
for developers

MSA라고 개발할 때 다르게 있나?



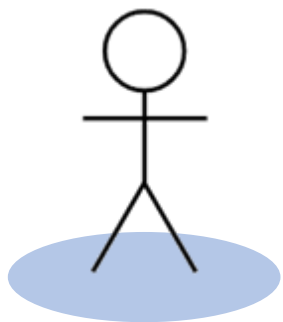
개발자

서비스가 API 호출을 많이하지



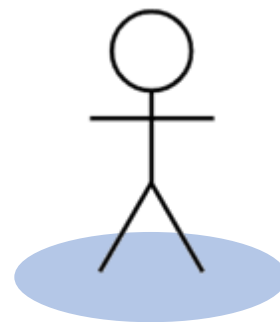
개발자

API 호출은 기존에도
했는데?



개발자

서비스가 API를 많이
호출하면 새로운 문제가
생길 수 있지

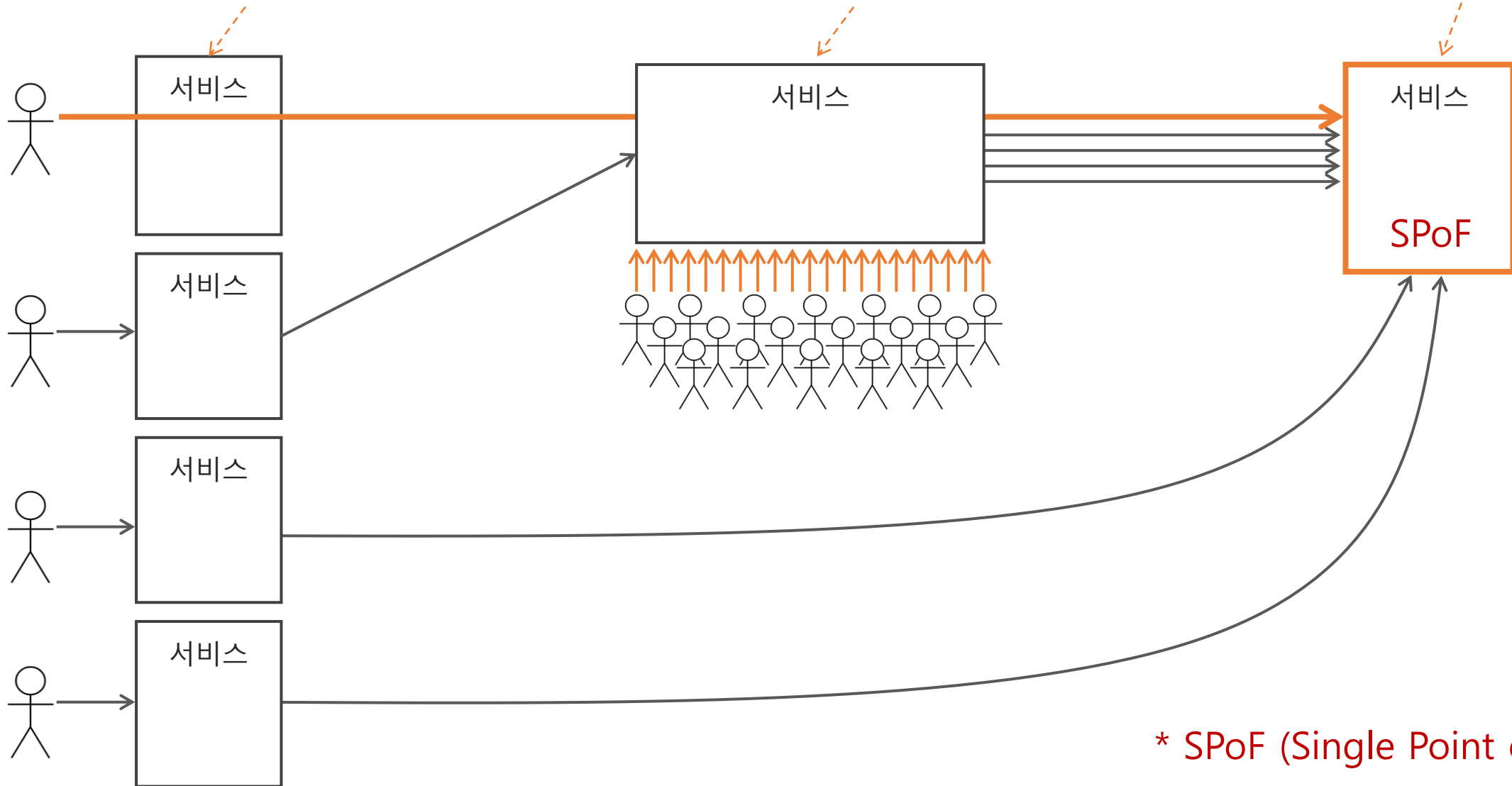


개발자

① 긴 API 호출 체인으로
처리 속도 저하

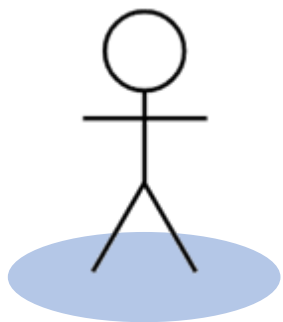
② 부하 증가로 느려지면
다른 서비스도 느려짐

③ 장애 시
전체 시스템 장애



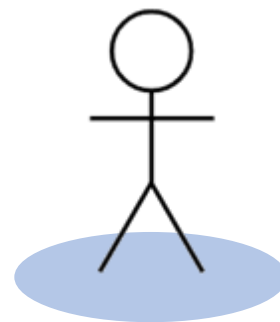
* SPoF (Single Point of Failure)

API 호출을 줄이려면
어떤 스킬이 필요하지?



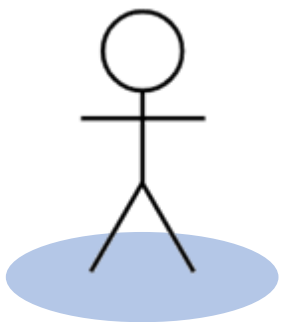
개발자

- ① 서비스 구조를 잘 설계하고
- ② 코드를 잘 작성하고
- ③ 필요할때만 API를 호출해야지

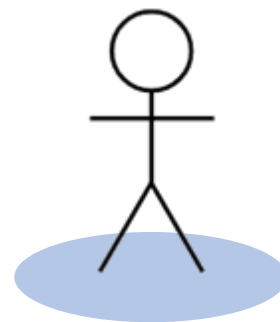


개발자

그리고 로컬 캐시말고
Redis 쓰면 안되?



개발자

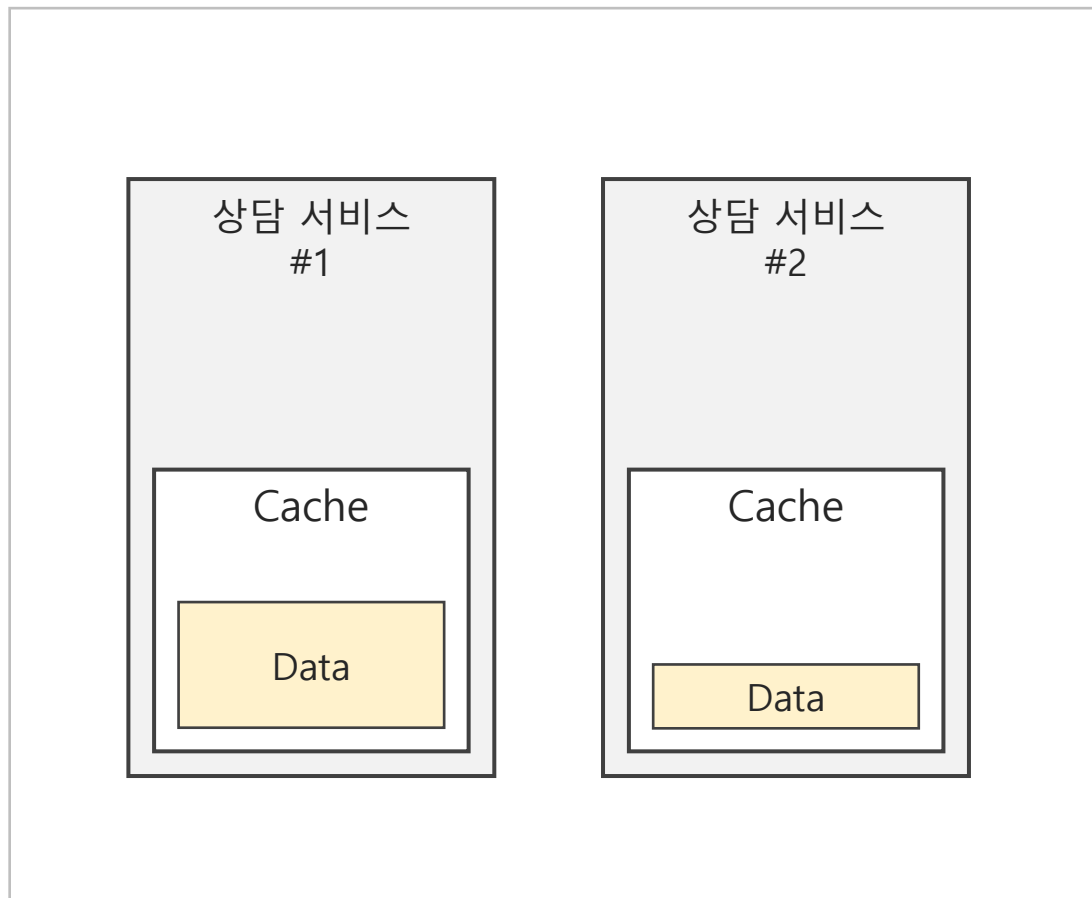


개발자

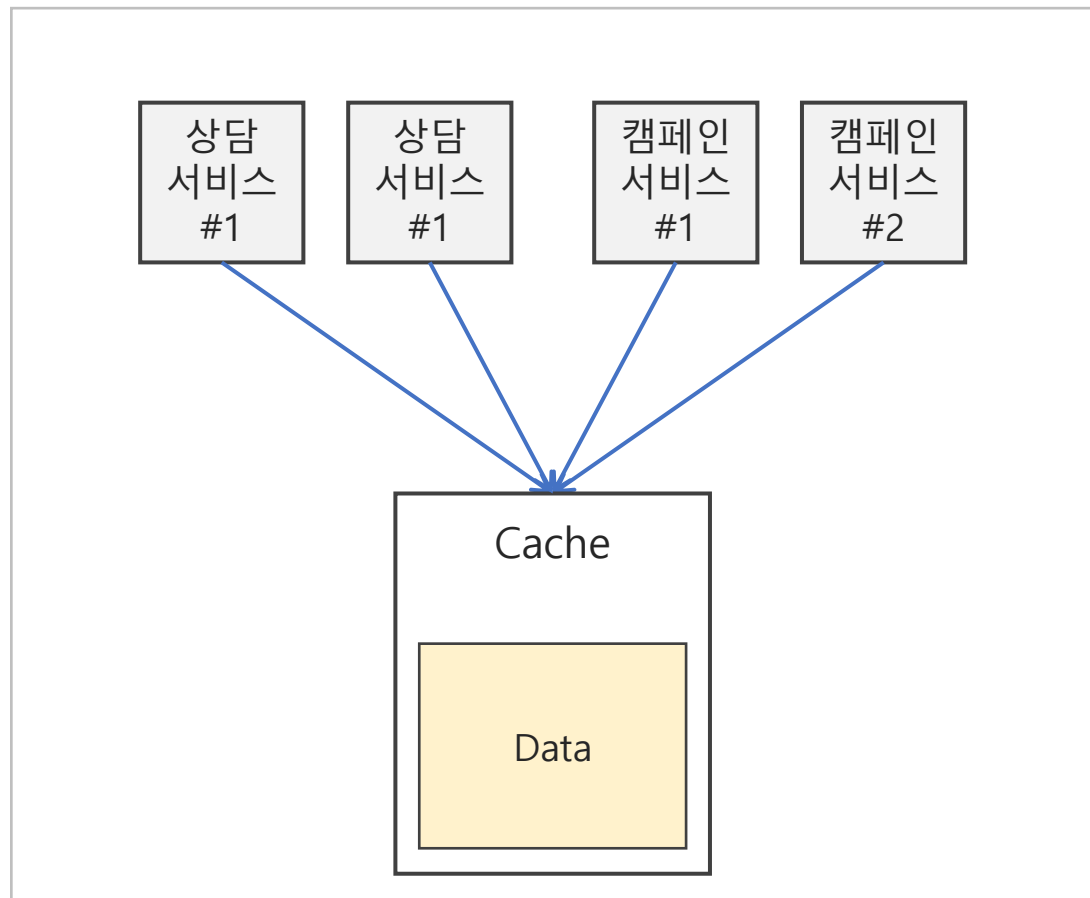
로컬 캐시 vs 글로벌 캐시

로컬 캐시는 메모리에 데이터를 저장하여 조회 속도가 매우 빠르지만 캐시 별로 데이터가 다를 수 있음
글로벌 캐시는 중앙 서버에 데이터를 저장하여 일관된 데이터를 빠르게 제공

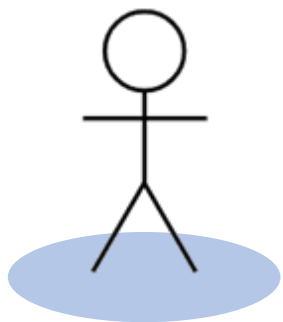
로컬 캐시



글로벌 캐시

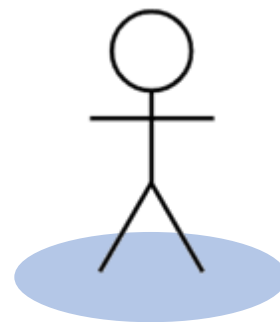


그리고 로컬 캐시 말고
Redis 쓰면 안되?

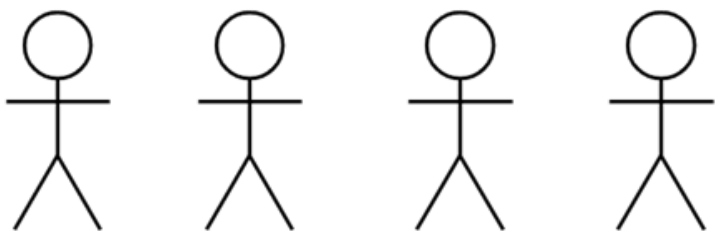


개발자

글쎄....
아닐걸?



개발자



튼튼하고 안전한
대형 버스



+

?

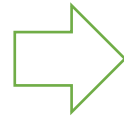
골목 길도 잘 갈 수 있는
민첩한 이동수단

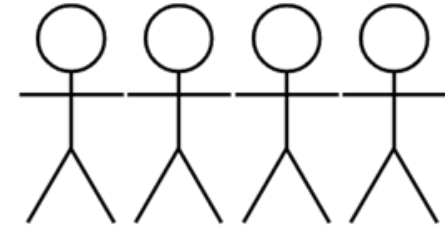
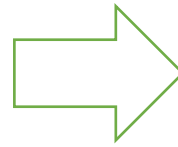
로컬 캐시



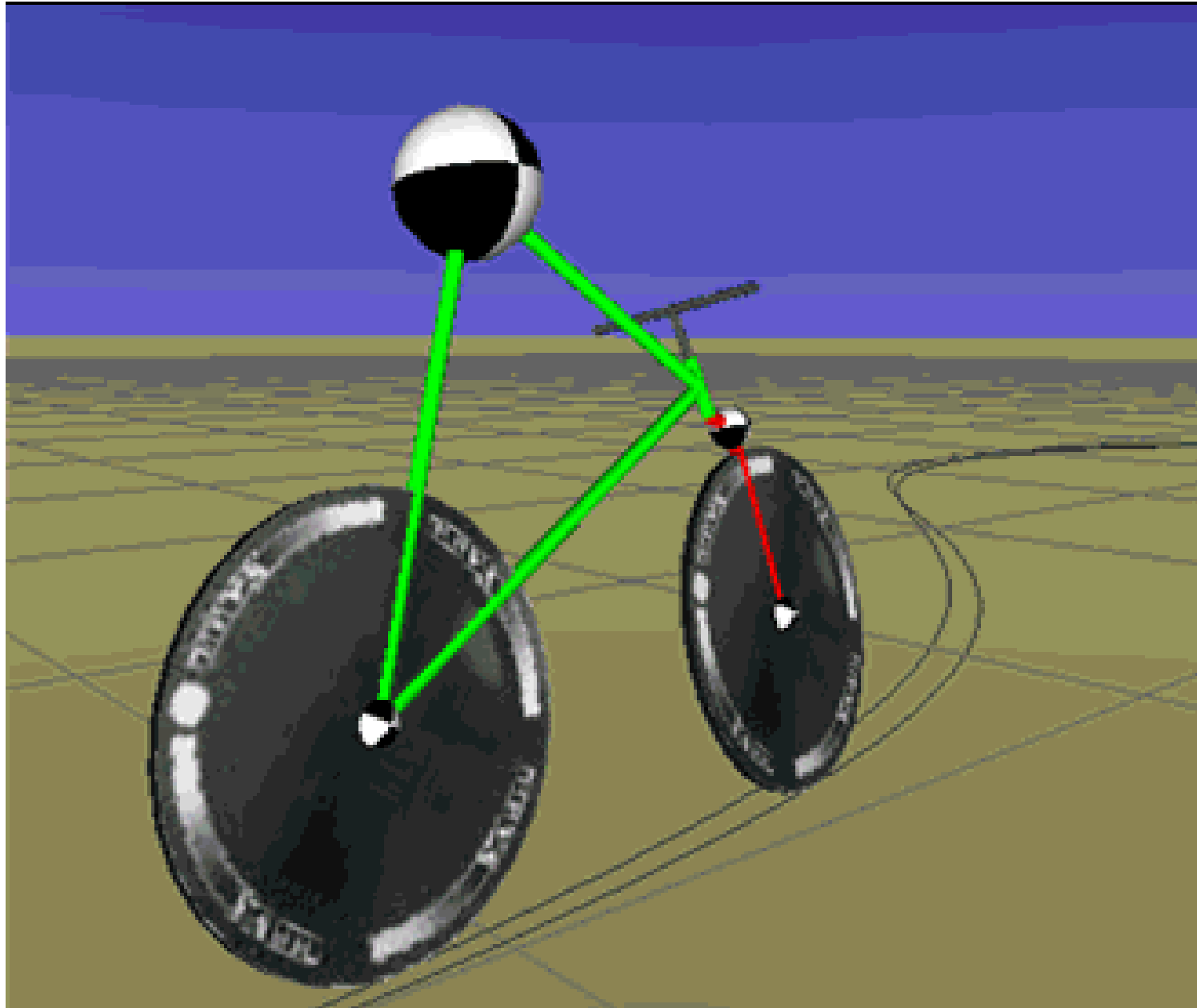
글로벌 캐시



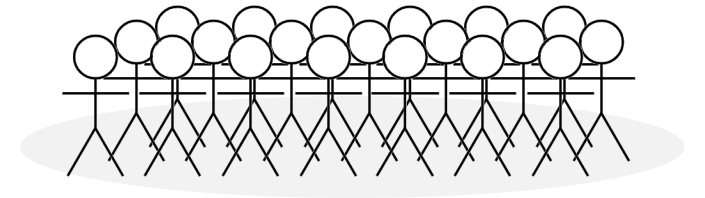




* SPoF (Single Point of Failure)

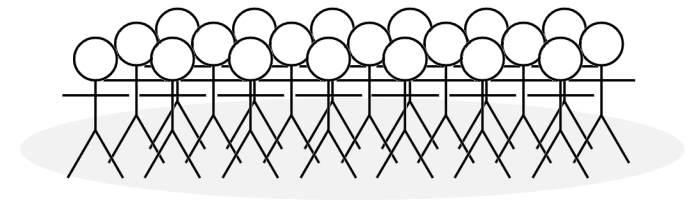


두 발인데
당연히 넘어지겠지!





보조 바퀴를 달면
되겠네!



성능을 대략 계산하려면 기준이 있어야 하는데...

이건... 나갈 돈

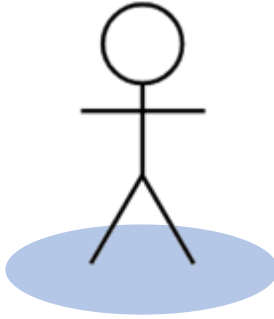


이건... 쓸 돈



절대적인 기준은 없습니다만...

하지만 어림잡아 계산할 수
있는 기준은 있어요...



시니어
개발자

경험적인 기준 ① 체감 속도

사용자 체감 속도

처리 시간	비고
< 0.1초	매우 빠름
< 0.3초	빠름
< 1초	느리지 않음
> 3초	느림

조회 건 별 DB 쿼리 속도

데이터 수	시간 (초)
1	0.001
100	0.01
1,000	0.04
3,000	0.1
10,000	0.4

경험적인 기준 ② 저장소 별 성능

간단한 쿼리의 실행 시간

요청 대상	처리 속도 (초)
타 서비스	0.01 (?)
DB	0.005
Redis	0.00005
로컬 캐시	0.000001
내부 메모리	< 0.000001

x 100배
x 20배

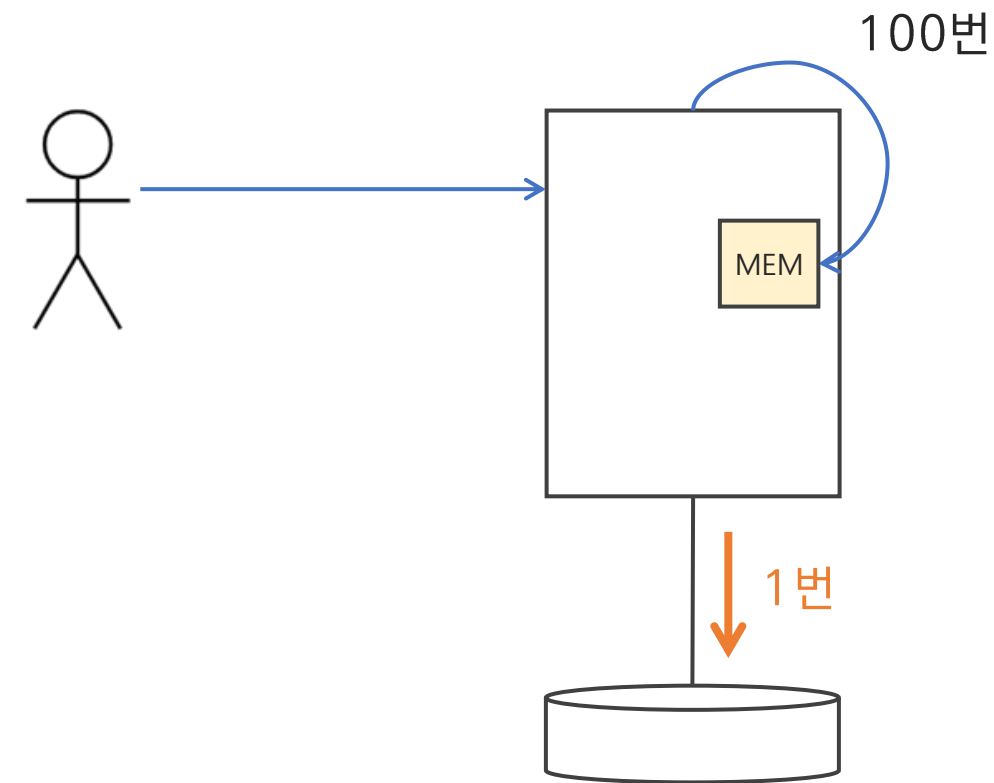
간단한 쿼리의 동시 성능

요청 대상	처리량 (RPS)
DB	1,000
Redis	100,000
로컬 캐시	2,000,000

※ 한 테이블 쿼리

* 세부 수치 보다는 상대적인 배수가 중요합니다

예시 - DB 쿼리



간단 DB 쿼리 1번 >>> 메모리 100번 참조

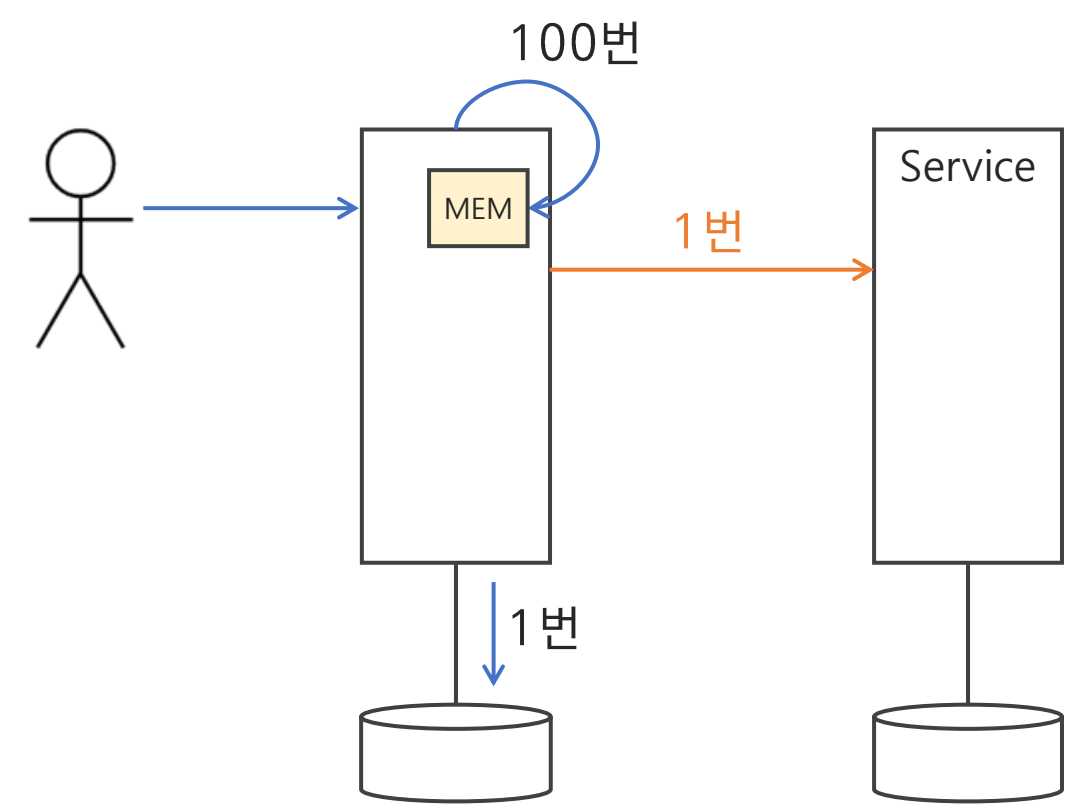
약 0.005초 (0.0051)
= DBx1 + 메모리x1백 (0.0051초)

요청 대상	호출 수	단위 속도	예상 시간
DB	1	0.005	0.005
내부 메모리	100	0.000001	0.0001

합계 0.0001

98%

예시 - DB 쿼리 + API 호출



타 서비스 API 1번 > 간단 DB 쿼리 1번

약 0.01초 (0.016)
= DBx1 + 타서비스 APIx1 + 메모리x1천

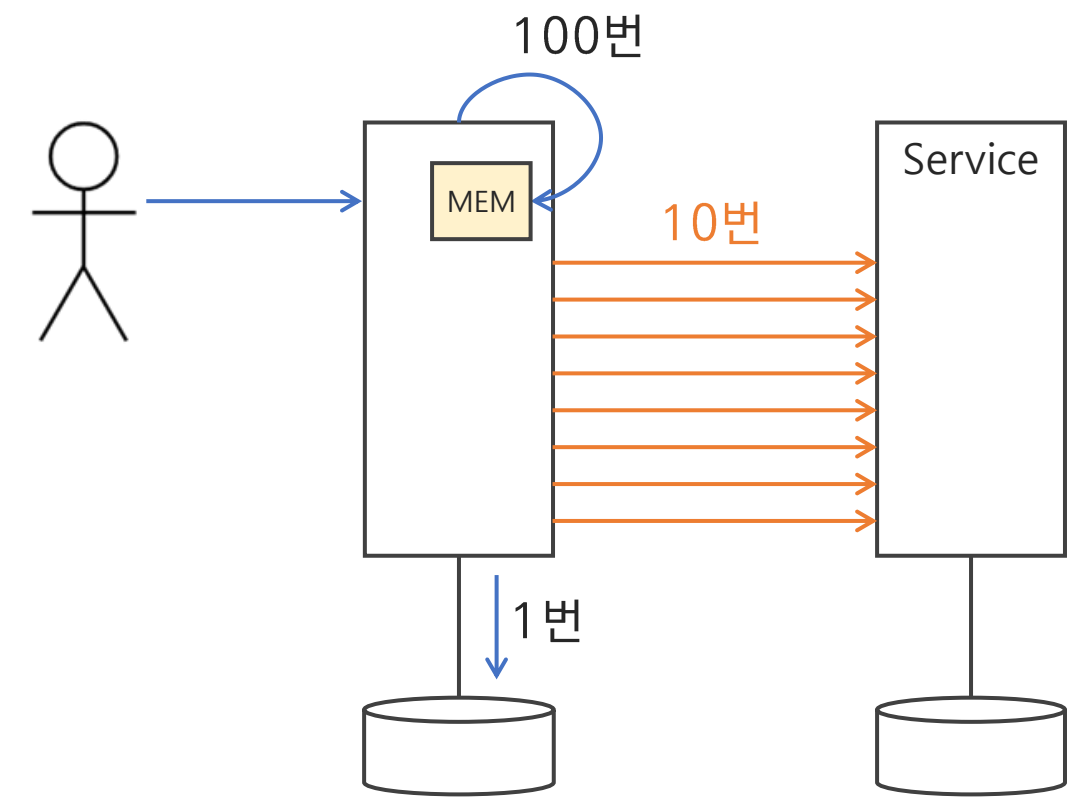
요청 대상	호출 수	단위 속도	예상 시간
타 서비스	1	0.01	0.01
DB	1	0.005	0.005
내부 메모리	1,000	0.000001	0.001

합계 0.016

63%

31%

예시 - DB 쿼리 + API 호출 여러 번



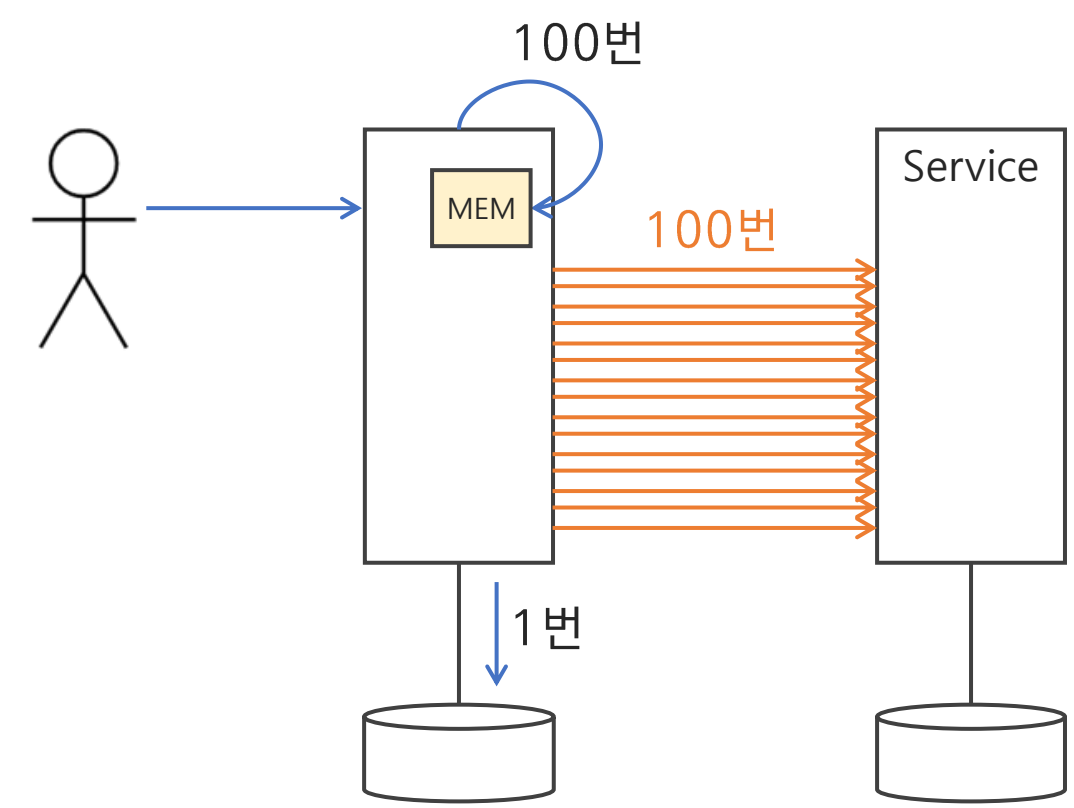
타 서비스 API 10번 >> 간단 DB 쿼리 1번

약 0.1초 (0.106)
= DBx1 + 타서비스 APIx10 + 메모리x1천

요청 대상	호출 수	단위 속도	예상 시간
타 서비스	10	0.01	0.1
DB	1	0.005	0.005
내부 메모리	1,000	0.000001	0.001
합계 0.106			

94%

예시 - DB 쿼리 + API 호출 많이 많이



타 서비스 API 10번 >>> 간단 DB 쿼리 1번

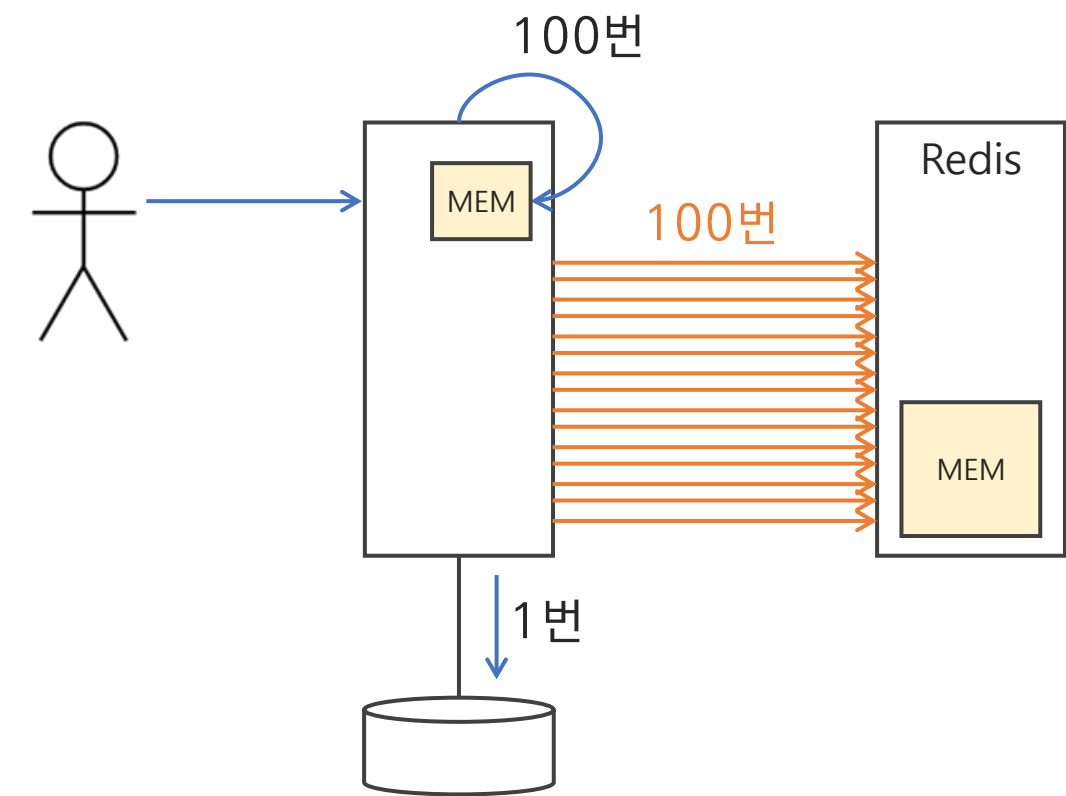
약 1초 (1.006)
= DBx1 + 타서비스 APIx100 + 메모리x1천

요청 대상	호출 수	단위 속도	예상 시간
타 서비스	100	0.01	1.0
DB	1	0.005	0.005
내부 메모리	1,000	0.000001	0.001

합계 1.006

99%

예시 - DB 쿼리 + Redis (글로벌 캐시)



약 0.01초 (0.011)
= DBx1 + Redisx100 + 메모리x1천

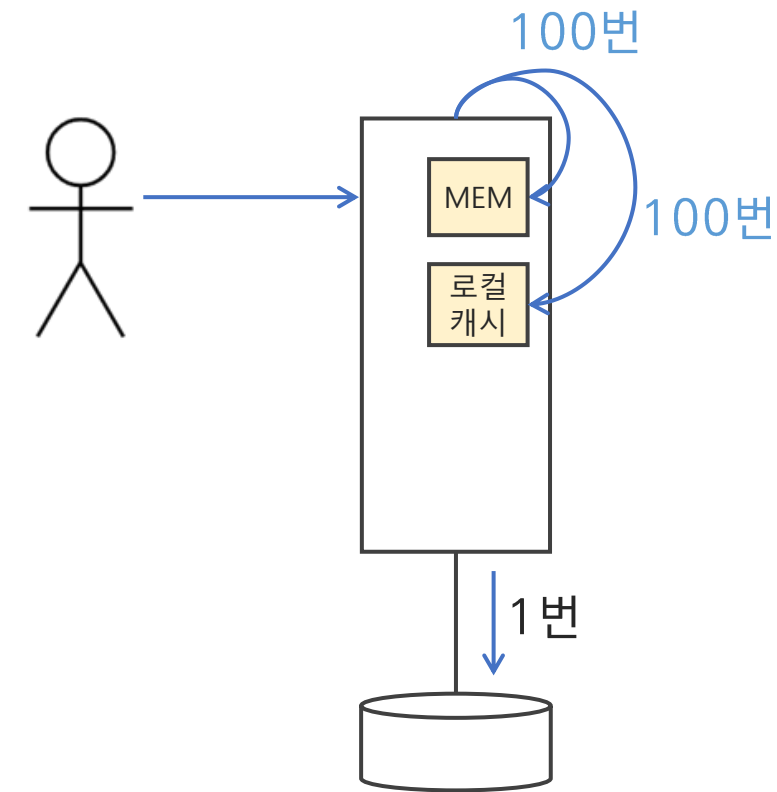
요청 대상	호출 수	단위 속도	예상 시간
Redis	100	0.00005	0.005
DB	1	0.005	0.005
내부 메모리	1,000	0.000001	0.001

합계 0.011

45%

45%

예시 - DB 쿼리 + 로컬 캐시 (내부 메모리)



약 0.006초 (0.0061)
= DBx1 + Redisx100 + 메모리x1천

요청 대상	호출 수	단위 속도	예상 시간
로컬 캐시	100	0.000001	0.0001
DB	1	0.005	0.005
내부 메모리	1,100	0.000001	0.001

합계 0.0061

2%
82%

MSA만의 성능 저하 원인

성능 저하 원인

인덱스 부재, 과도한 Join 등
긴 트랜잭션, 락 경합, 데드락

비효율적인 비즈니스 로직

비효율적인 API 호출

메모리 과다, GC

서비스 리소스 (쓰레드, 커넥션 풀 등)

서버 자원 (CPU, 메모리, DISK I/O)

네트워크 속도/대역 부족

원인 유형

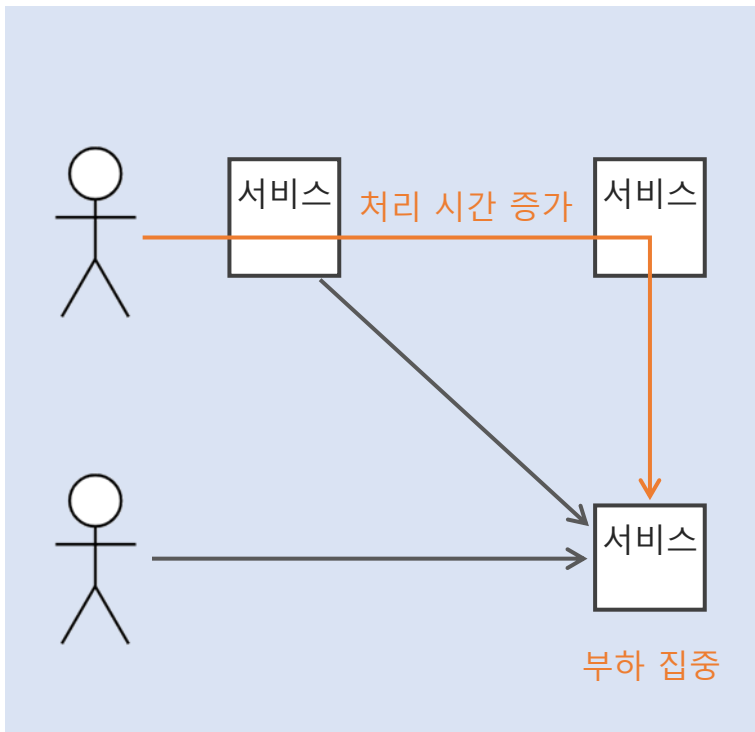
- ① 비효율적인 구조
- ② 비효율적인 코드
- ③ 같은 데이터 반복 조회

개선 방법

- ① 기능/데이터 재배치
- ② 데이터 복제&동기화
- ③ 알고리즘 개선
- ④ 캐시

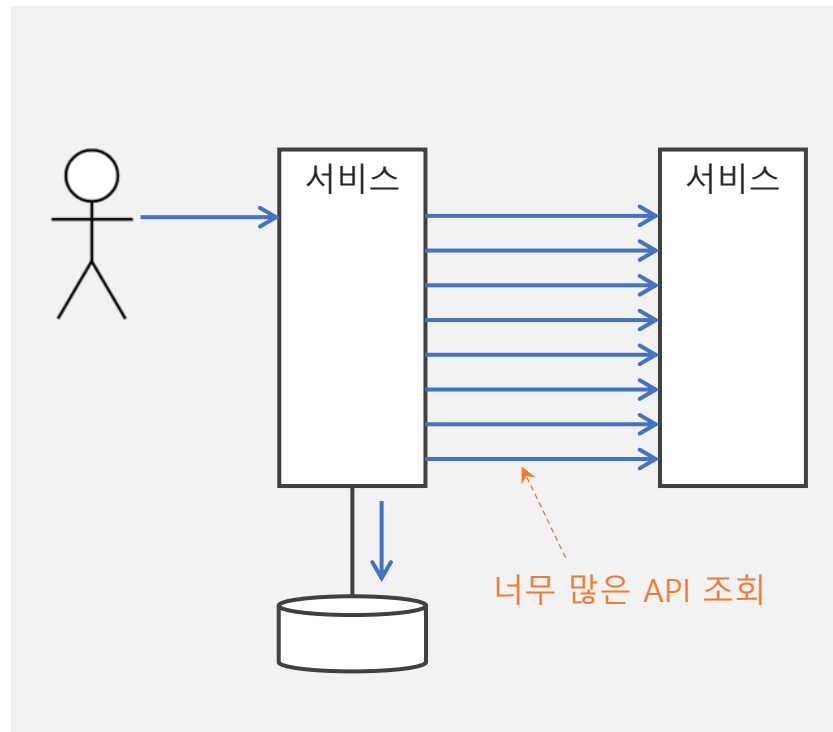
MSA의 성능 저하 유형

① 비효율적인 구조



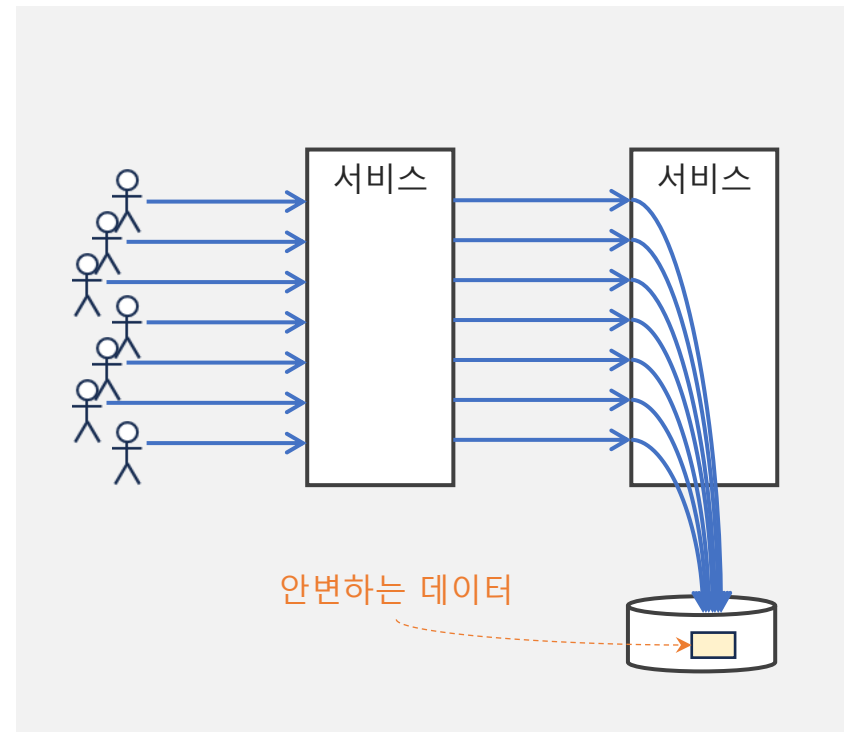
API 호출을 과하게 유발하는 구조

② 비효율적인 코드



코드가 API를 과도하게 호출

③ 같은 데이터를 반복해서 조회

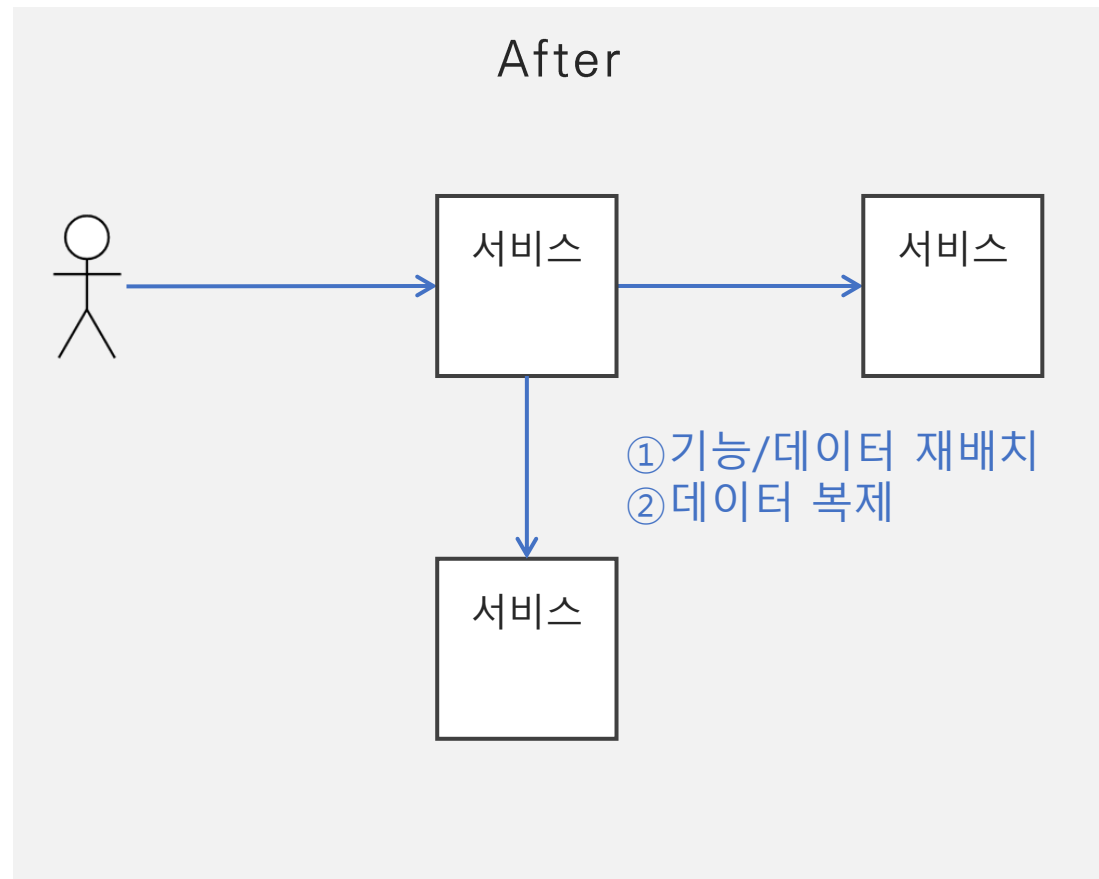
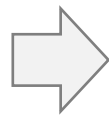
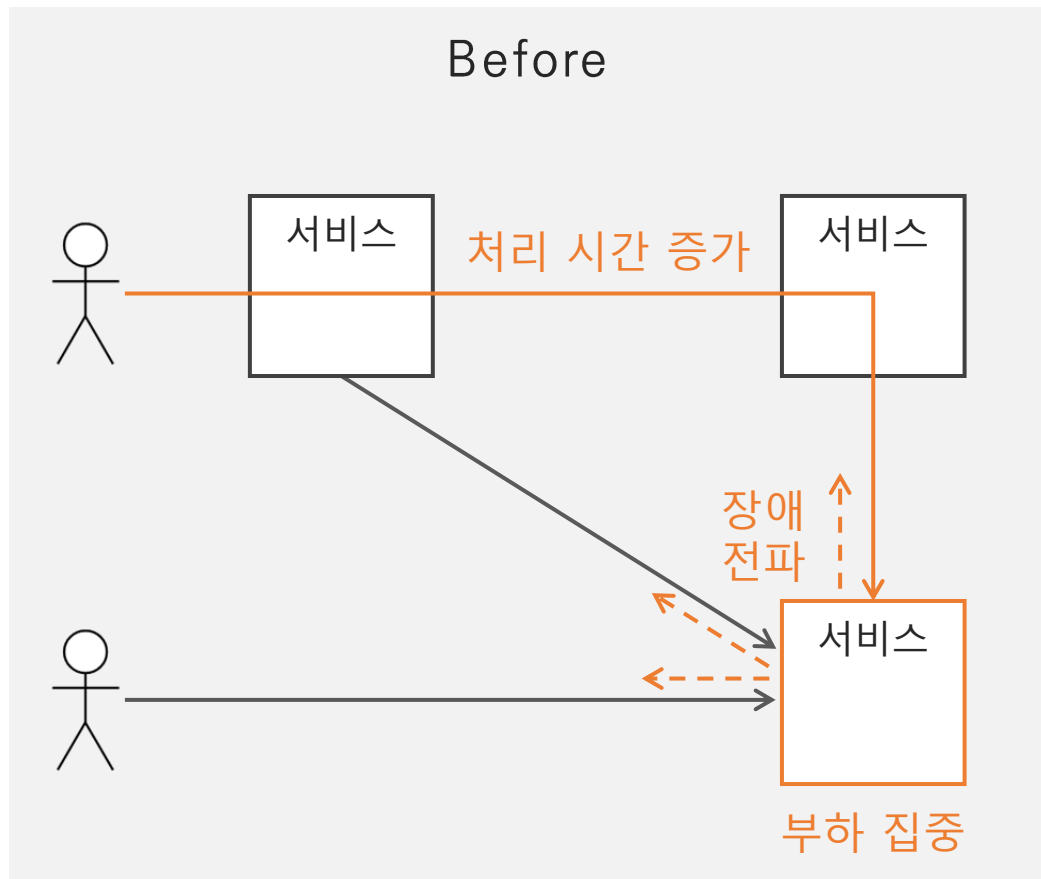


필요가 없는데 API 호출

3. Case Study

① 비효율적인 구조

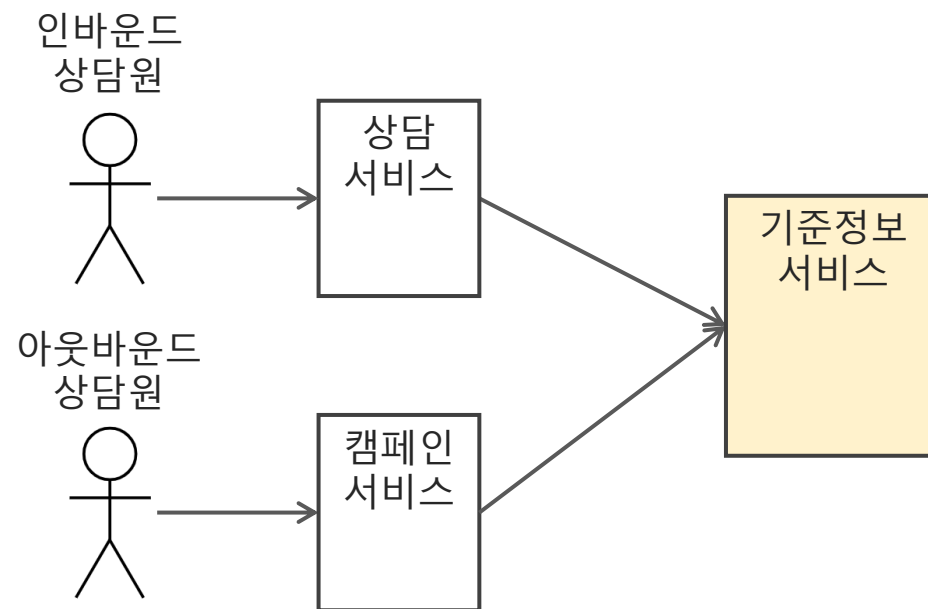
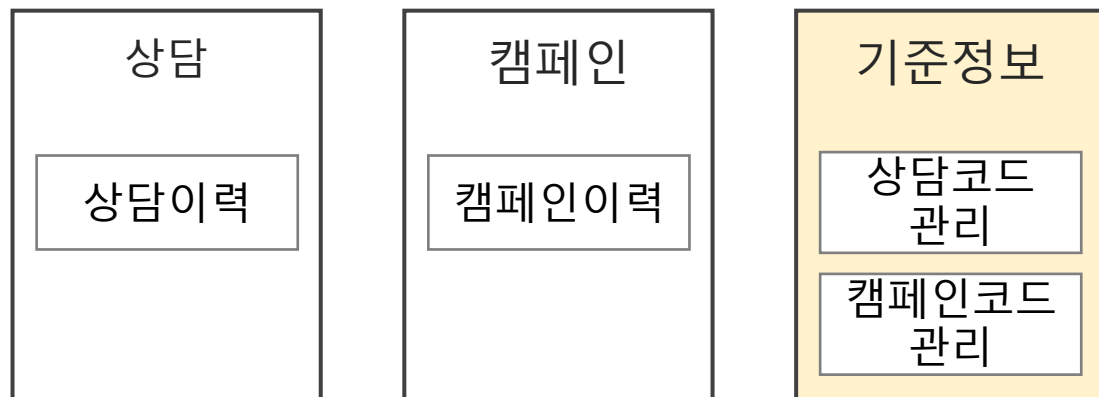
API 호출 체인이 긴 경우 처리 속도 및 처리량이 저하되고 시스템 안정성이 저하됨
서비스 정의 시점에 검토하여 구조를 변경하거나 데이터를 복제하여 호출 체인을 단축해야 함



I. 기준 정보 서비스

① 비효율적인 구조

Before



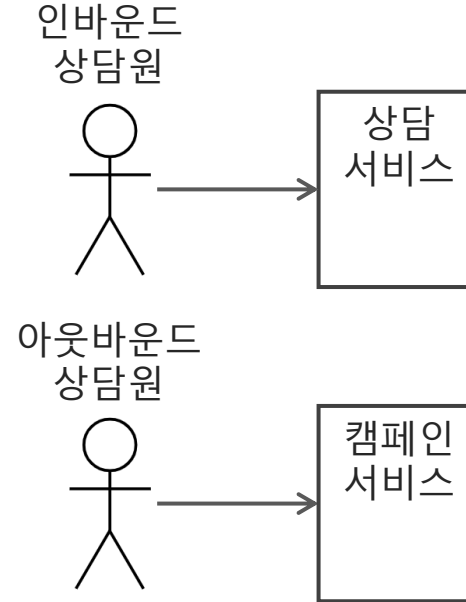
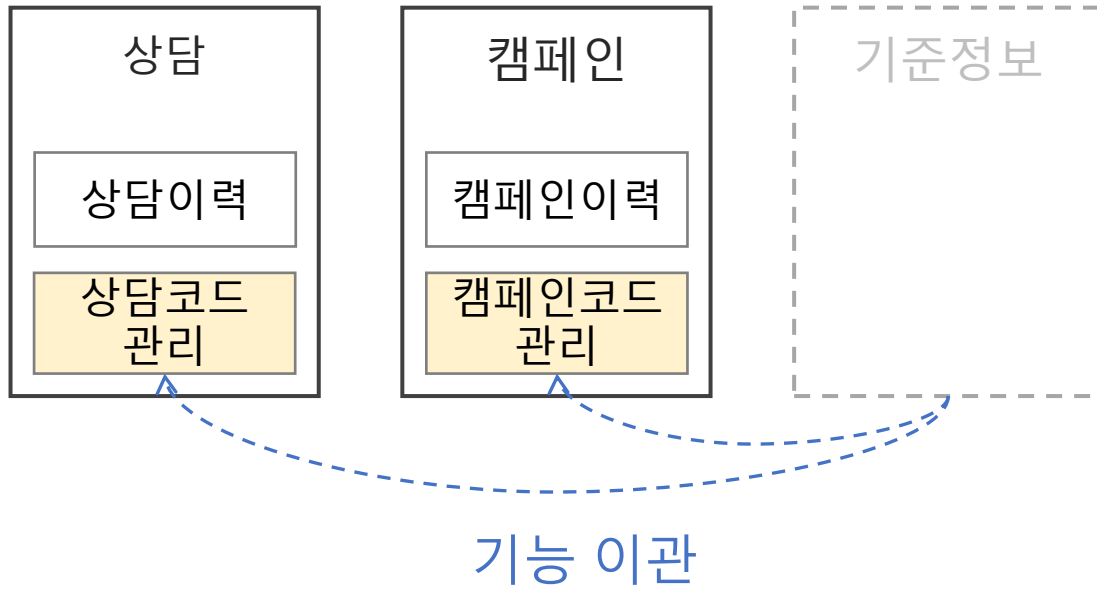
※ 기준정보 예시

- . 상품: 상품 카테고리, 품명, 규격 등
- . 공급 업체: 거래처코드, 업체명, 결제조건 등

I. 기준 정보 서비스 (cont.)

① 비효율적인 구조

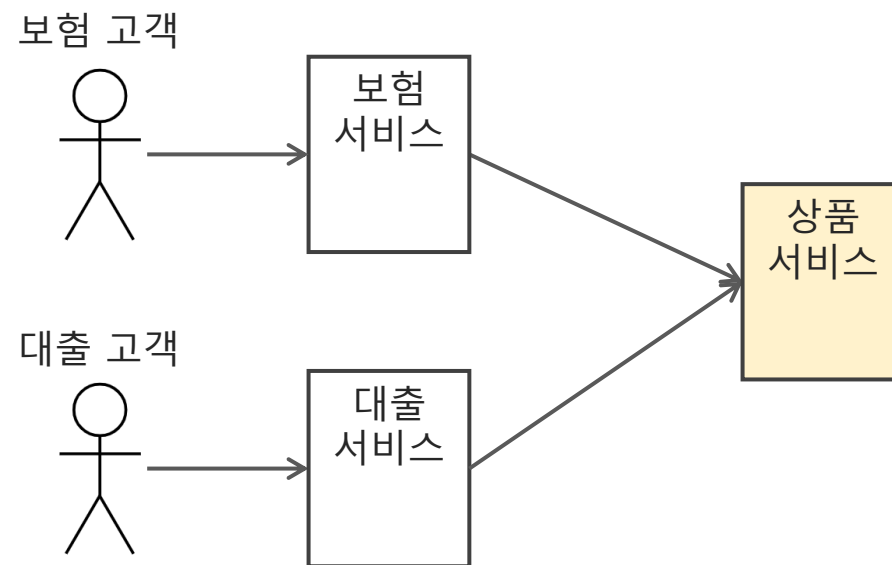
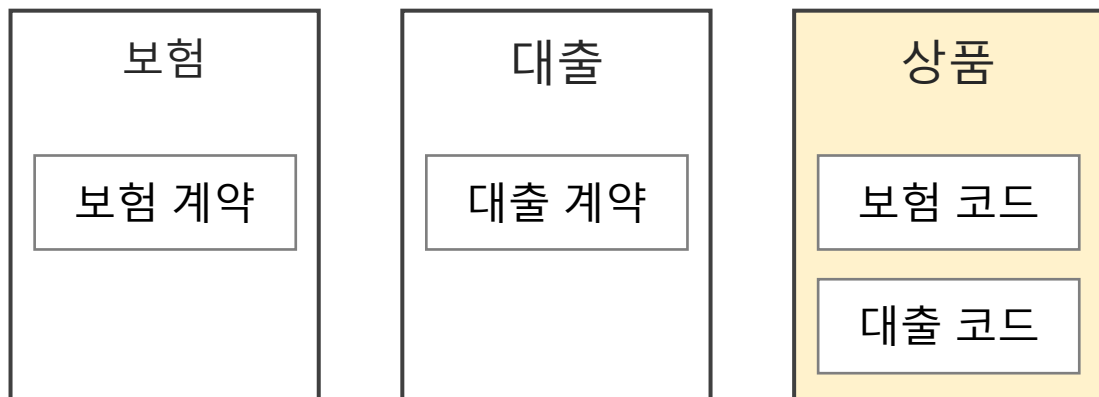
After



II. 마스터 정보 서비스

① 비효율적인 구조

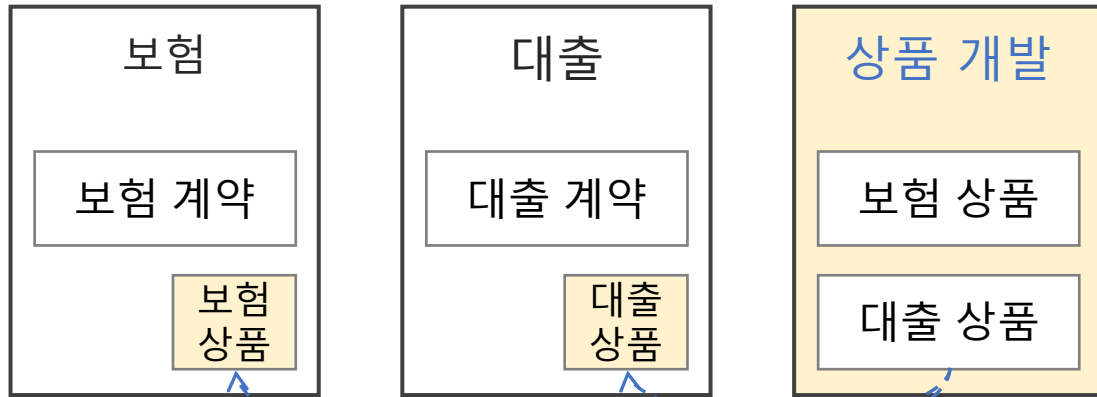
Before



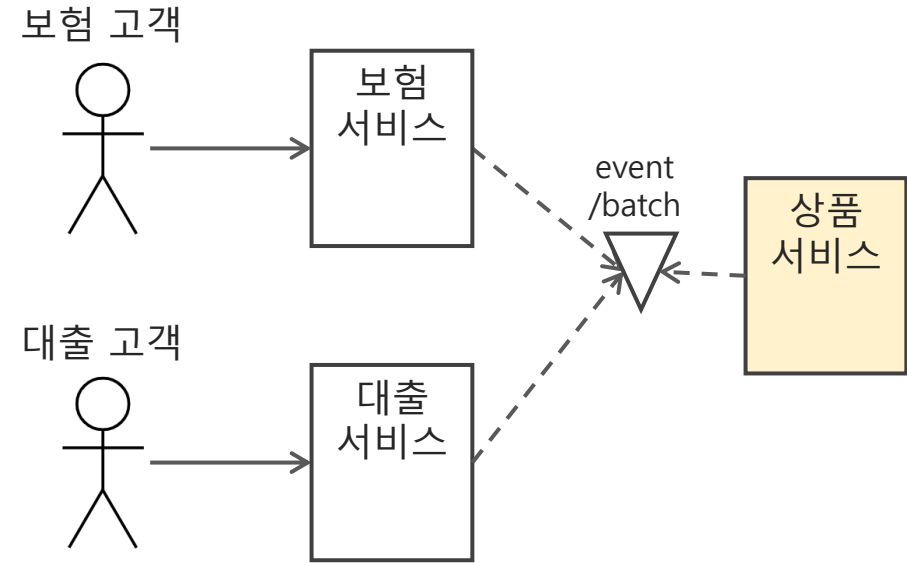
II. 마스터 정보 서비스 (cont.)

① 비효율적인 구조

After



상품 출시 시점에
ReadOnly 복제

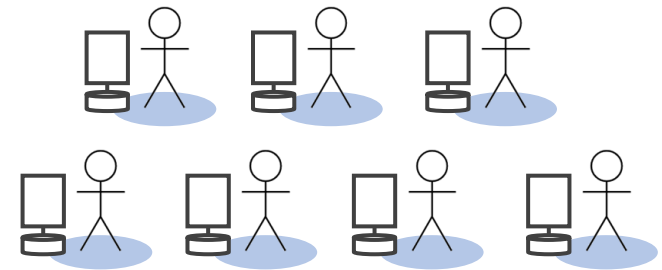


MSA에서 테이블 복제를 한다고?

데이터 복제 != 테이블 복제



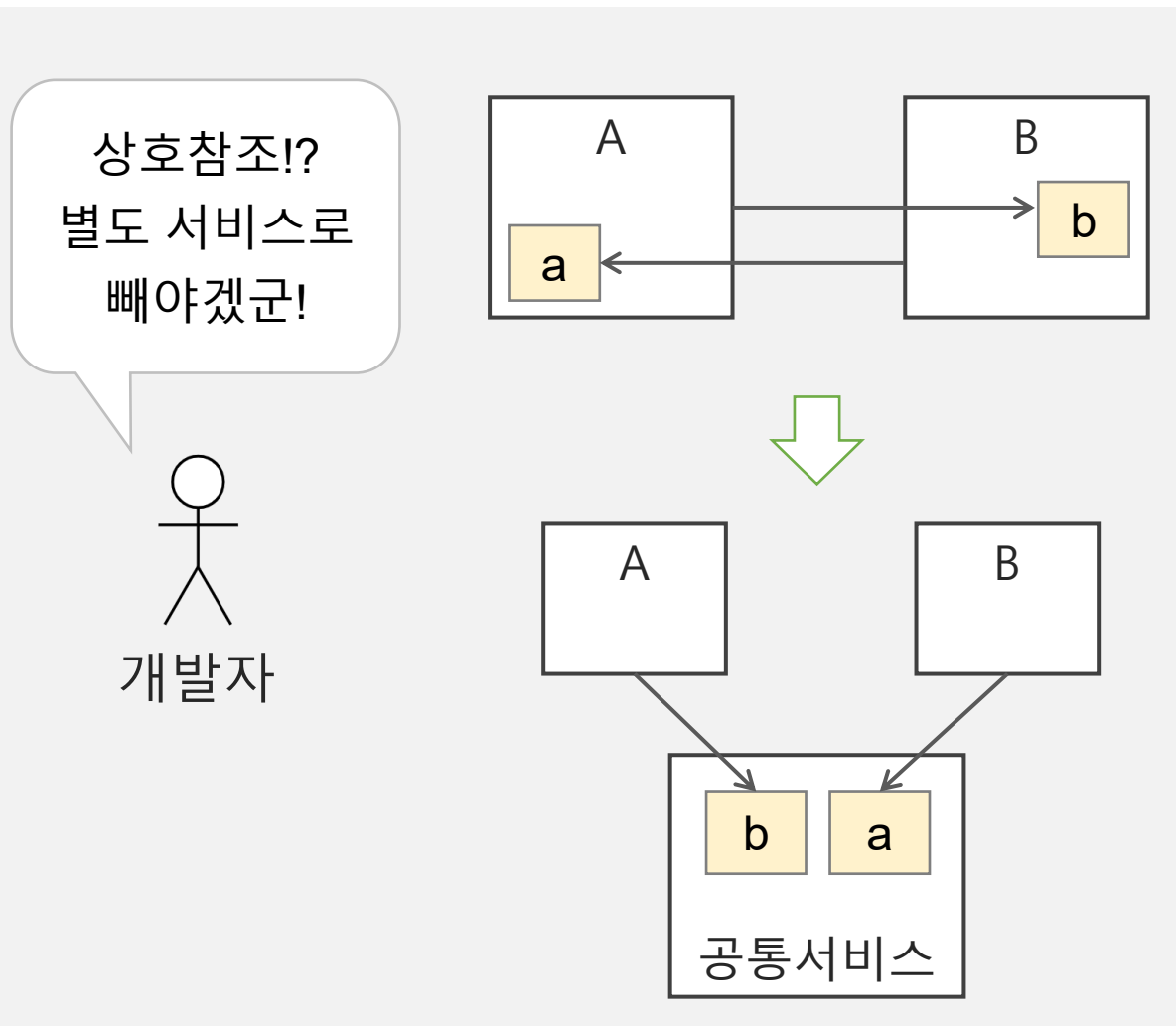
일요일 아침에
OGG 끊고 Table Alter합니다.
모든 팀에서 최소 1명씩
출근해서 대기하세요~



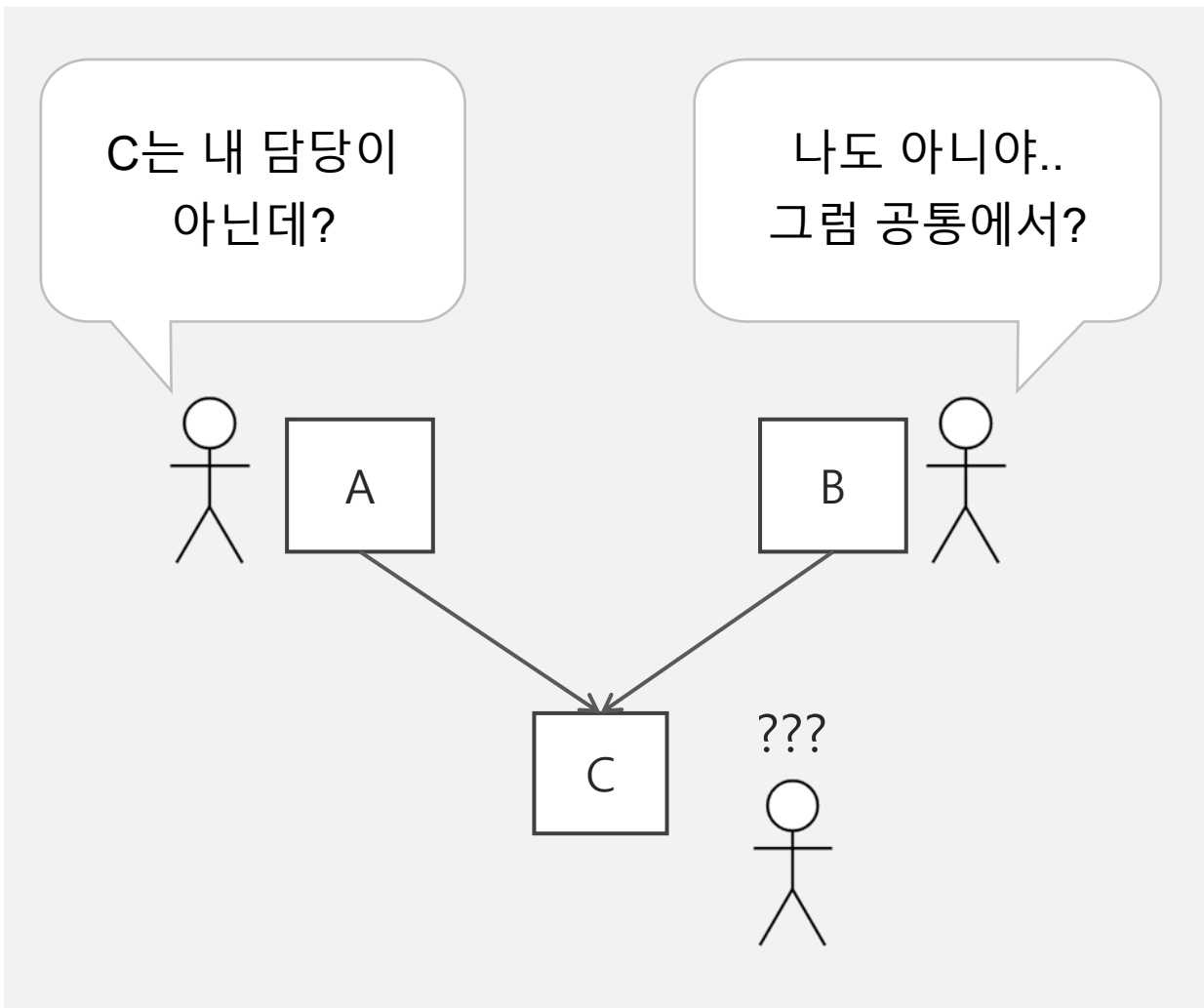
III. 공통 업무 서비스

① 비효율적인 구조

상호 참조

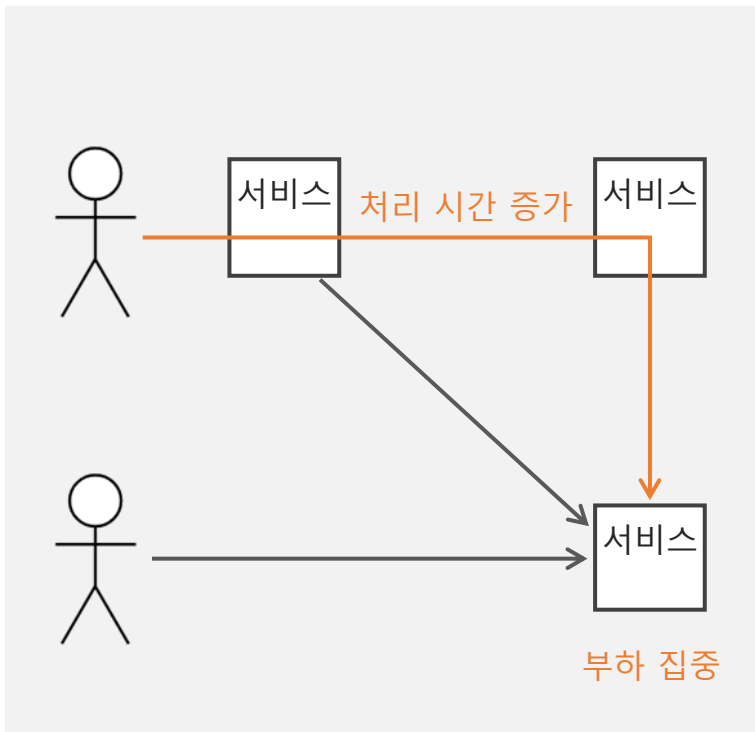


모두의 업무



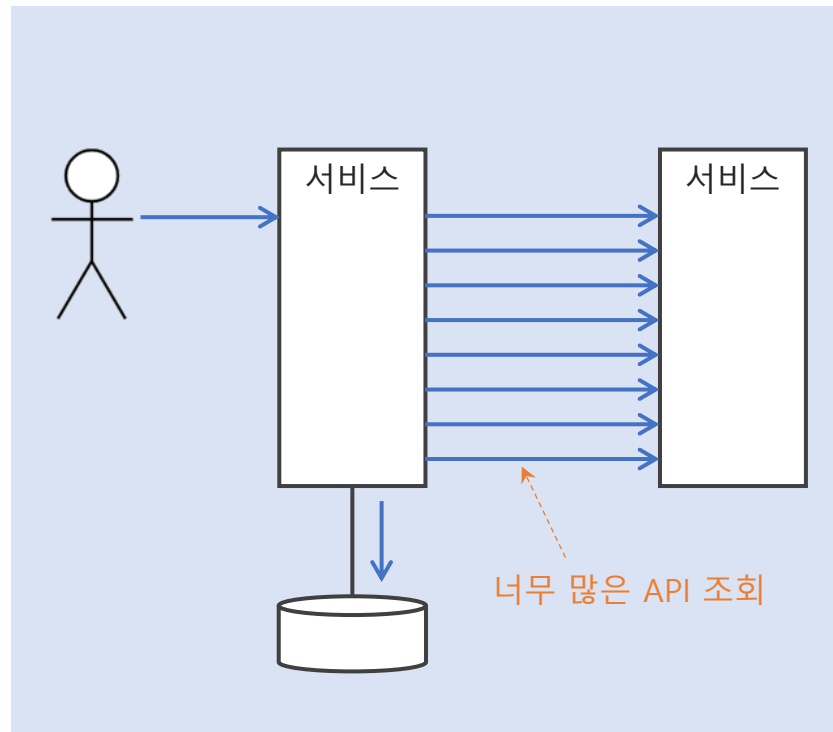
MSA의 성능 저하 유형

① 비효율적인 구조



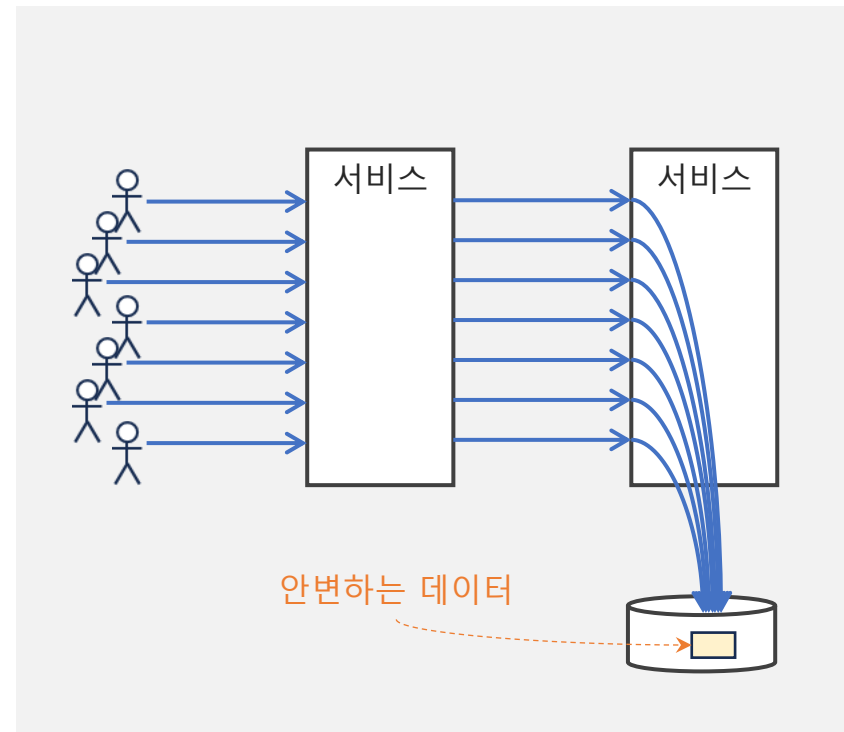
API 호출을 과하게 유발하는 구조

② 비효율적인 코드



코드가 API를 과도하게 호출

③ 같은 데이터를 반복해서 조회



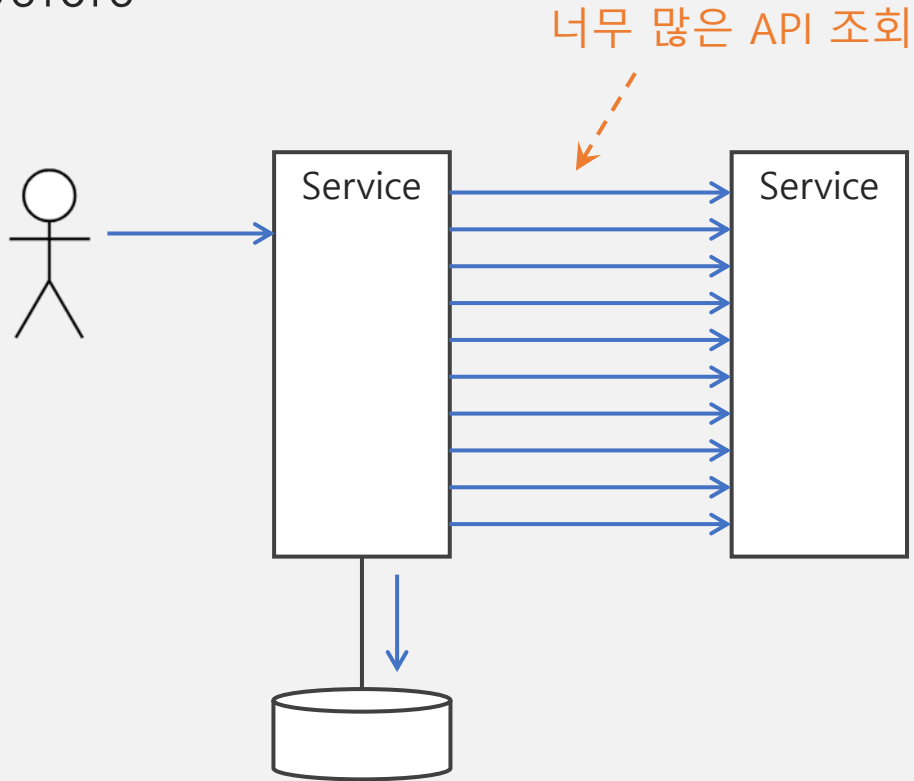
필요가 없는데 API 호출

② 비효율적인 코드

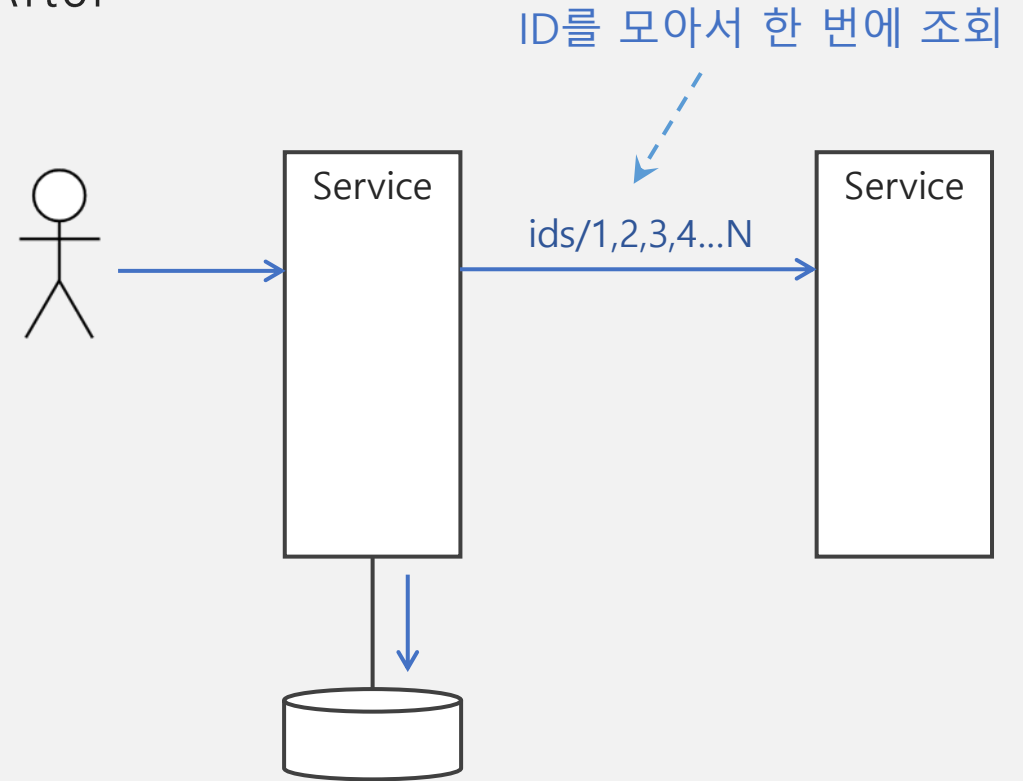
문제점: 타서비스의 API를 과도하게 호출하도록 구현하여 전반적인 성능 저하 발생

개선방법: 코드 레벨 알고리즘 개선

Before



After



예시1 - 상세 정보 조회

② 비효율적인 코드

수강 현황

수강생별 상세 현황

이름	진도율	시험 성적	과제 점수	최종
 김지우	95%	92 / 100	92 / 100	수료
 이수민	88%	85 / 100	85 / 100	진행
 박동현	45%	70 / 100	70 / 100	재도
 최유리	50%	98 / 100	98 / 100	특기
 장우재	80%	80 / 100	80 / 100	특기

수강
서비스

2. 시험/과제 조회

시험/과제
서비스

1. 수강 현황 조회

수강현황

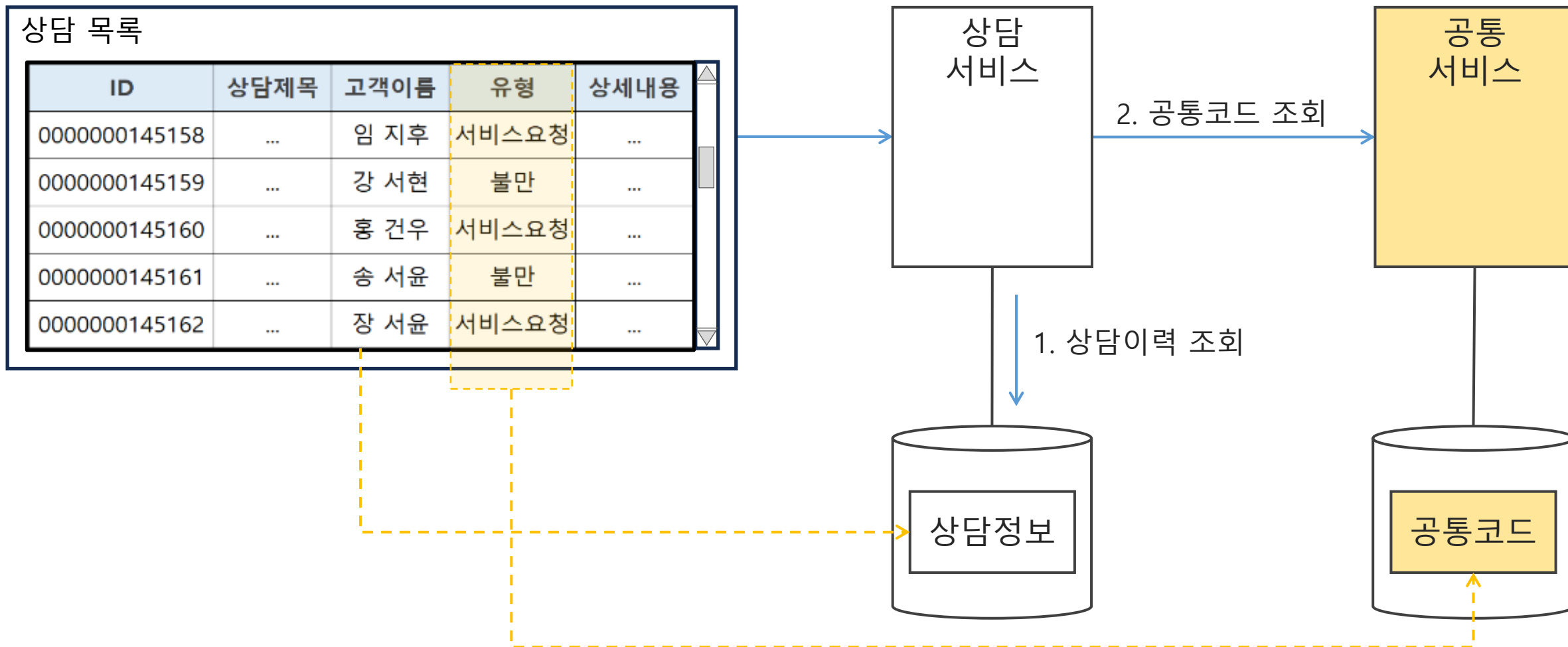
시험결과

과제결과

예시2 - 부가 정보 조회

② 비효율적인 코드

공통 코드



예시 – 단건 조회

```
public class ProductService {  
  
    public ProductDTO fetchProductById(final String id) {  
        //1. 상품 정보 조회  
        ProductDTO product = productRepository.findById(id);  
  
        //2. 판매자/판매처 정보 추가  
        setExtraInfo(product);  
        return product;  
    }  
  
    private ProductDTO setExtraInfo(ProductDTO product) {  
        //1. 판매자  
        SellerDTO seller = sellerGateway.findById(product.getSellerId());  
        product.setSellerName(seller.getName());  
        //2. 판매처  
        OrganizationDTO organization = organizationGateway.findById(product.getOrgId());  
        product.setOrgName(organization.getName());  
  
        return product;  
    }  
}
```

예시 - 여러 건 조회?

```
public ProductResponse fetchProductsByIds_Take1(final List<String> ids) {  
    //1. 상품 정보 조회  
    List<ProductDTO> products = productRepository.findAllByIdIn(ids);  
  
    //2. 판매자/판매처 정보 추가  
    for(ProductDTO product : products) {  
        setExtraInfo(product);  
    }  
    return new ProductResponse(products);  
}
```

```
//1. 상품 정보 조회  
ProductDTO product = productRepository.findById(id);  
  
//2. 판매자/판매처 정보 추가  
setExtraInfo(product);  
return product;
```

단건 조회 API의 메소드를 활용

```
public ProductResponse fetchProductsByIds(final List<String> ids) {
    List<ProductDTO> products = productRepository.findAllByIdIn(ids);
    setExtraInfos(products);
    return new ProductResponse(products);
}

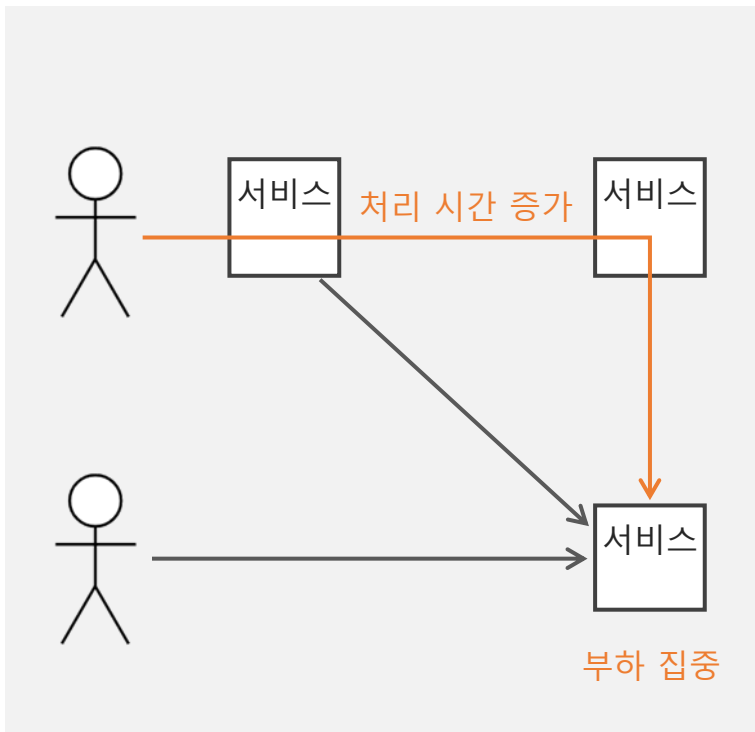
private void setExtraInfos(List<ProductDTO> products) {
    // 1. 판매자 ID 수집 & 조회
    Set<String> sellerIds = products.stream().map(ProductDTO::getSellerId).collect(Collectors.toSet());
    Map<String, SellerDTO> sellerMap = sellerGateway.findByIds(sellerIds);

    // 2. 판매처 ID 수집 & 조회
    Set<String> orgIds = products.stream().map(ProductDTO::getOrgId).collect(Collectors.toSet());
    Map<String, OrganizationDTO> orgMap = organizationGateway.findByIds(orgIds);

    // 3. 판매자/판매처 조합
    for (ProductDTO product : products) {
        SellerDTO seller = sellerMap.get(product.getSellerId());
        if (seller == null) {
            Log.error("FAILED TO FIND ProductDTO id:{})", product.getSellerId());
        } else {
            product.setSellerName(seller.getName());
        }
        OrganizationDTO org = orgMap.get(product.getOrgId());
        if (org == null) {
            Log.error("FAILED TO FIND ProductDTO id:{})", product.getOrgId());
        } else {
            product.setOrgName(org.getName());
        }
    }
}
```

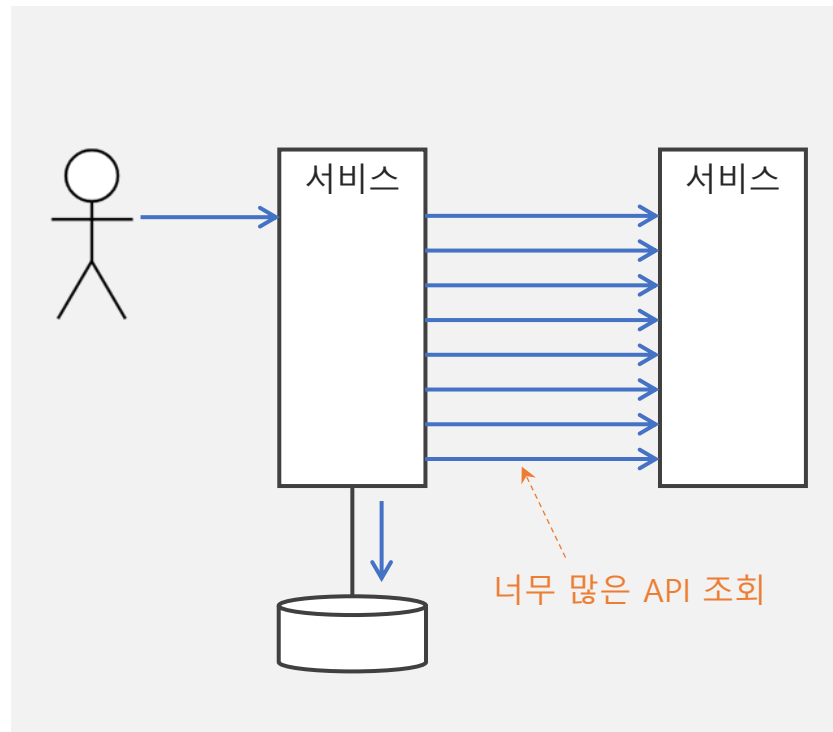
MSA의 성능 저하 유형

① 비효율적인 구조



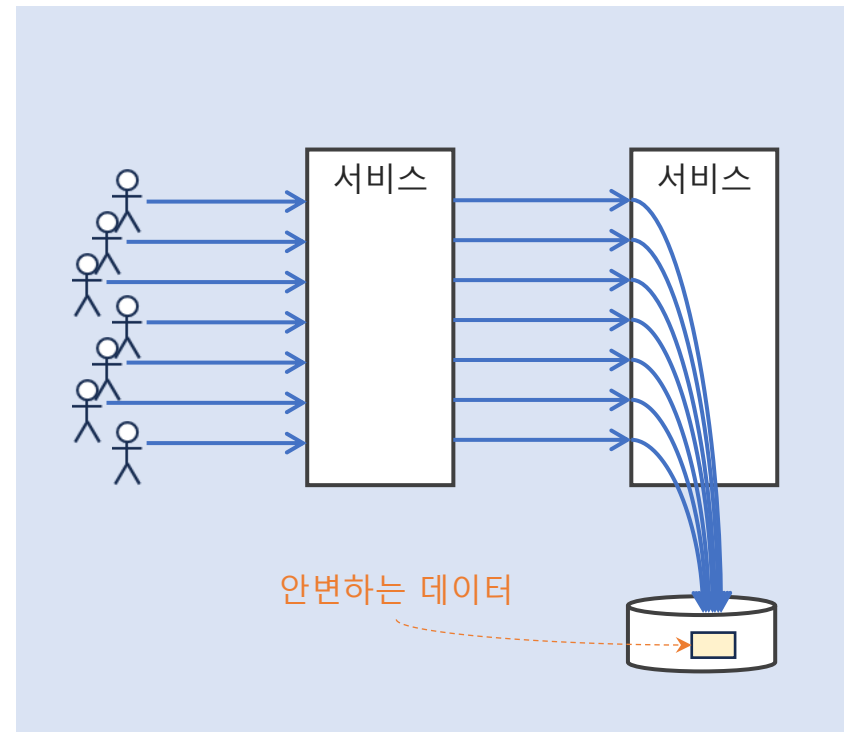
API 호출을 과하게 유발하는 구조

② 비효율적인 코드



코드가 API를 과도하게 호출

③ 같은 데이터를 반복해서 조회

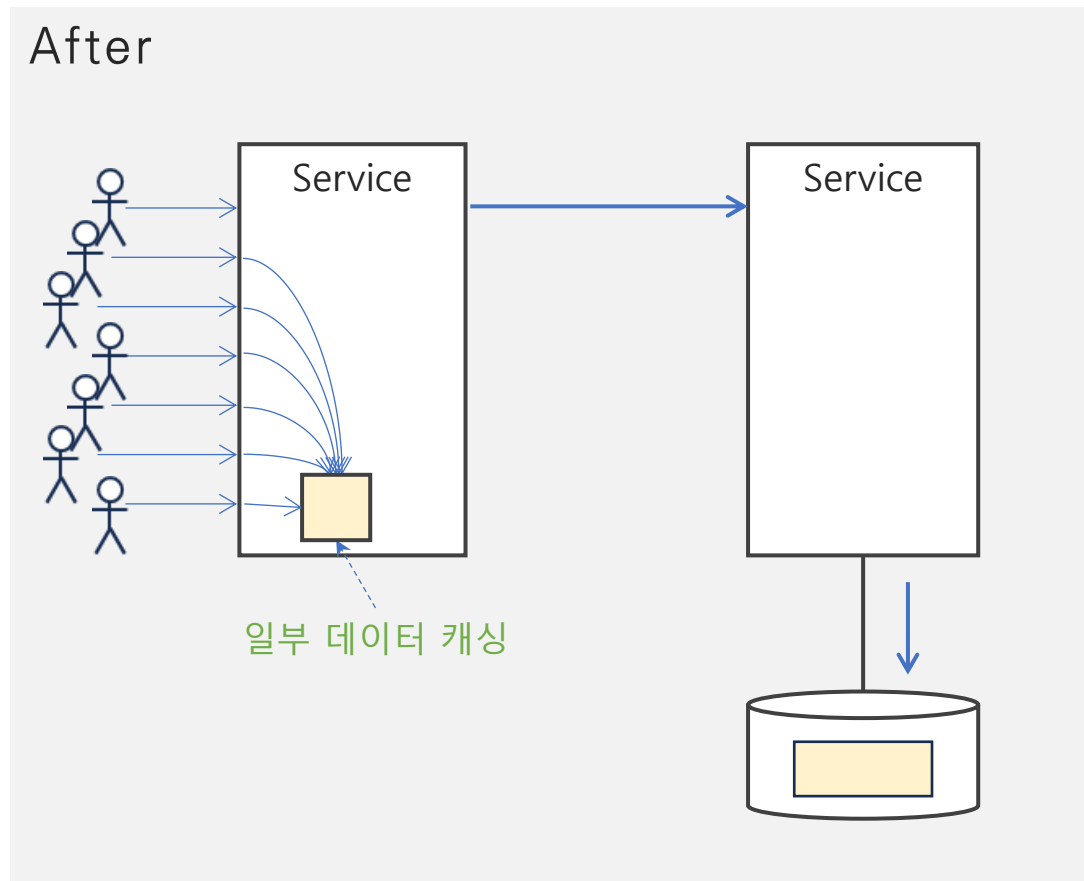
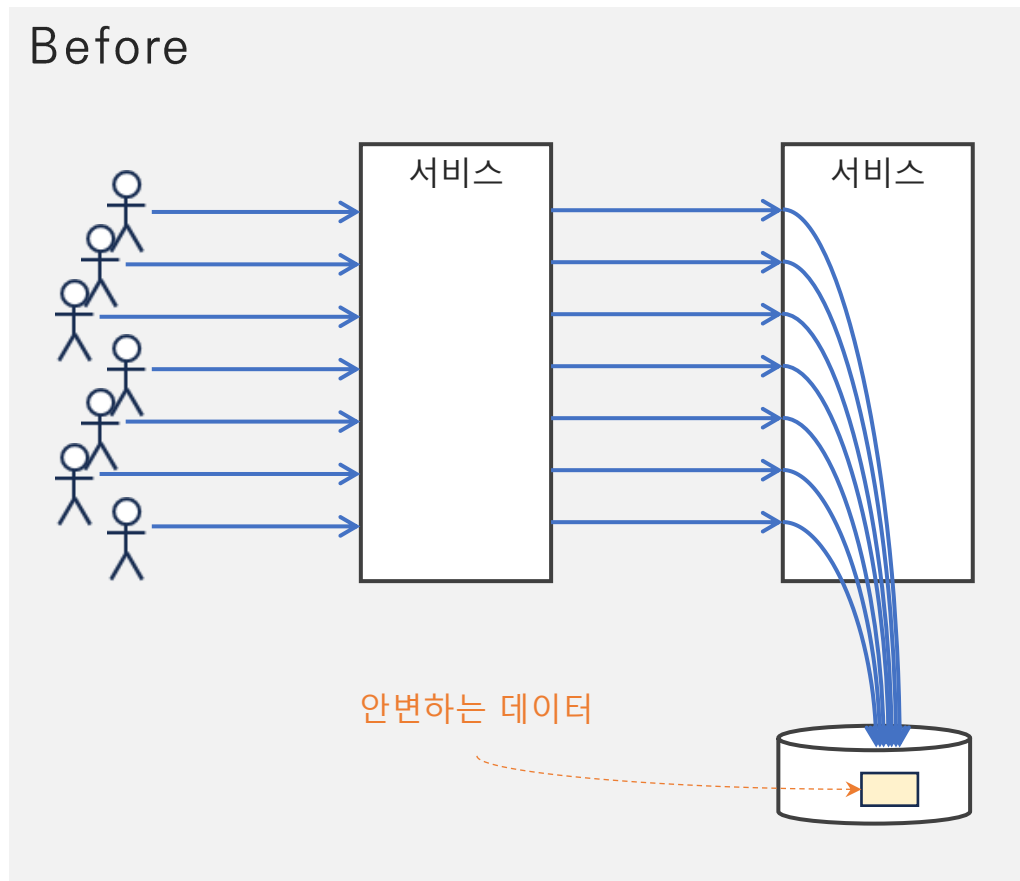


필요가 없는데 API 호출

③ 같은 데이터를 반복해서 조회

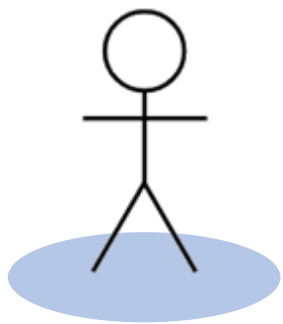
문제점: 변하지 않는 데이터를 API로 매번 반복해서 조회하여 전반적인 성능 저하 발생

개선방안: 데이터를 캐싱하여 API 중복 조회 최소화



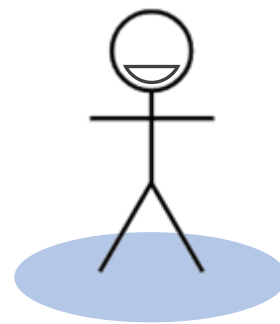
캐싱?

데이터가 변경되면
정합성이 깨지자나!?
너무 위험한 거 아냐?



개발자

아니 괜찮아.
변하지 않는 데이터도 많지만
변해도 알차게 적용하는 경우가 많아.

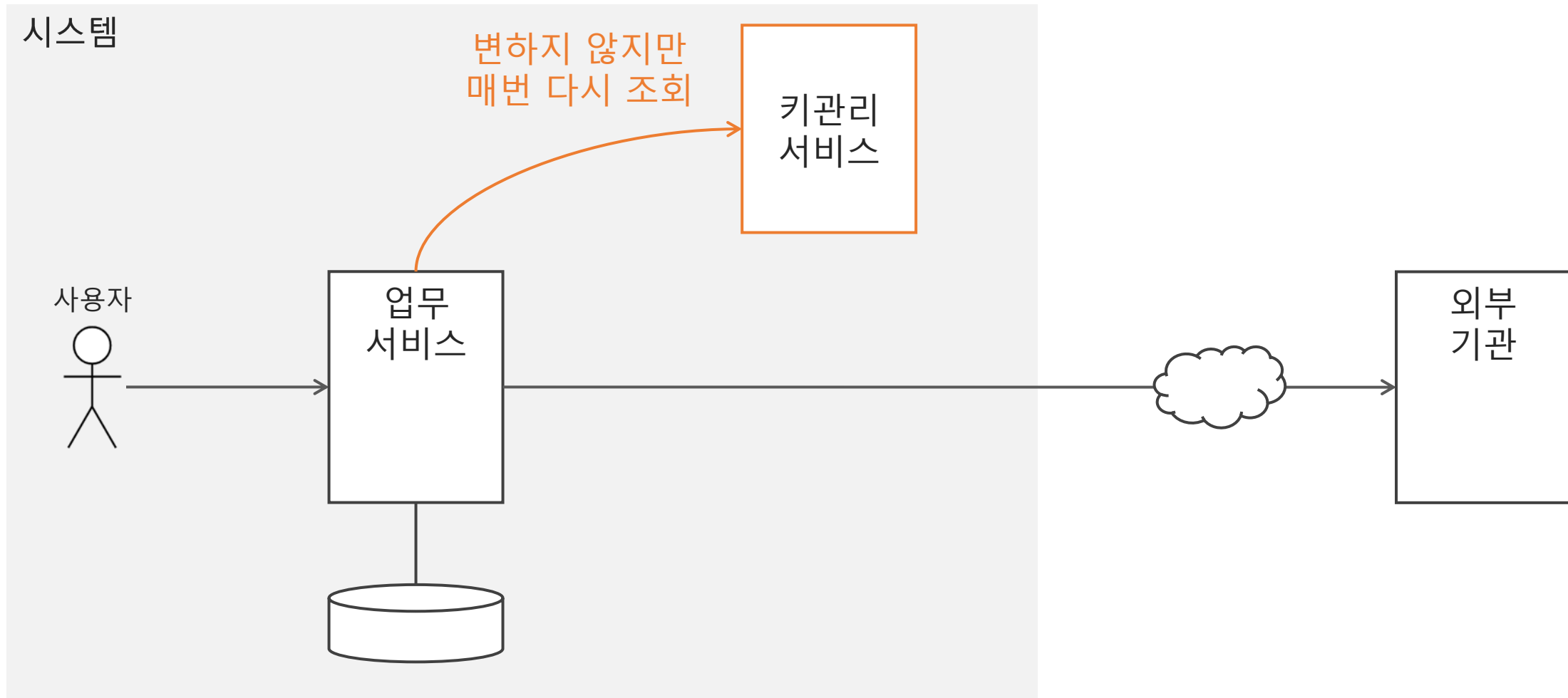


개발자

I. 안 변하는 데이터

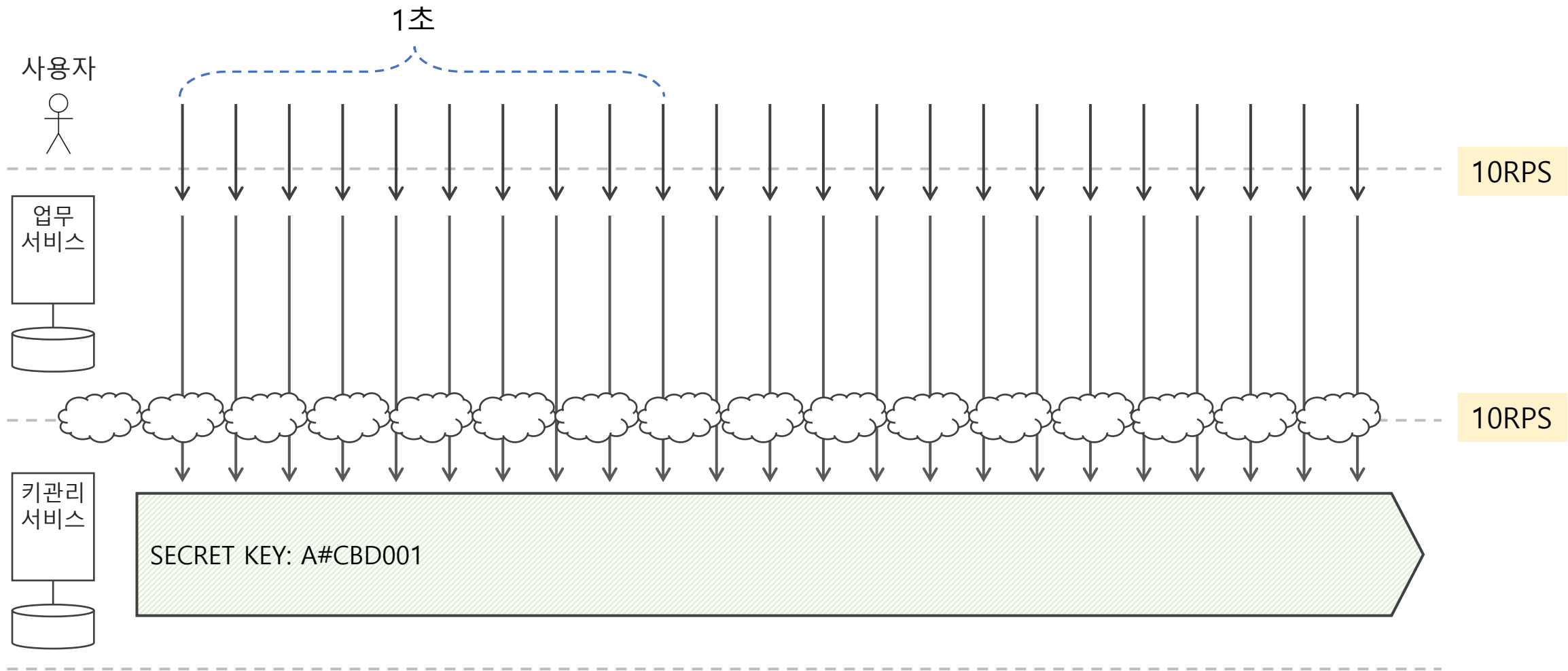
③ 같은 데이터를 반복해서 조회
I. 안 변하는 데이터

비밀 키



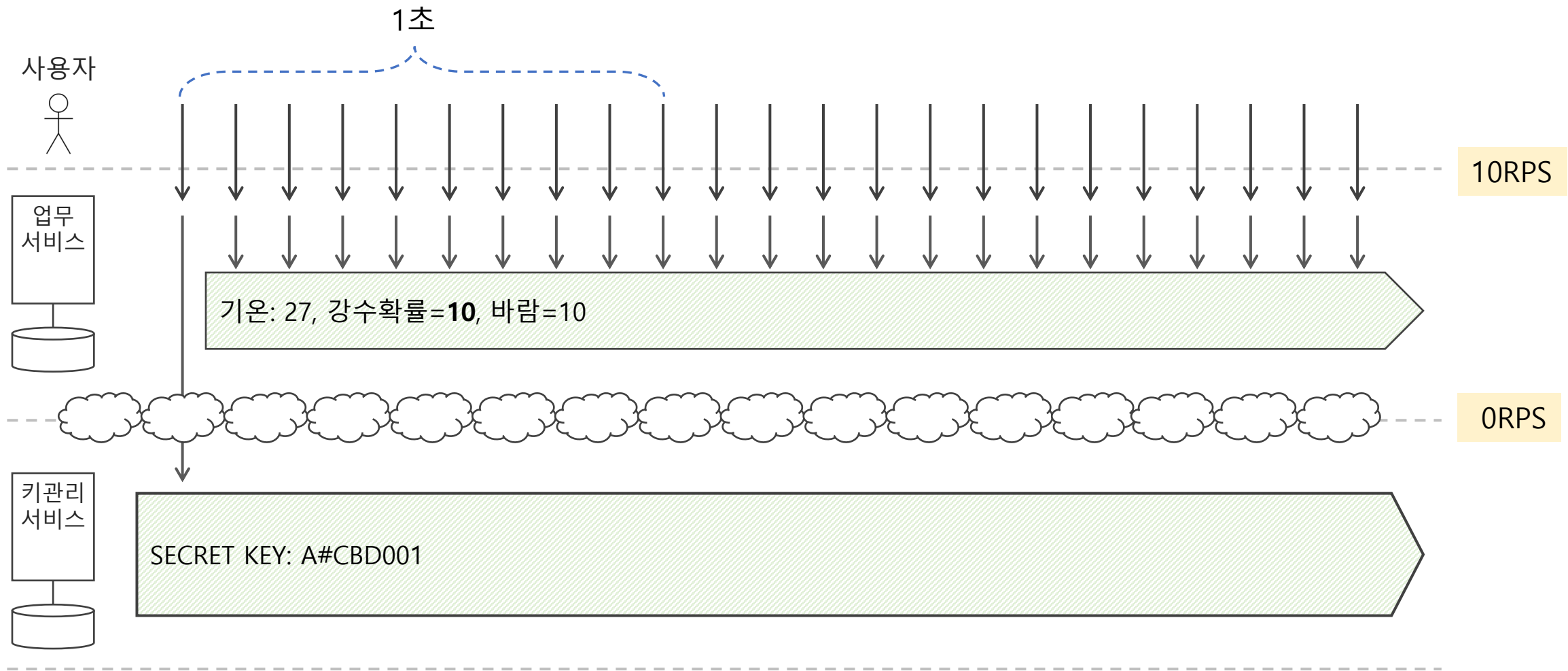
비밀 키 – Before

③ 같은 데이터를 반복해서 조회
I. 안 변하는 데이터



비밀 키 - After

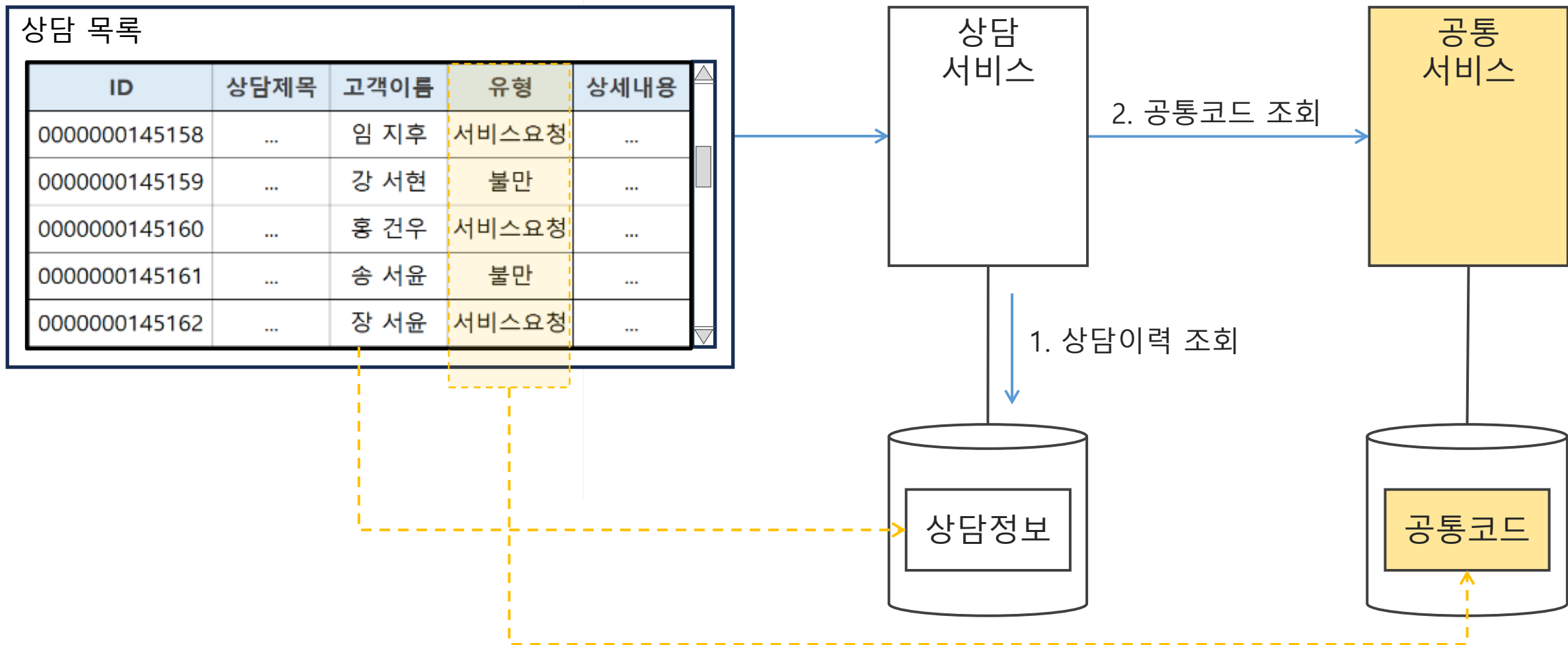
③ 같은 데이터를 반복해서 조회
1. 안 변하는 데이터



II. 실질적으로 안변해

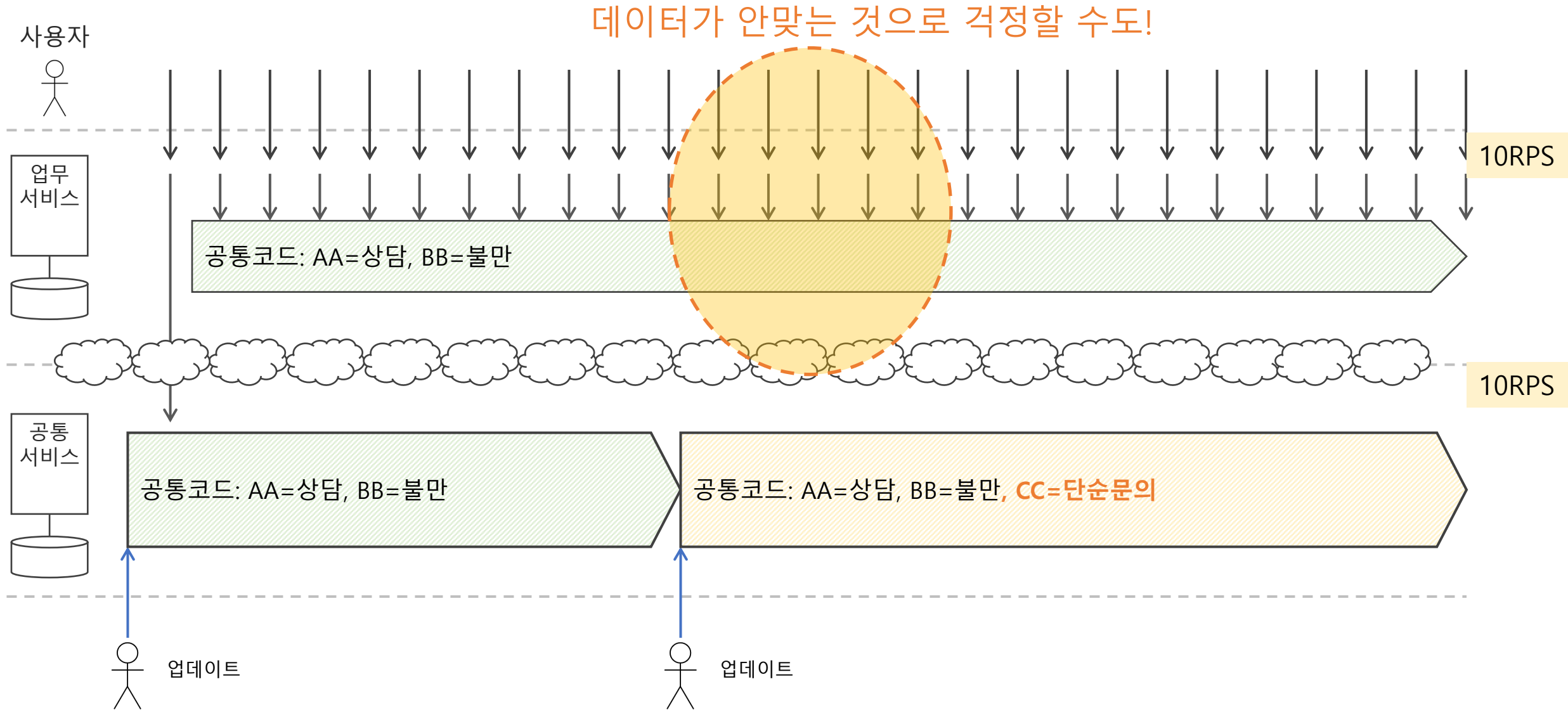
③ 같은 데이터를 반복해서 조회
II. 실질적으로 안변해

공통 코드



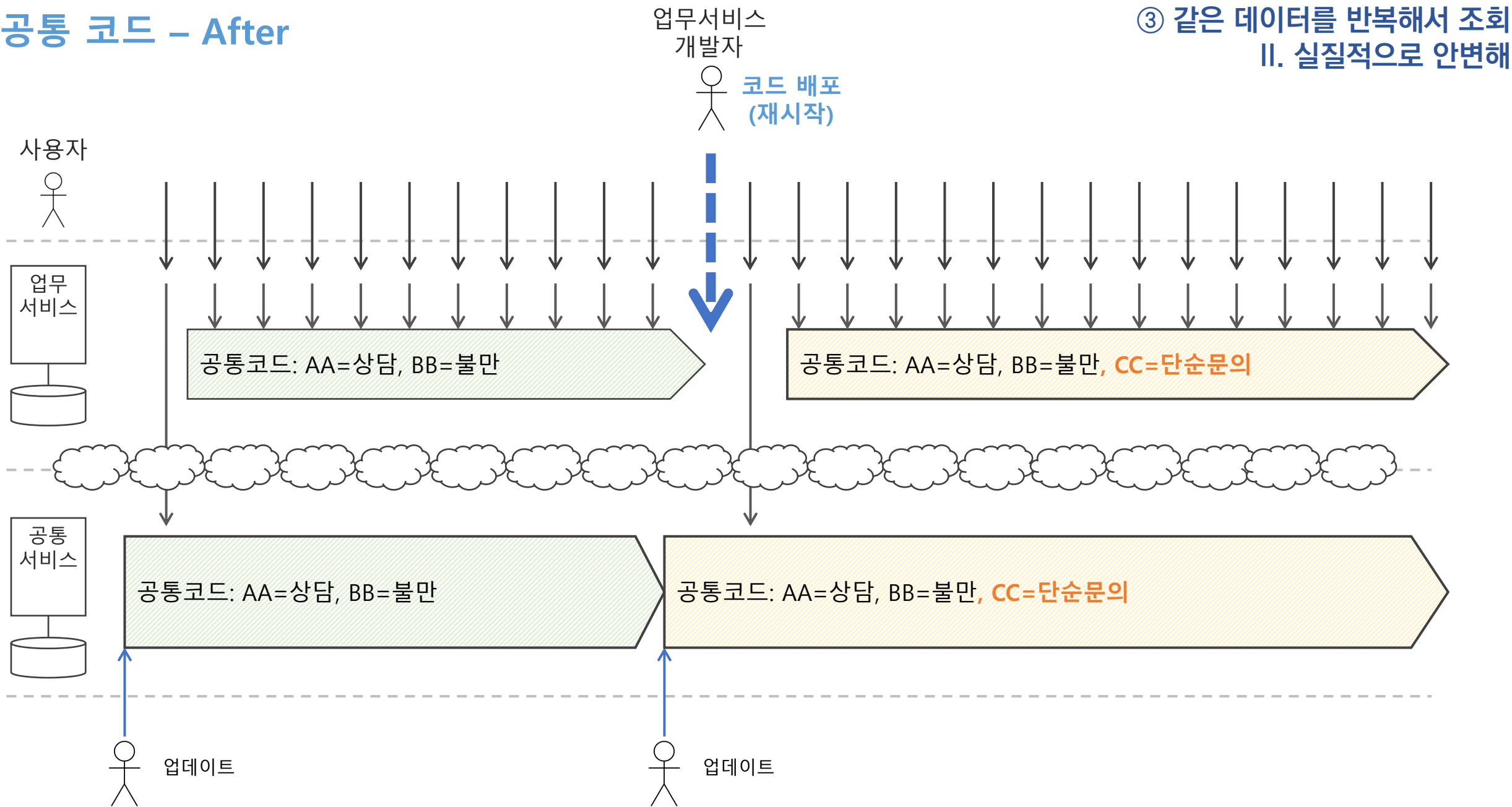
공통 코드 - Before

③ 같은 데이터를 반복해서 조회
II. 실질적으로 안변해



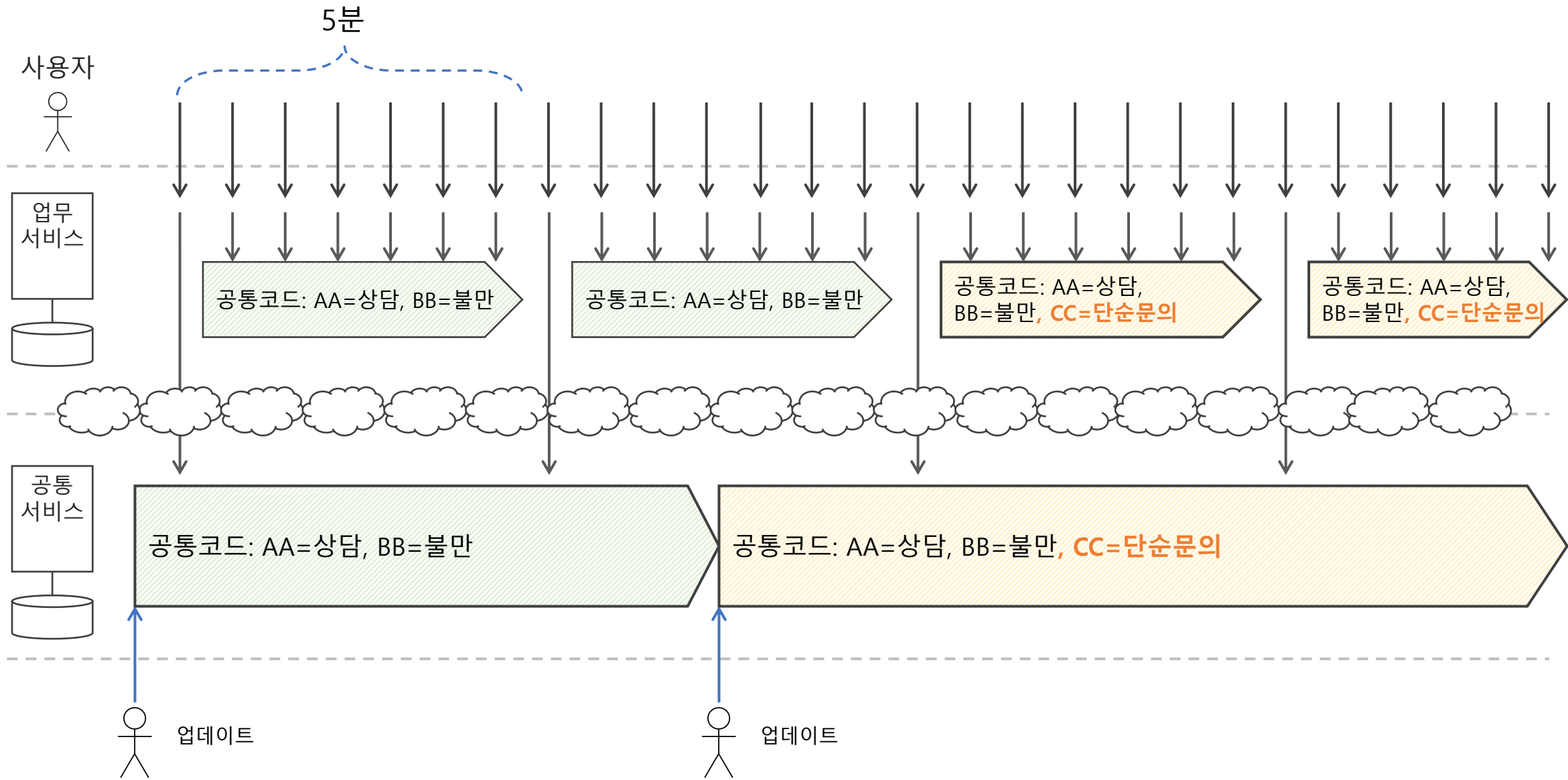
공통 코드 - After

③ 같은 데이터를 반복해서 조회
II. 실질적으로 안변해



공통 코드 – After (2)

③ 같은 데이터를 반복해서 조회
II. 실질적으로 안변해



III. 변하지만, 늦게 봐도 괜찮아

③ 같은 데이터를 반복해서 조회
III. 변하지만, 늦게 봐도 괜찮아

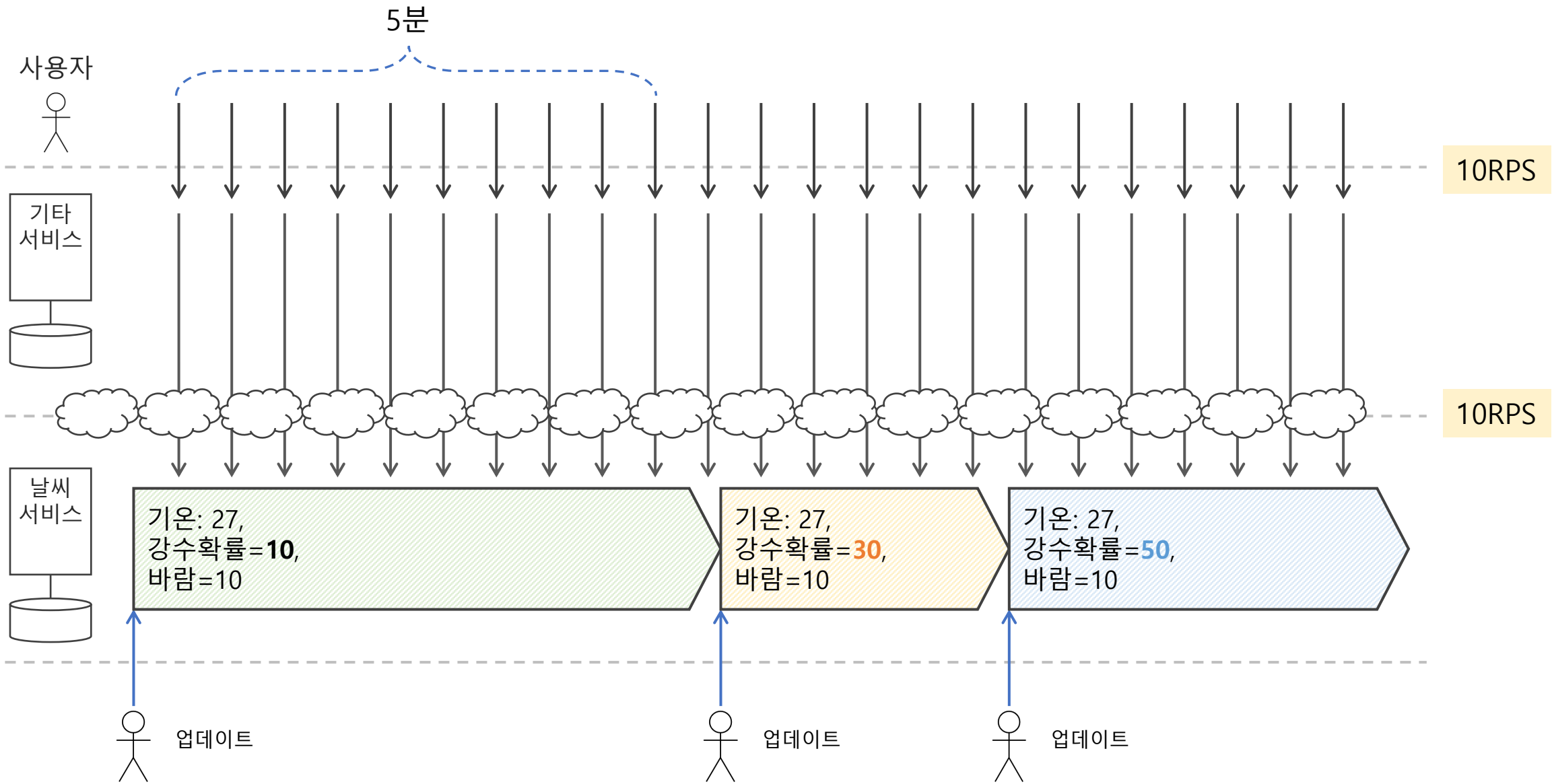
날씨 정보

시스템



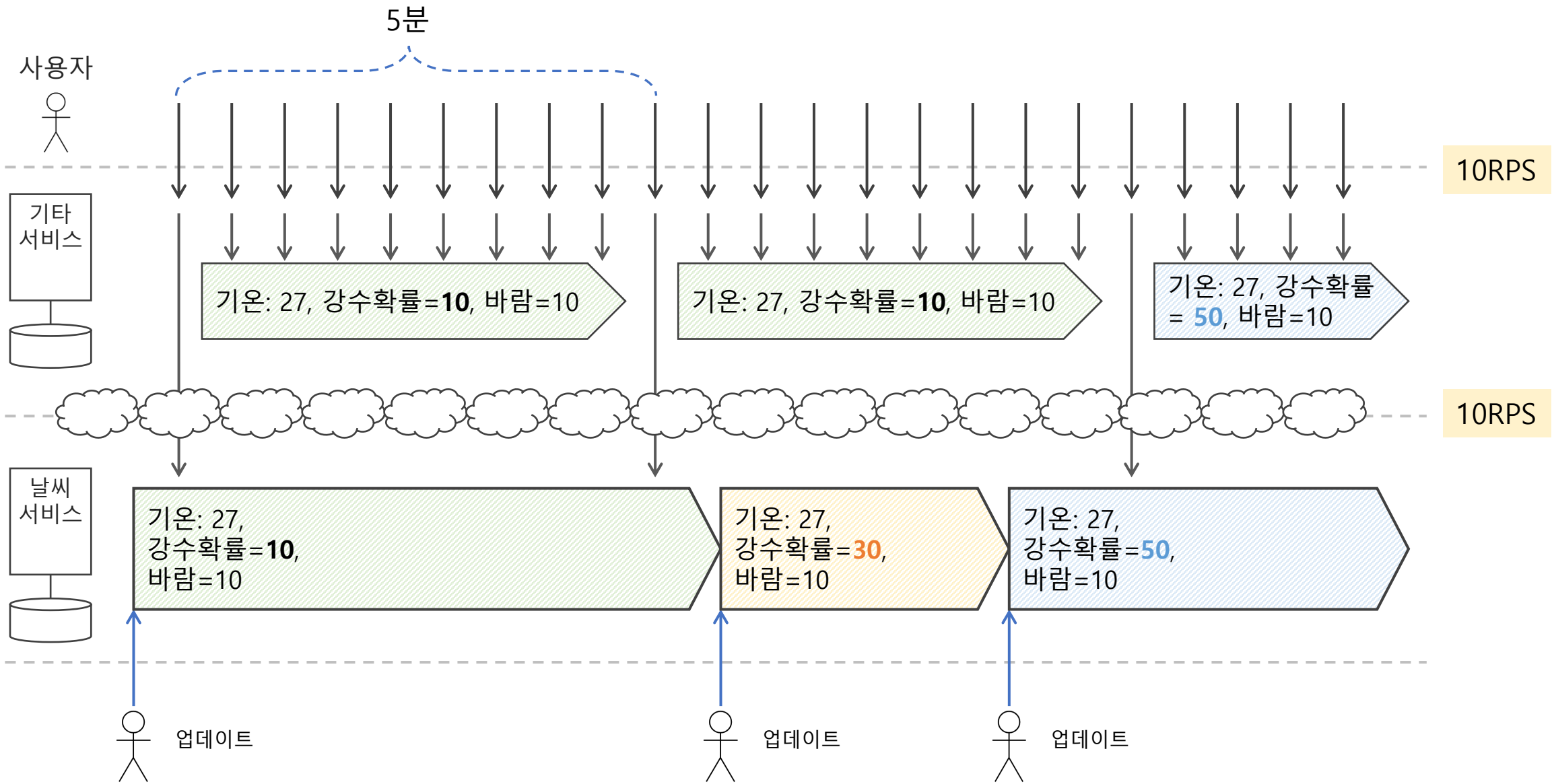
날씨 정보 - Before

③ 같은 데이터를 반복해서 조회
III. 변하지만, 늦게 봐도 괜찮아



날씨 정보 - After

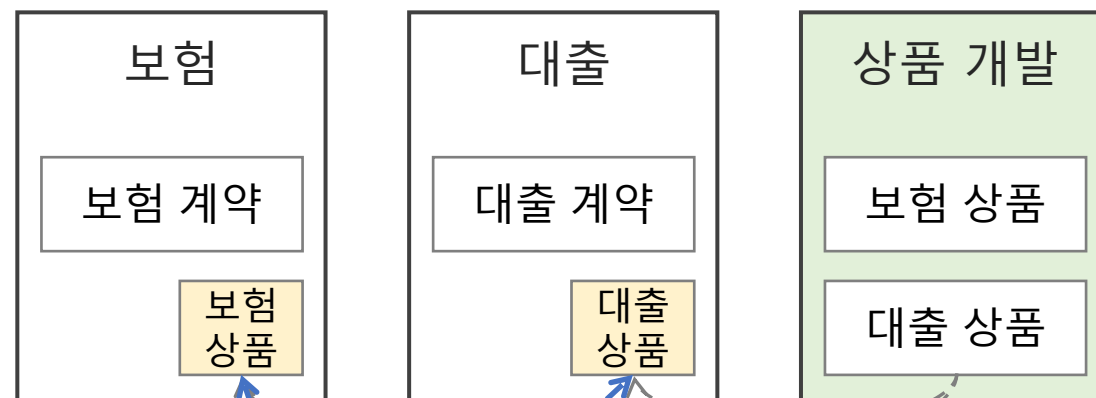
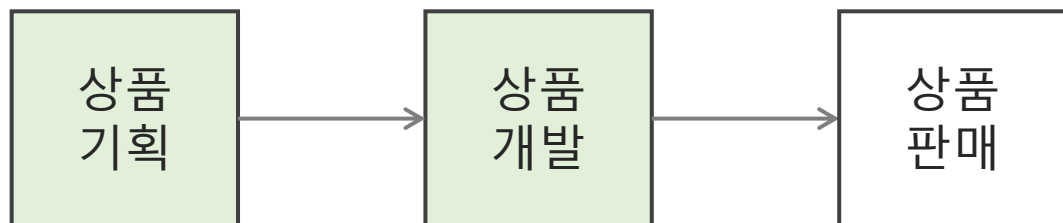
③ 같은 데이터를 반복해서 조회
III. 변하지만, 늦게 봐도 괜찮아



IV. 변하지만 패턴이 있어

③ 같은 데이터를 반복해서 조회
IV. 변하지만 패턴이 있어

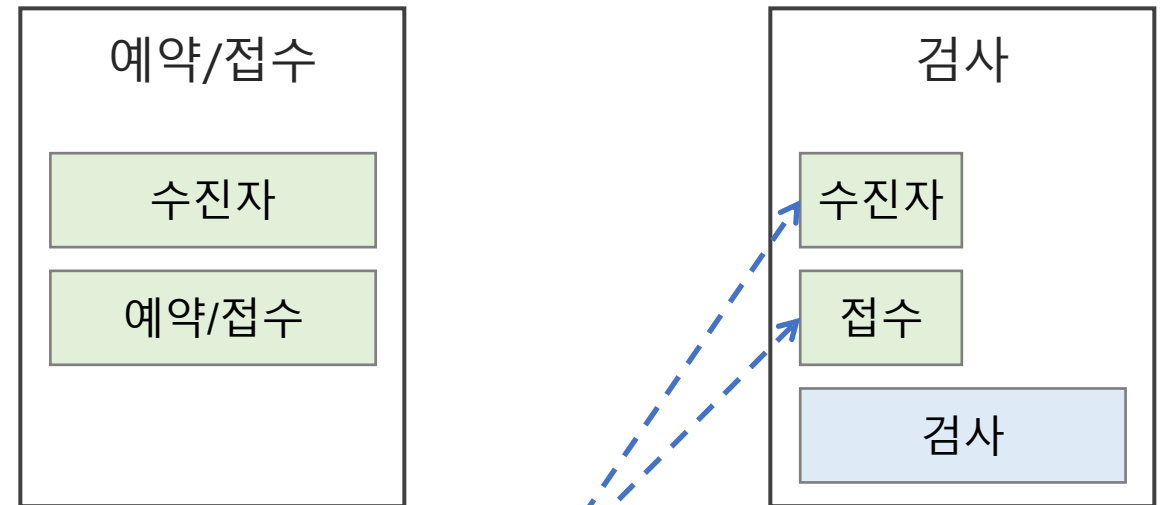
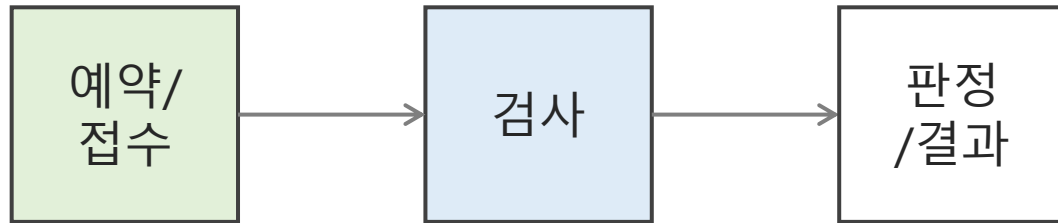
보험/대출



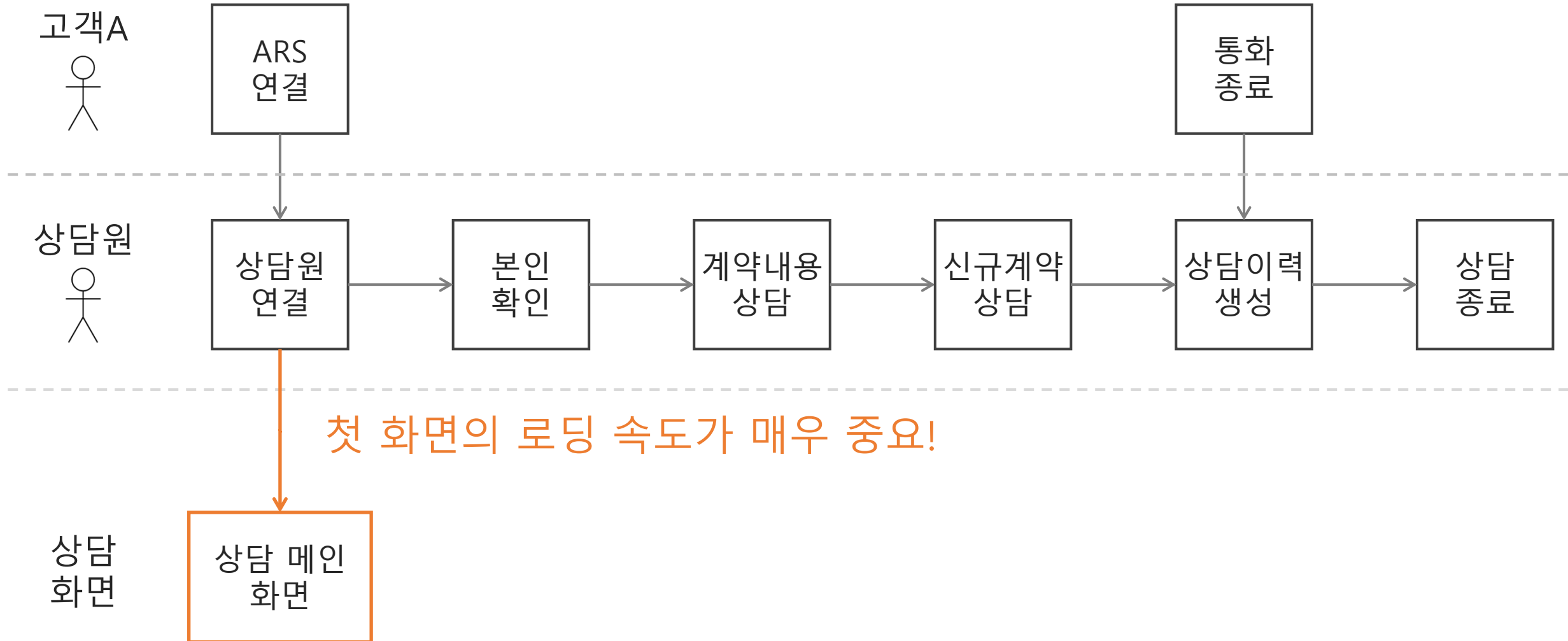
판매 후에는 변하지 않음

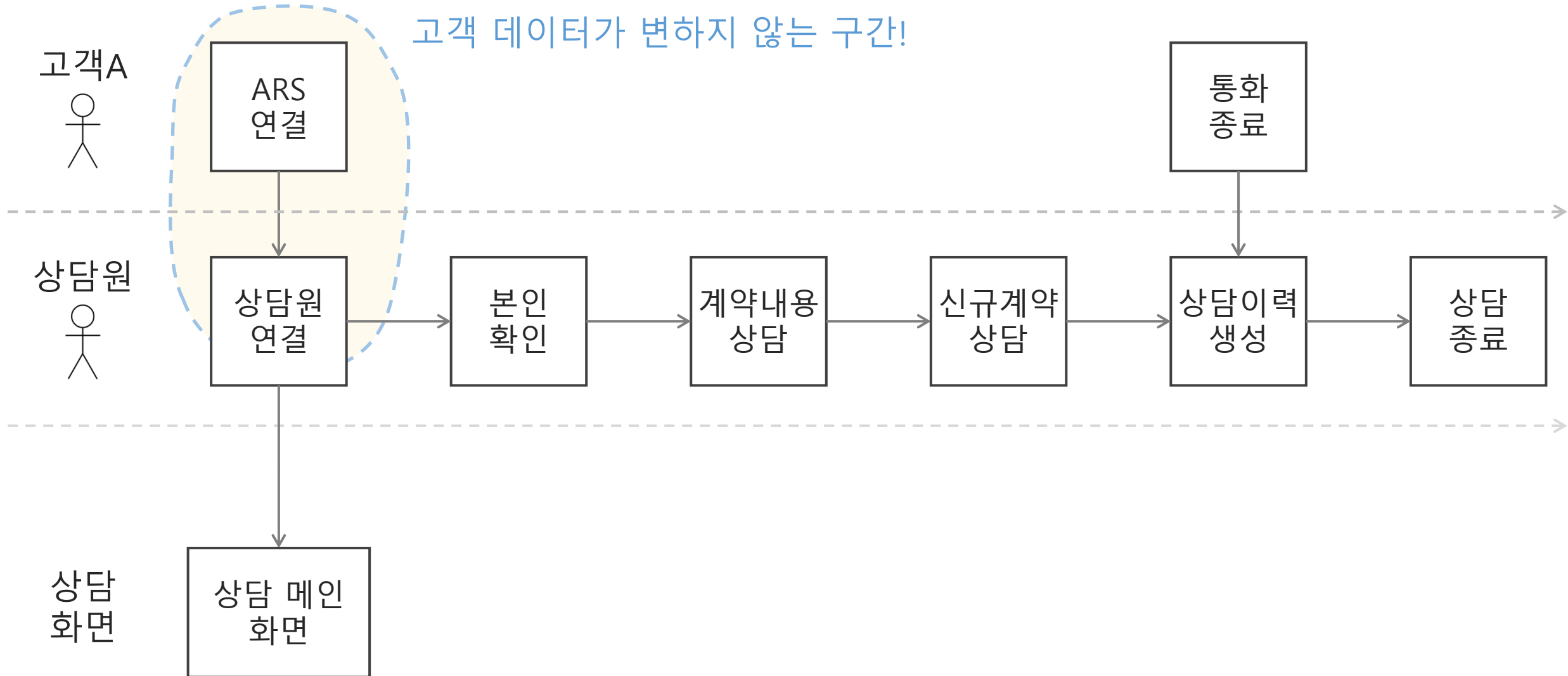
③ 같은 데이터를 반복해서 조회
IV. 변하지만 패턴이 있어

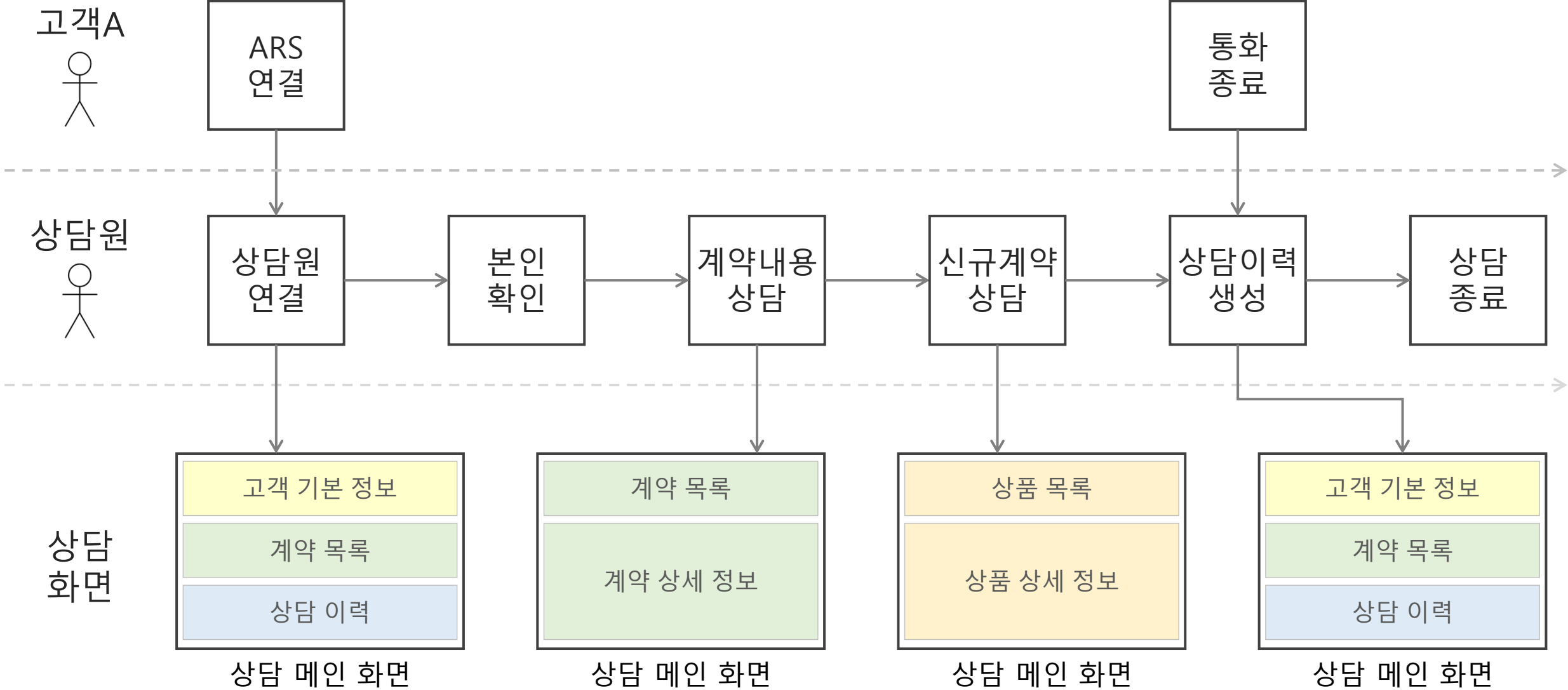
건강 검진

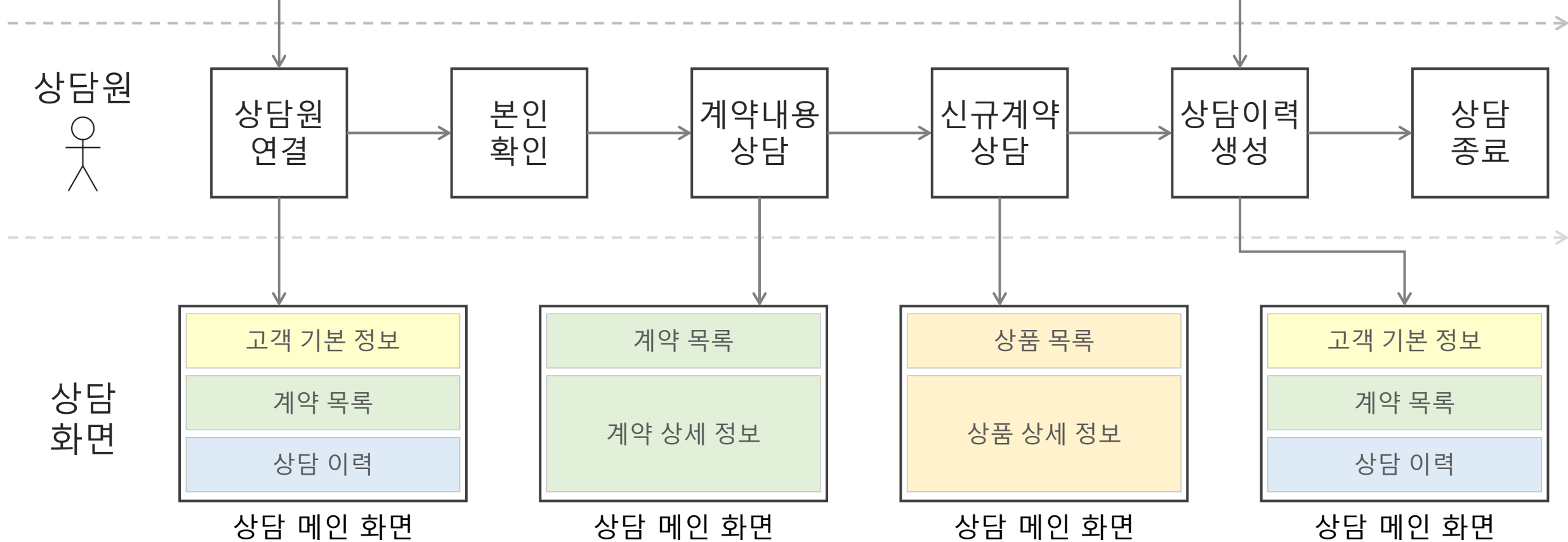


검진 받는 중에는
잘 변하지 않음





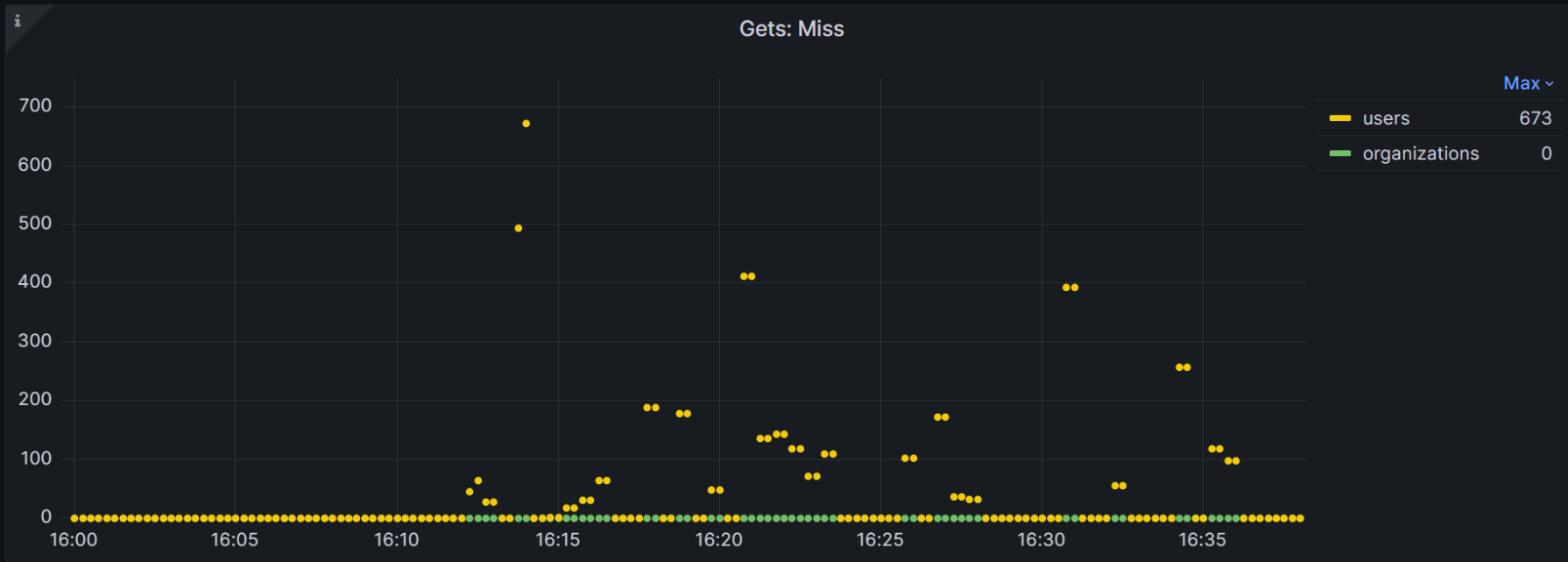
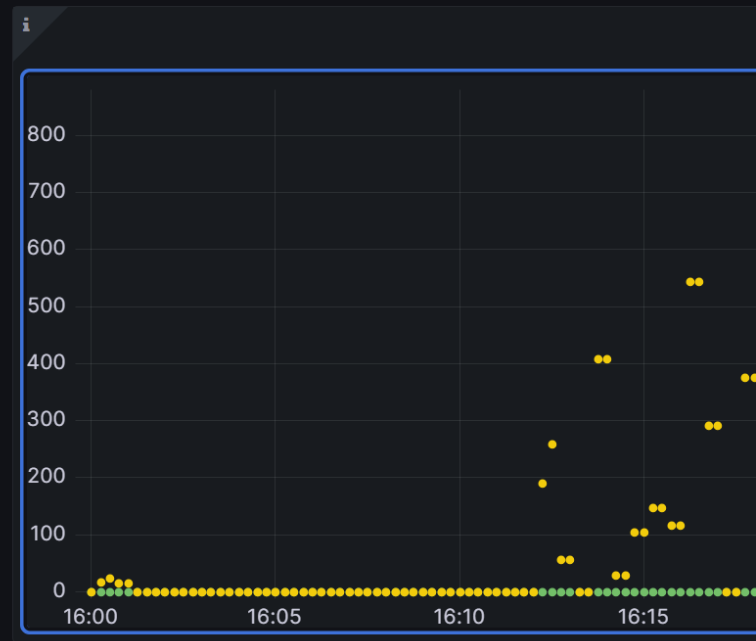
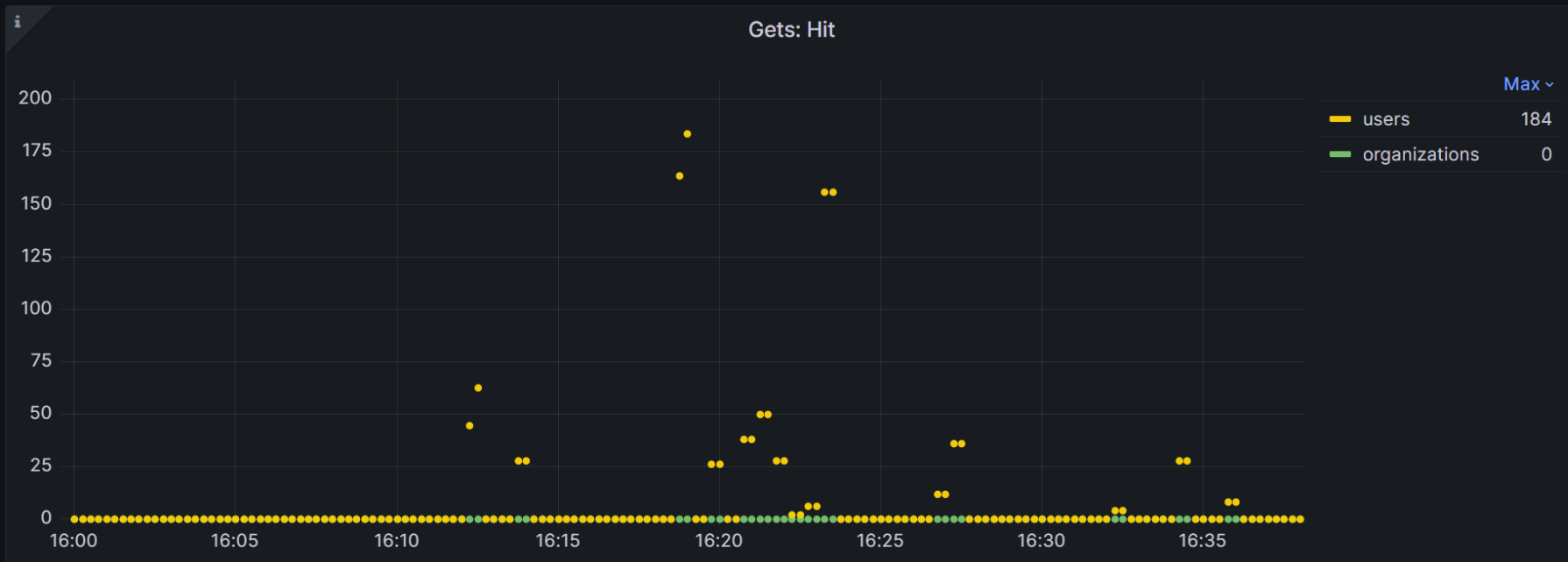




많은 데이터를 중복해서 조회하는데...

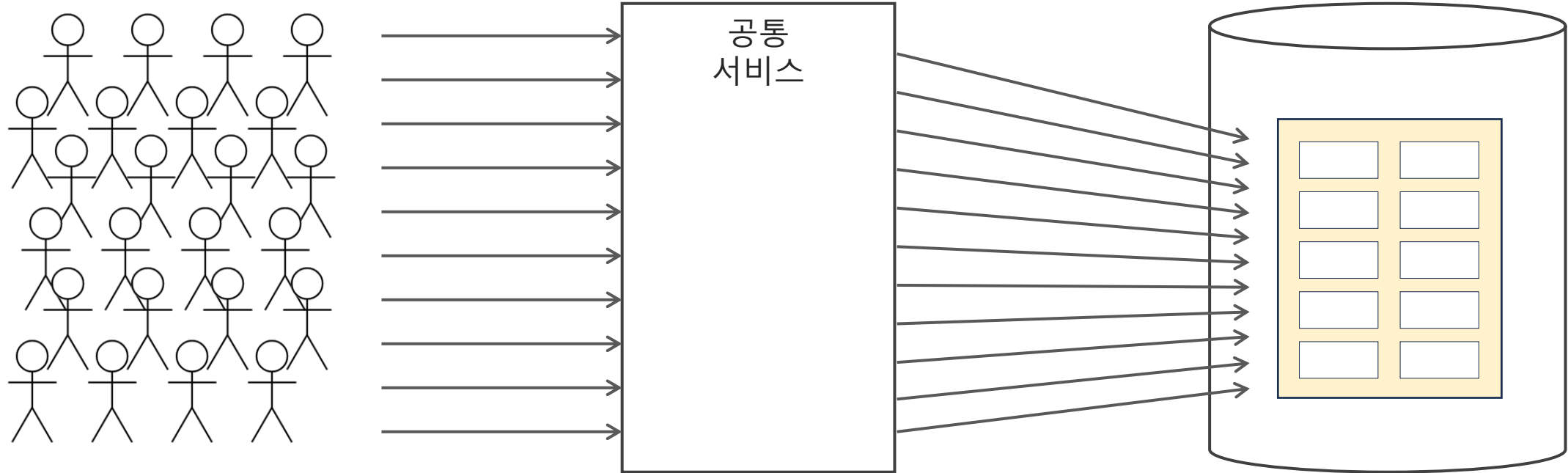
- ① 고객A의 기본정보 API 2회
- ② 고객A의 계약목록 3회
- ③ 고객A의 계약상세 X회
- ④ 고객A의 상담이력 2회...

▾ Cache Manager: cacheManager



V. 변하지만, 매우 빨리 조회해

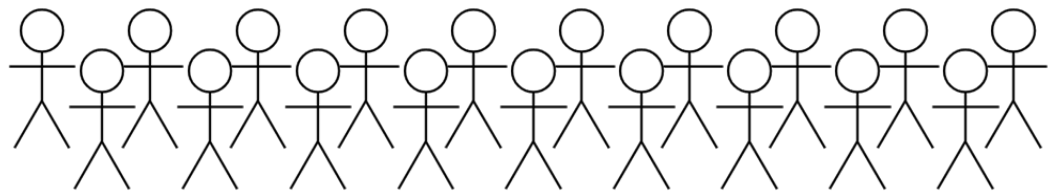
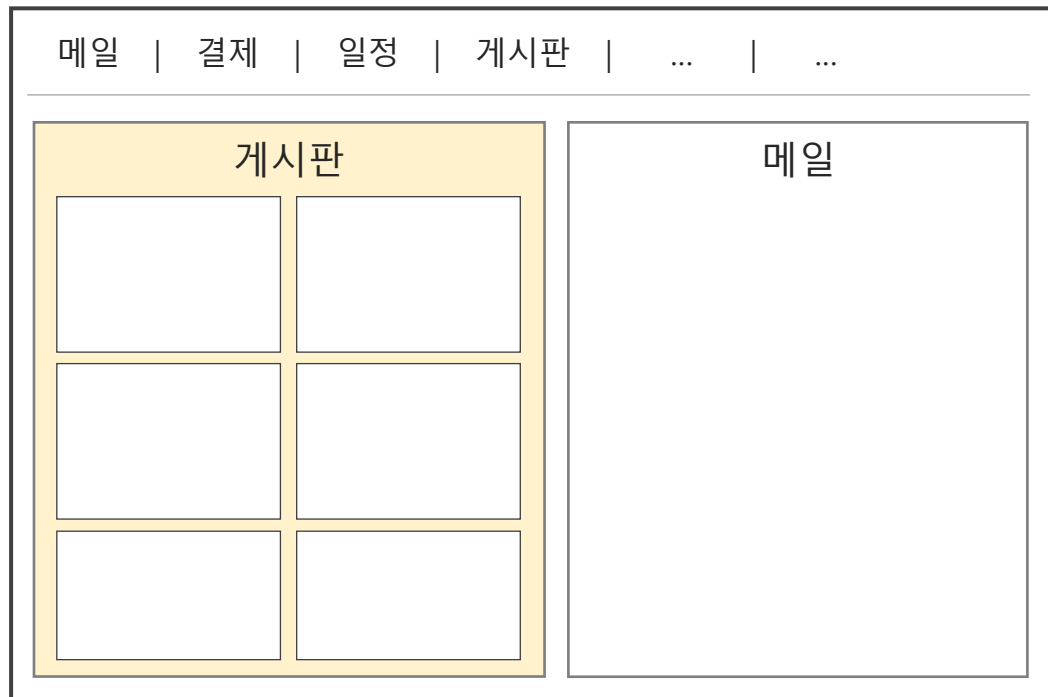
③ 같은 데이터를 반복해서 조회
V. 변하지만, 매우 빨리 조회해



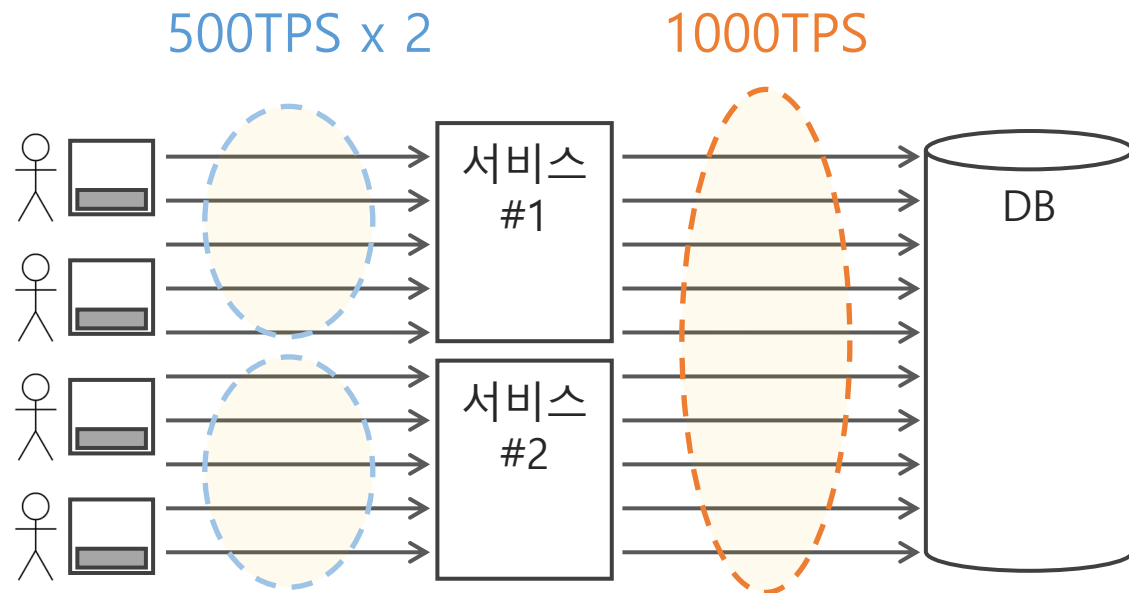
V. 변하지만, 매우 빨리 조화해

③ 같은 데이터를 반복해서 조회
V. 변하지만, 매우 빨리 조화해

게시판



임직원 200,000명



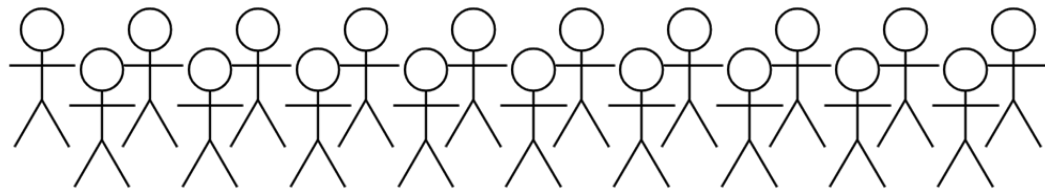
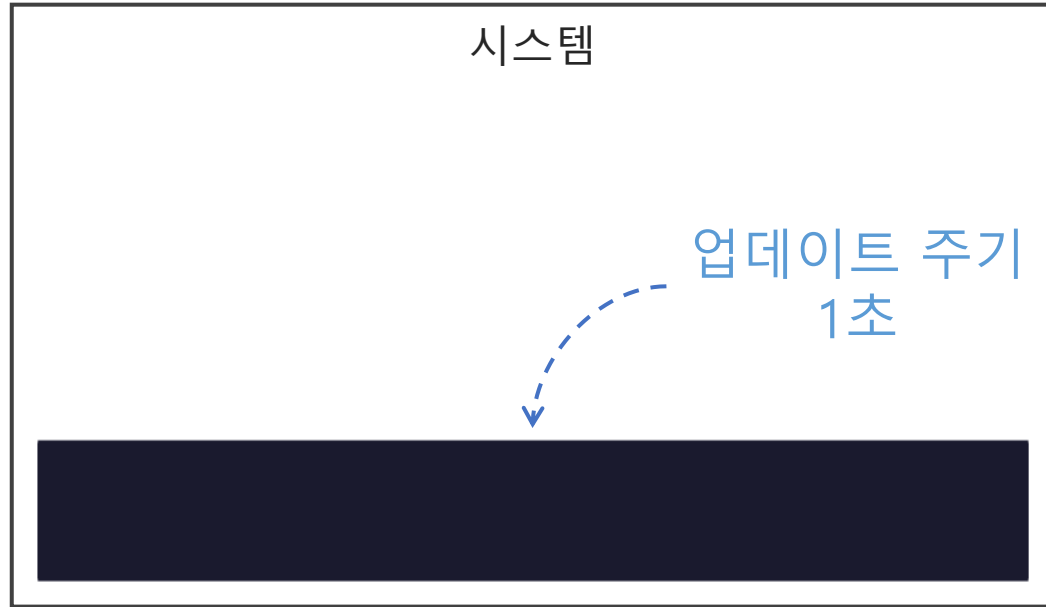
1000TPS: 20만명 x 60% x 10회 / 1200초

- 사용자: 20만명
- 출근 시간대 60% 이용
- 로그인 후 평균 10회 클릭
- 출근 피크 시간: 20분 (1200초)

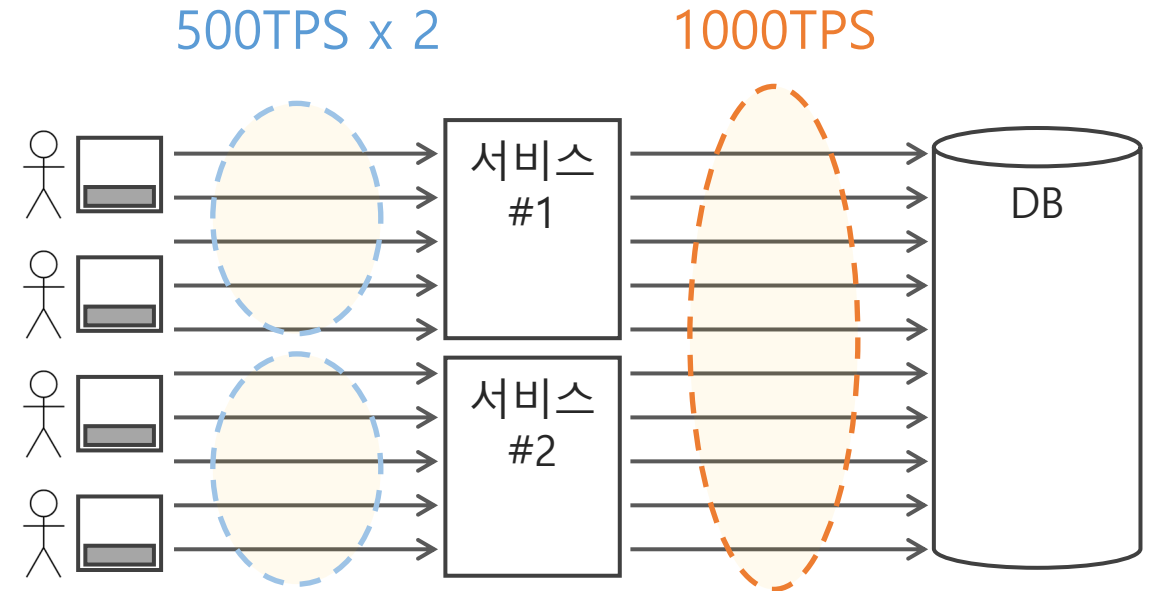
V. 변하지만, 매우 빨리 조회해

③ 같은 데이터를 반복해서 조회
V. 변하지만, 매우 빨리 조회해

공지 사항



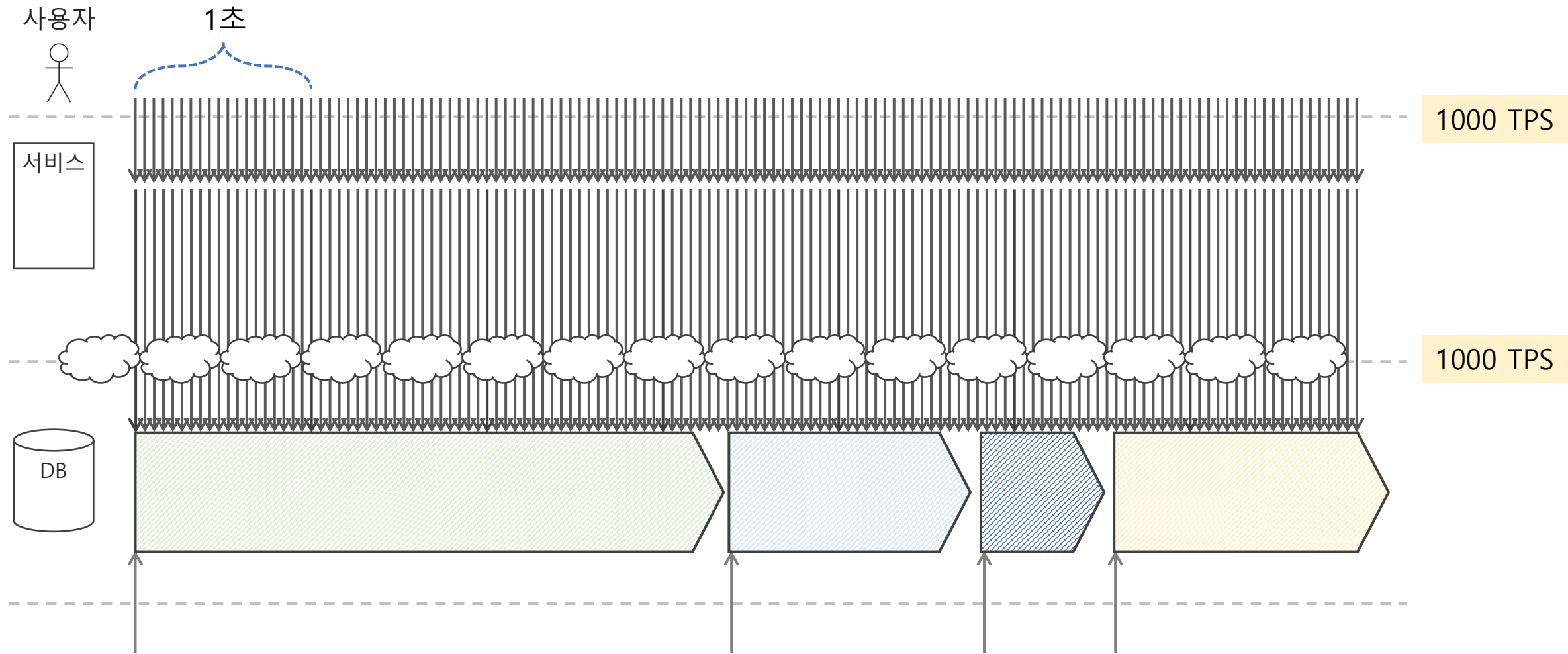
상담사 1000명



1000TPS: 1000명 x 1TPS

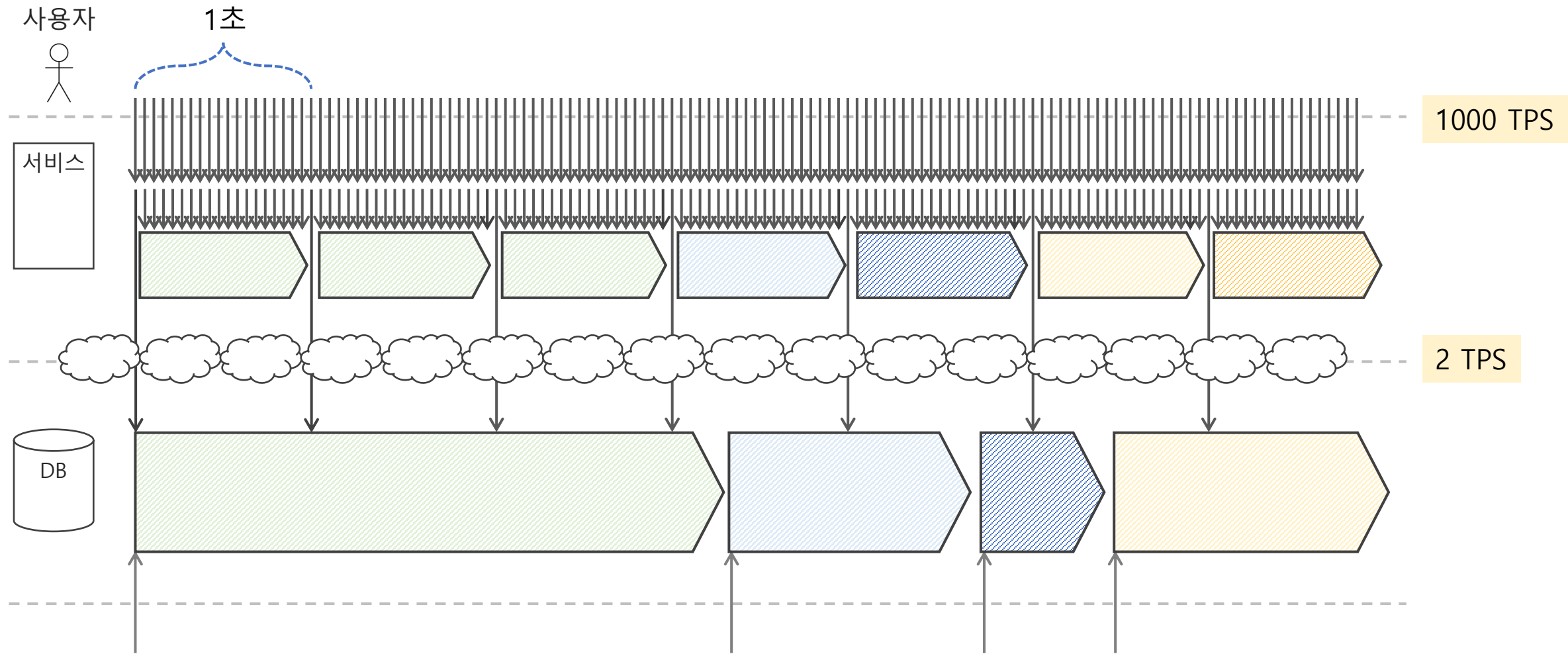
대기 현황 – Before

③ 같은 데이터를 반복해서 조회
V. 변하지만, 매우 빨리 조회해



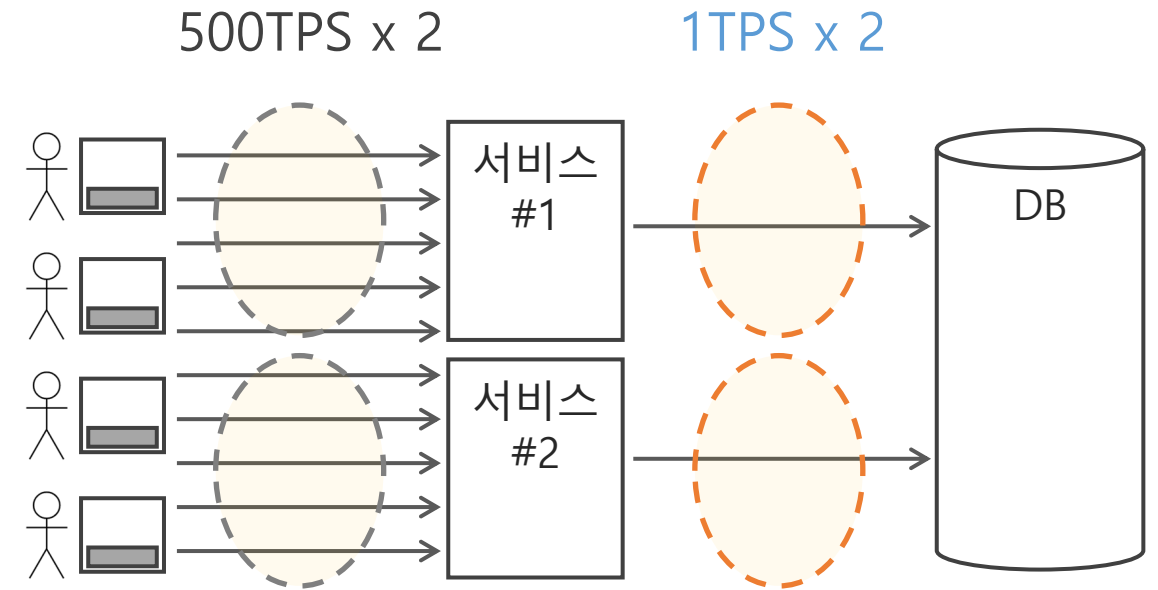
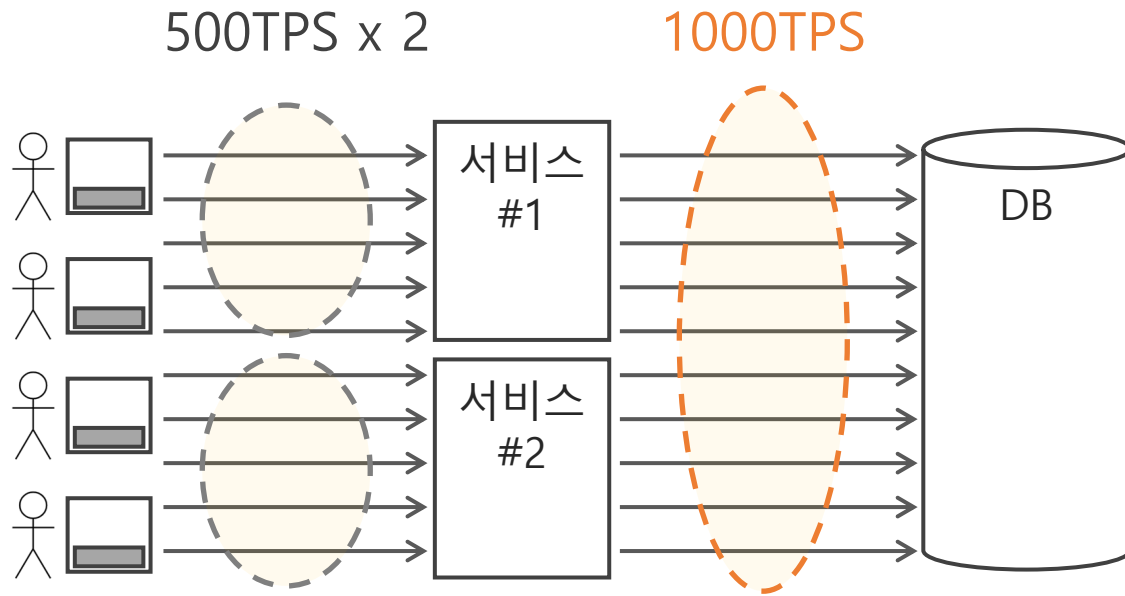
대기 현황 – After

③ 같은 데이터를 반복해서 조회
V. 변하지만, 매우 빨리 조회해



V. 변하지만, 매우 빨리 조회해 - tobe

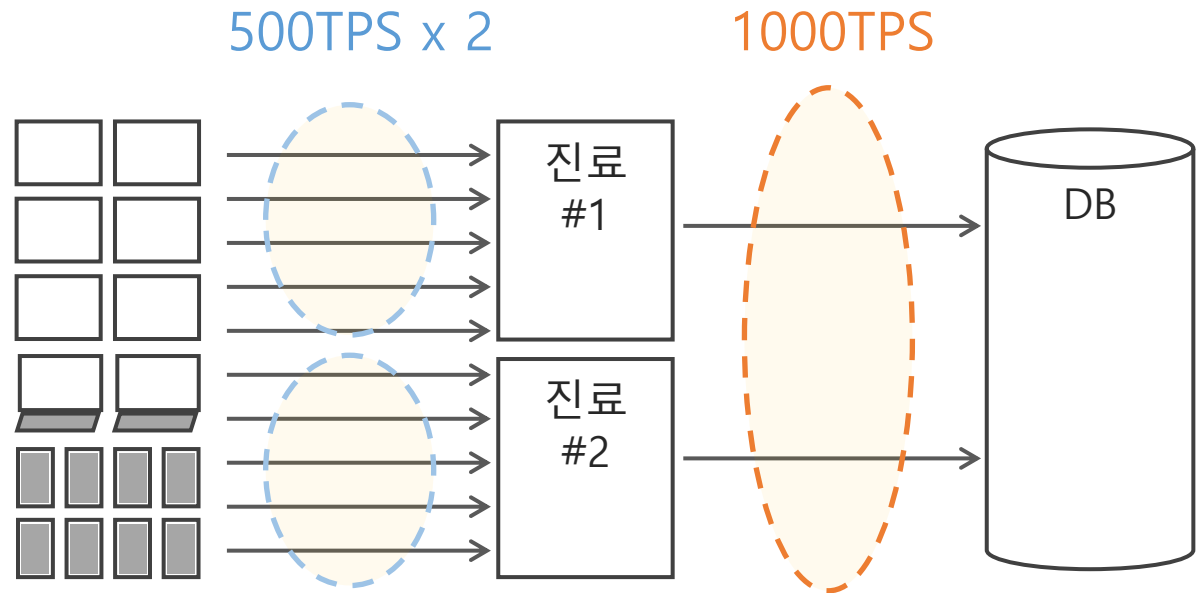
③ 같은 데이터를 반복해서 조회
V. 변하지만, 매우 빨리 조회해



V. 변하지만, 매우 빨리 조회해 (2)

③ 같은 데이터를 반복해서 조회
V. 변하지만, 매우 빨리 조회해

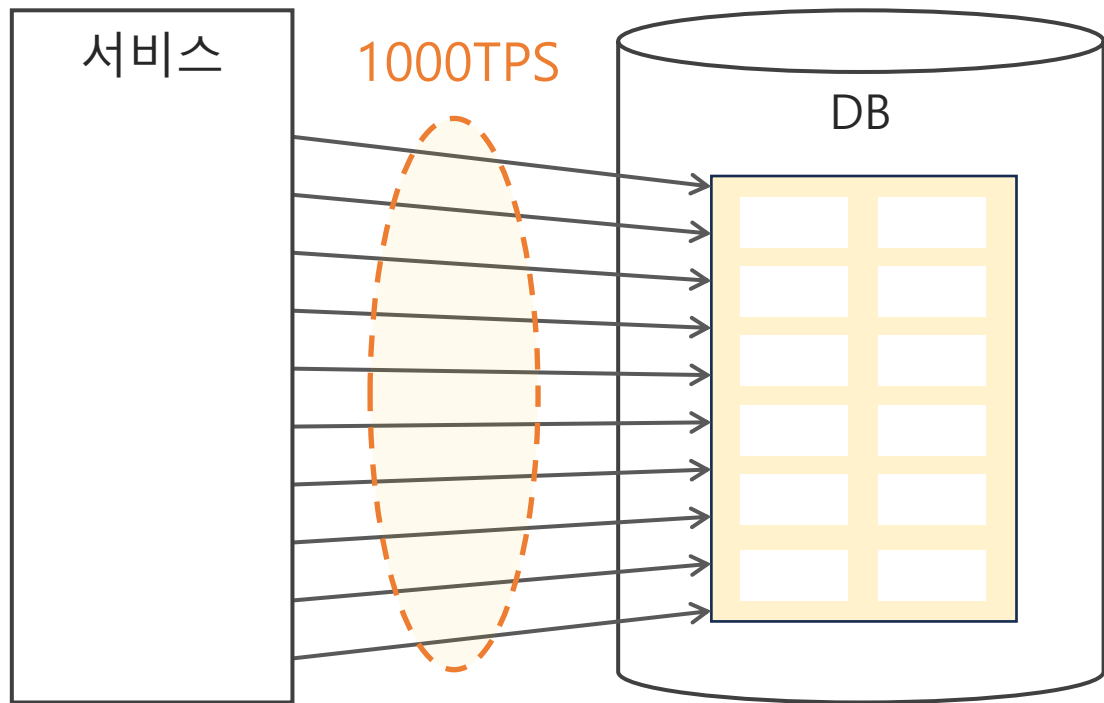
대기현황



1000TPS: 20만명 x 60% x 10회 / 1200초

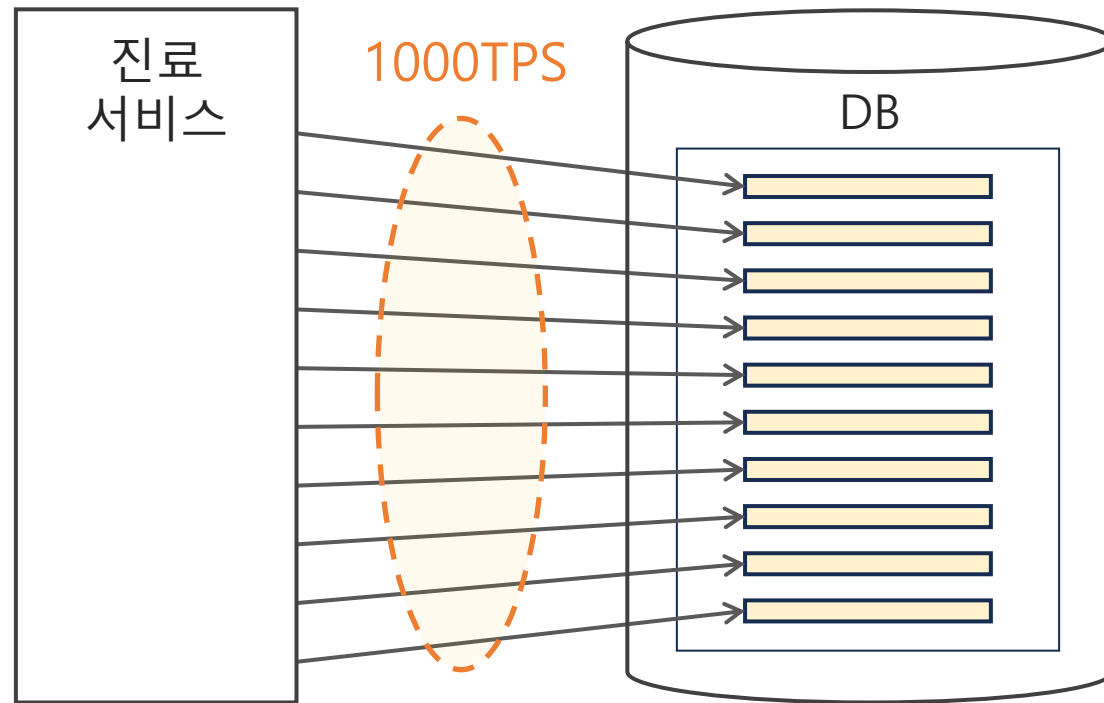
- 진료실: 100개
- 동시 진료 인원: 1000명
- 클라이언트 100대 (모니터 + PC + 모바일)
- 클라이언트 폴링 주기 : 1초

한 데이터를 빠르게 많이 조회
→ 로컬 캐시가 효과적



게시판

빠르게 많이 조회하지만
조회하는 데이터가 각각 달라
→ 로컬 캐시?



대기현황

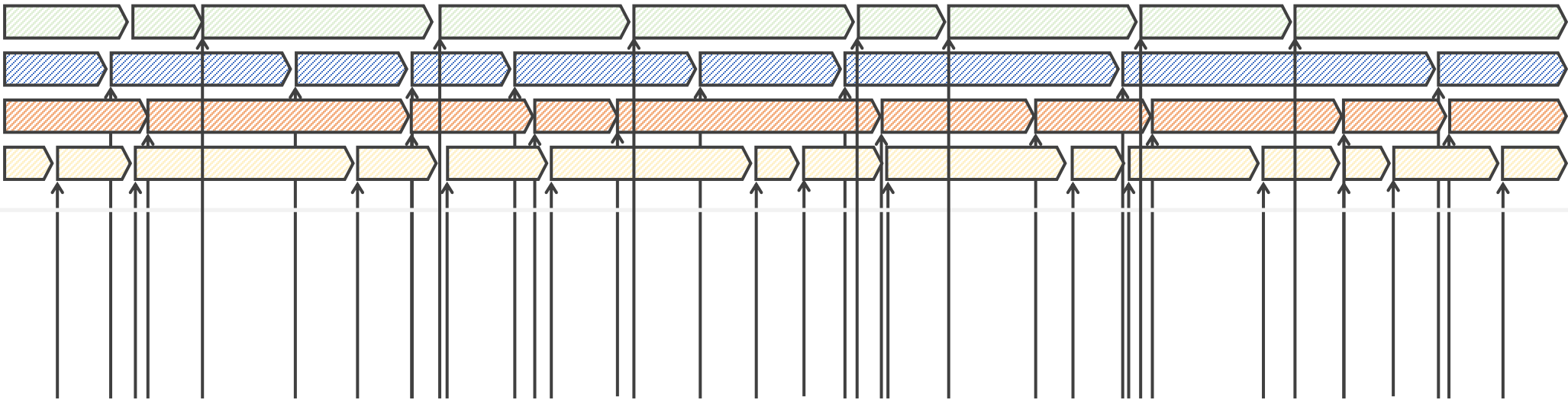
공지사항 – Before

③ 같은 데이터를 반복해서 조회
V. 변하지만, 매우 빨리 조회해

1초

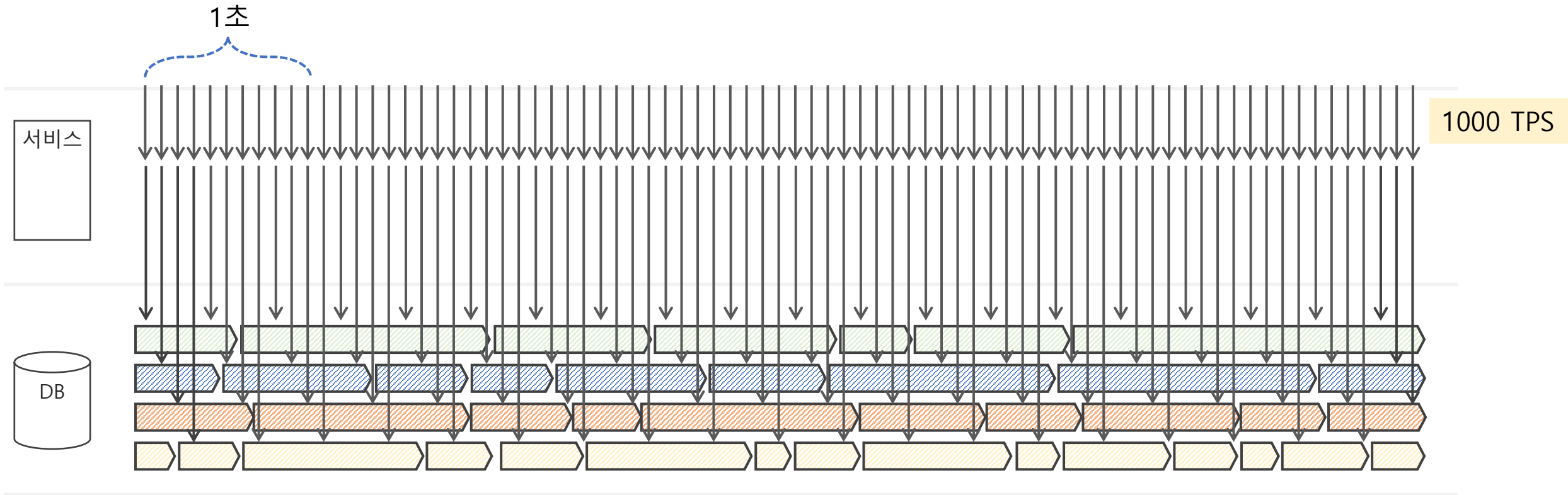
서비스

DB



공지사항 – Before

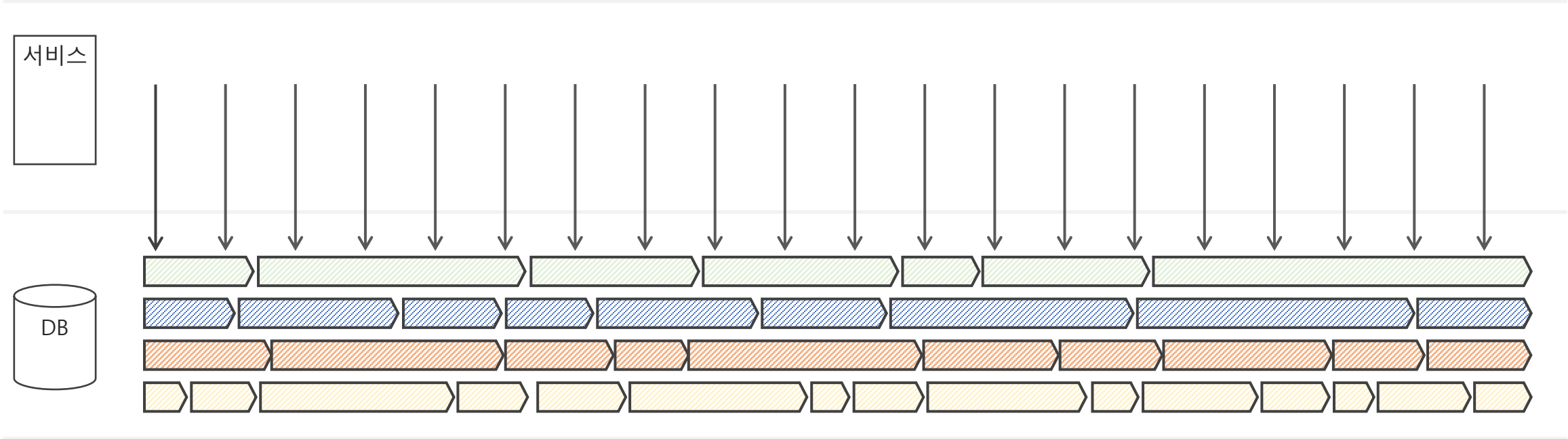
③ 같은 데이터를 반복해서 조회
V. 변하지만, 매우 빨리 조회해



공지사항 – Before

③ 같은 데이터를 반복해서 조회
V. 변하지만, 매우 빨리 조회해

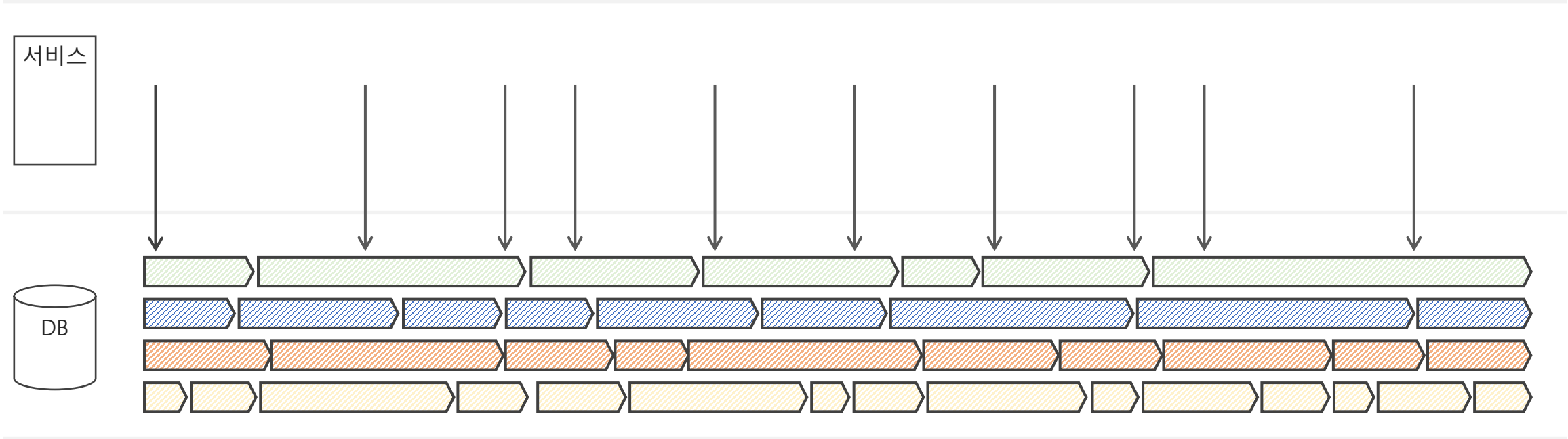
1초
인스턴스 2대 → 약 2.5 TPS



공지사항 – Before

③ 같은 데이터를 반복해서 조회
V. 변하지만, 매우 빨리 조회해

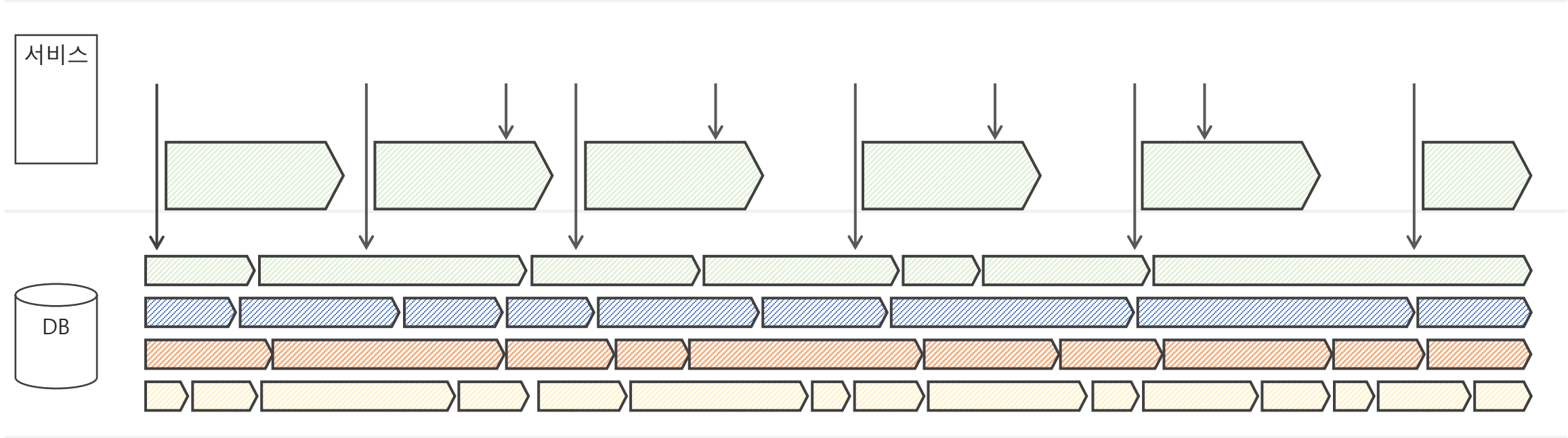
1초
인스턴스 1대 → 약 1.25 TPS ??

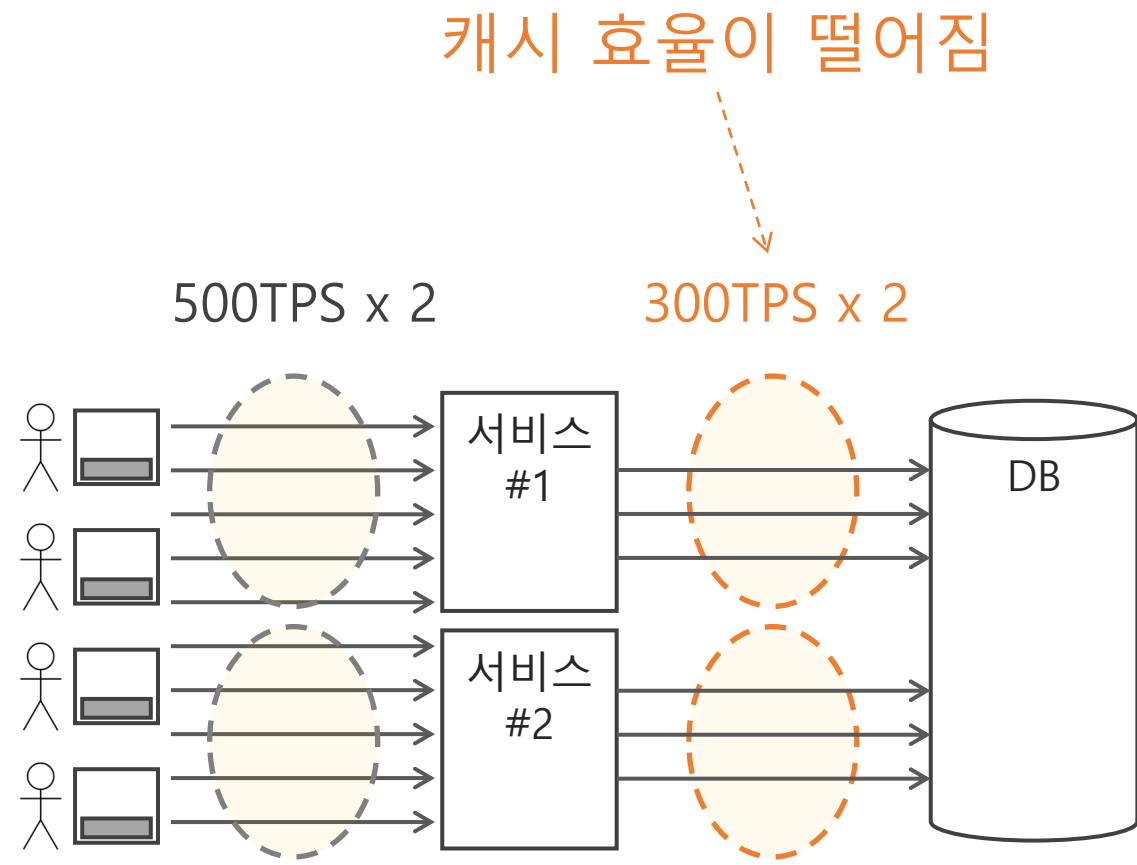
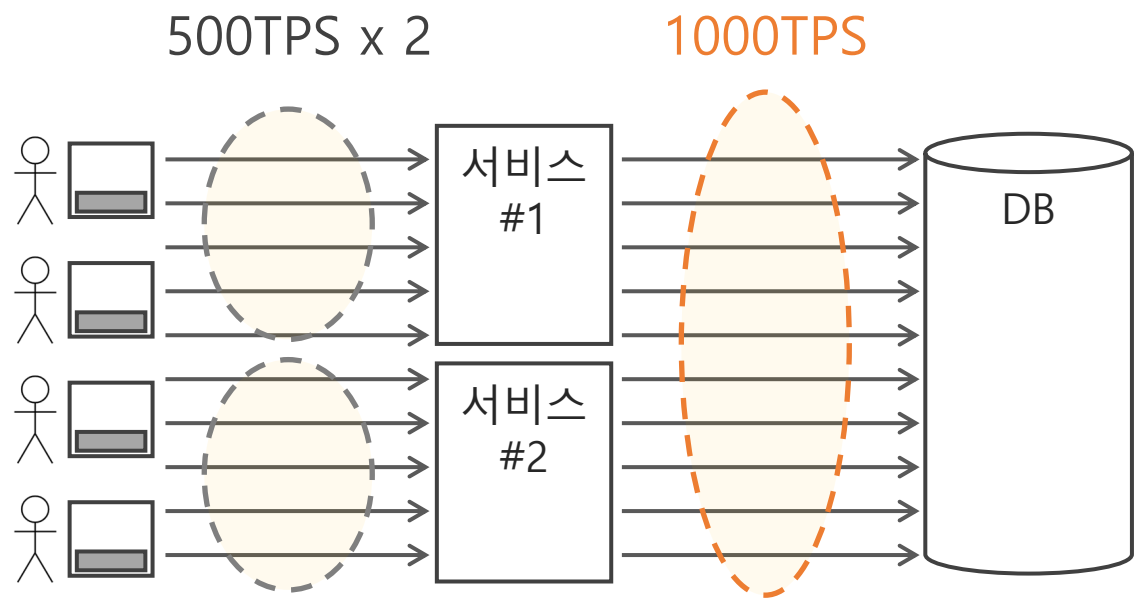


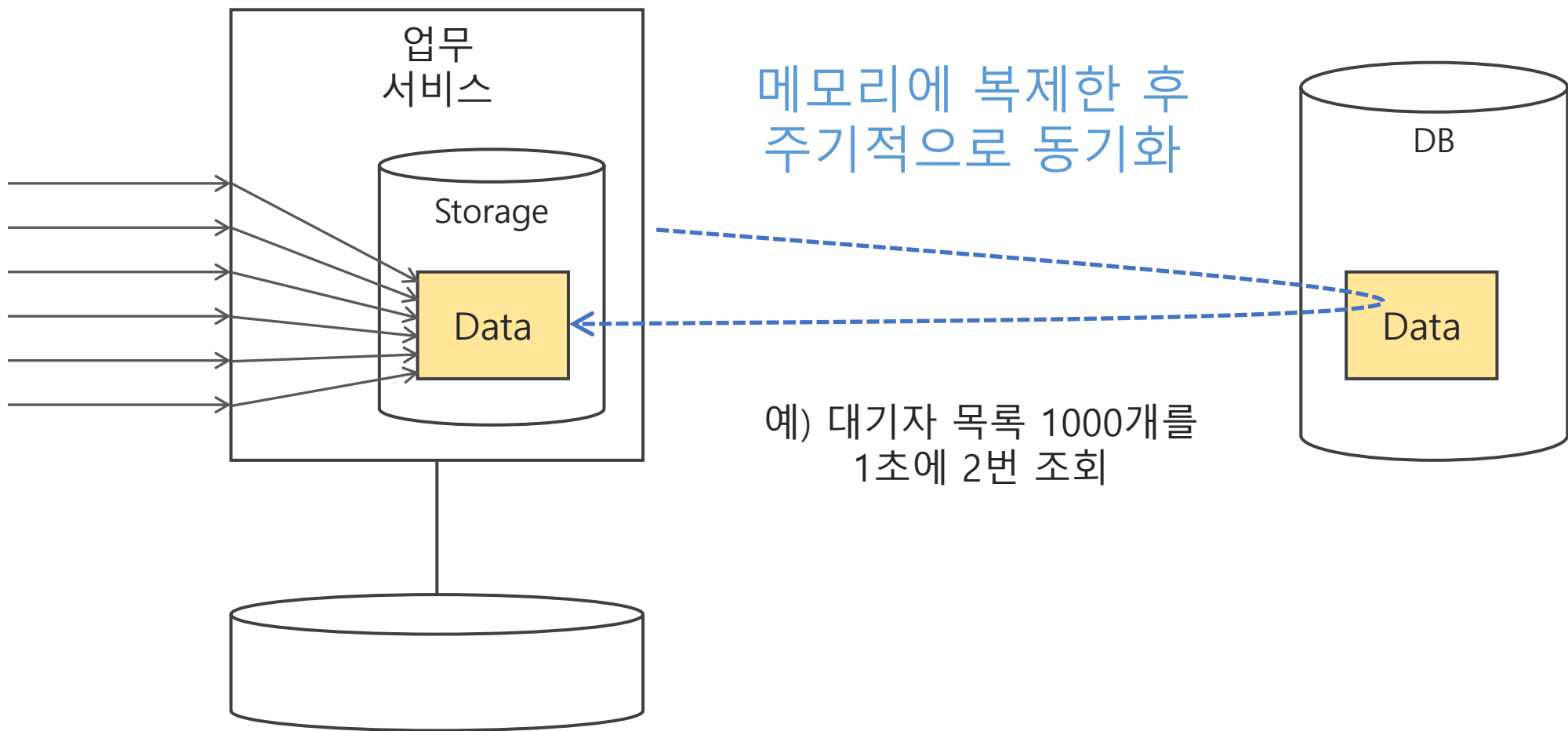
공지사항 – Before

③ 같은 데이터를 반복해서 조회
V. 변하지만, 매우 빨리 조회해

1초
인스턴스 x 2대 → 1/2 RPS

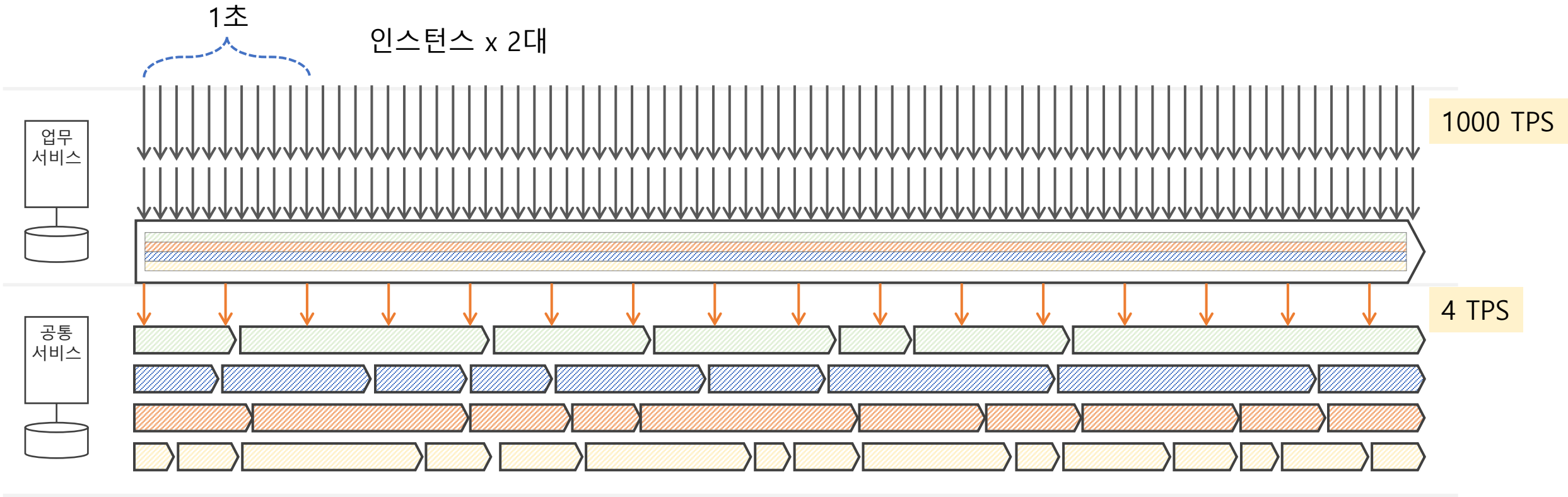


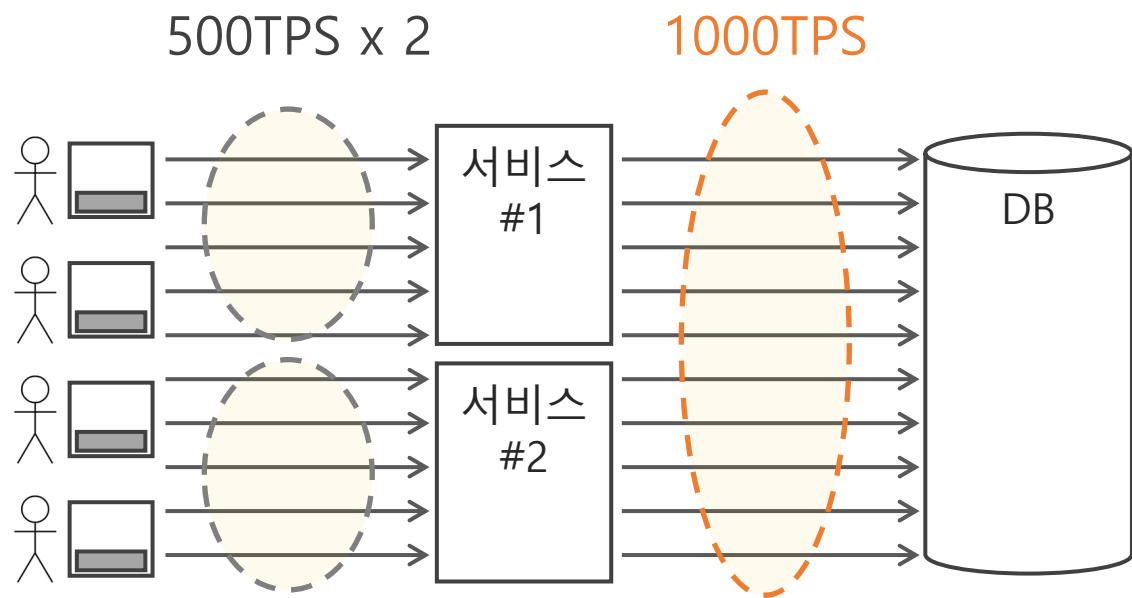




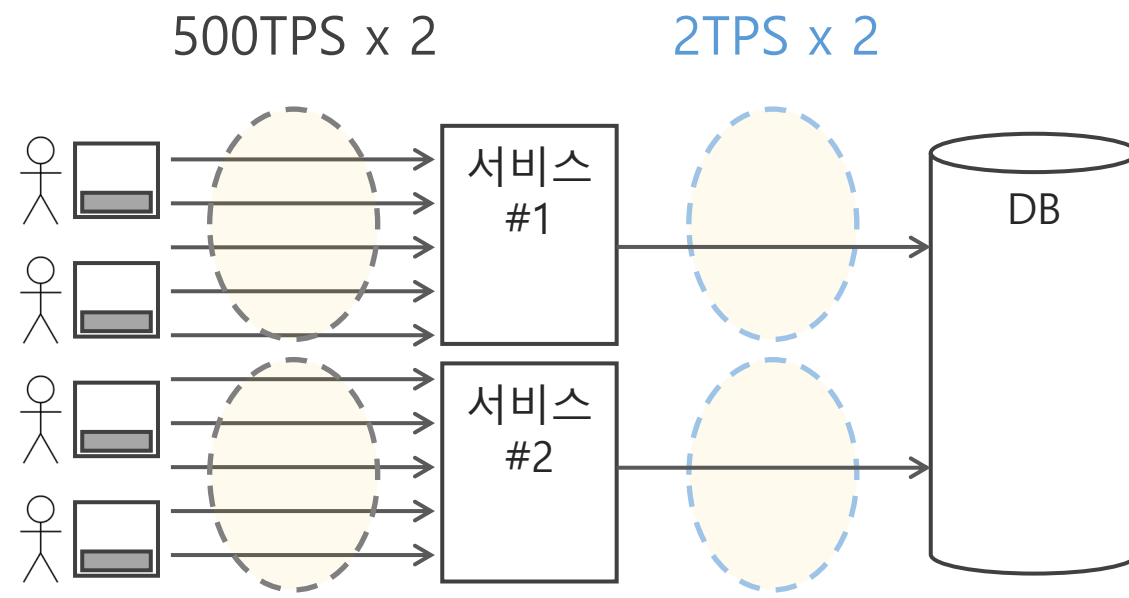
공지사항 – After

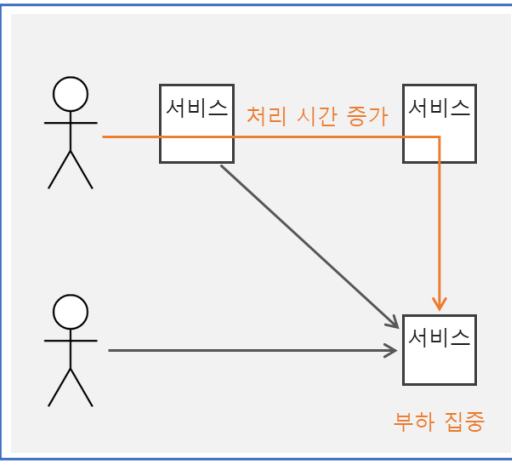
③ 같은 데이터를 반복해서 조회
V. 변하지만, 매우 빨리 조회해





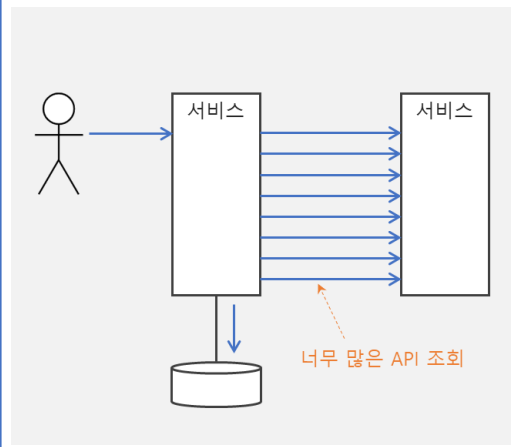
전 데이터를 2TPS로 조회





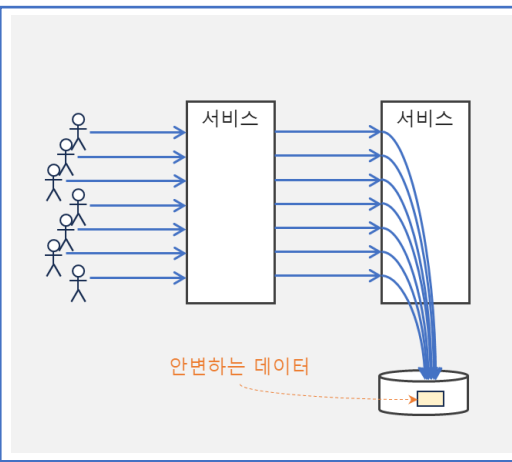
① 비효율적인 서비스 구조

- I. 기준 정보 서비스
- II. 마스터 정보 서비스
- III. 공통 업무 서비스



② 비효율적인 코드

- I. API N+1



③ 같은 데이터 반복 조회

- I. 안 변하는 데이터
- II. 실질적으로 안변해
- III. 변하지만, 늦게 봐도 괜찮아
- IV. 변하지만 패턴이 있어
- V. 변하지만, 매우 빨리 조회해

① 서비스 기능/데이터 재분배

② 데이터 복제 & 동기화

③ 코드 알고리즘 개선

④ 데이터 캐싱

감사합니다.